Contents

1 A tutorial introduction **3**

2 Types, Operators and Expressions **25**

3 Control Flow **40**

4 Functions and Program Structure **51**

5 Pointers and Arrays **113**

6 Structures **152**

7 Input and Output **174**

8 The UNIX System Interface **189**

Introduction

This textbook demonstrate my solutions for all exercises of the "The C Programming Language" book by Kernighan and Ritchie.
It is organized in the manner respective to the book's structure, having exercises for each chapter placed in separate one.
All sources were tested under platform with following parameters:

- Linux kernel 4.x (which is the modern UNIX kernel implementation, although is important only for chapter 8, where system calls are used instead of standart C library);
- GNU version of the C standart library;
- gcc compiler;
- bash shell.

For some examples not the full source is provided, but only the updated relatively to the former exercises parts.
Also, tabulation in listings is represented as 4 spaces, however, in original sources it had been presented as it is, equal to 8 spaces, and was changed to avoid longest lines leaving pages.

Chapter 1

A tutorial introduction

Exercise 1-1: Run the "hello, world" program on your system. Experiment with leaving out parts of the program, to see what error messages you get.

This program is enough simple and doesn't require any additional commentaries.

Exercise 1-2: Experiment to find out what happens when `printf`'s argument string contains \`c`, where `c` is some character not listed above.

This program is enough simple and doesn't require any additional commentaries.

Exercise 1-3: Modify the temperature conversion program to print the heading above the table.

```
/*
 * fahrenheit program
 */

#include <stdio.h>

#define LOW    0
#define HIGH    300
#define INC    10

int main(void)
{
    int fahr;

    /* header */
    printf("  F | C\n-----------\n");
    /* main work */
    for (fahr = LOW; fahr <= HIGH; fahr += INC)
        printf("%3d | %5.1f\n", fahr, (5.0 / 9) * (fahr - 32));
    return 0;
}
```

Exercise 1-4: Write a program to print the corresponding Celsius to Fahrenheit table.

```
/*
 * celcium program
```

```
 */

#include <stdio.h>

#define LOW    -30
#define HIGH    70
#define INC    10

int main(void)
{
    int c;

    /* header */
    printf("  C | F\n-----------\n");
    /* main work */
    for (c = LOW; c <= HIGH; c += INC)
        printf("%3d | %5.1f\n", c, 1.8 * c + 32);
    return 0;
}
```

Exercise 1-5: Modify the temperature conversion program to print the table in reverse order, that is, from 300 degrees to 0.

```
/*
 * fahrenheit program
 */

#include <stdio.h>

#define LOW    0
#define HIGH    300
#define INC    10

int main(void)
{
    int fahr;

    /* header */
    printf("  F | C\n-----------\n");
    /* main work */
    for (fahr = HIGH; fahr >= LOW; fahr -= INC)
        printf("%3d | %5.1f\n", fahr, (5.0 / 9) * (fahr - 32));
    return 0;
}
```

Exercise 1-6: Verify that the expression `getchar() != EOF` is 0 or 1.

This program is enough simple and doesn't require any additional commentaries.

Exercise 1-7: Write a program to print the value of `EOF`.

```
/*
```

```
 * EOF
 */
#include <stdio.h>

int main(void)
{
    printf("%d\n", EOF);
    return 0;
}
```

Exercise 1-8: Write a program to count blanks, tabs and newlines.

```
/*
 * count special symbols
 */

#include <stdio.h>

int main(void)
{
    int s, t, n;    /* interested */
    int c;        /* getchar */

    for (s = t = n = 0; (c = getchar()) != EOF; )
        switch (c) {
            case ' ': s++; break;
            case '\t': t++; break;
            case '\n': n++;
        }
    printf("' ': %d, '\\t': %d, '\\n': %d\n",
            s, t, n);
    return 0;
}
```

Exercise 1-9: Write a program to copy its input to its output, replacing each string of one or more blanks by a single blank.

```
/*
 * cutting blank lines
 */
#include <stdio.h>

int main(void)
{
    int c, nl, ns; /* new line marker, count of spaces */

    for (nl = 1; (c = getchar()) != EOF; nl = (c == '\n')) {
        if (nl) {
            /* we need count ' ', since may be "    s", need
             * output "    s" (not to lose 's')
             */
```

```
            for (ns = 0; c == ' '; c = getchar())
                ns++;
            /* full empty string -- the only case need to
             * place 1 space
             */
            if (c == '\n' && ns)
                ns = 1;
            while (ns--)
                putchar(' ');
            /* immediatly exit if EOF to avoid it's showing
             * with outside putchar() */
            if (c == EOF)
                return 0;
        }
        putchar(c);
    }
    return 0;
}
```

Exercise 1-10: Write a program to copy its input to its output, replacing each tab by \t, each backspace by \b, and each backslash by \\. This makes tabs and backspaces visible in unambiguous way.

```
/*
 * replacing the symbols
 */

#include <stdio.h>

int main(void)
{
    int c;

    while ((c = getchar()) != EOF)
        switch (c) {
            case '\t':
                printf("\\t");
                break;
            case '\b':
                printf("\\b");
                break;
            case '\\':
                printf("\\\\");
                break;
            default:
                putchar(c);
        }
    return 0;
}
```

Exercise 1-11: How would you test the word count program? What kinds of input are most likely to uncover bugs if there are any?

It's enough to give it a few thousand lines files with different types of content as input.

Exercise 1-12: Write a program that prints its input one word per line.

It's enough to place newline characters while first blank symbols is reached.

```
/*
 * print words one per line, ignores empty lines
 * (I think it's desirable)
 */

#include <stdio.h>

#define INW    1
#define OUTW    0

int main(void)
{
    int c, state;

    state = OUTW;
    while ((c = getchar()) != EOF) {
        if (c == ' ' || c == '\t' || c == '\n') {
            if (state == INW)
                putchar('\n');
            state = OUTW;
        } else {
            state = INW;
            putchar(c);
        }
    }
    return 0;
}
```

Exercise 1-13: Write a program to print a histogram of the lengths of words in its input. It is easy to draw the histogram with the bars horizontal; a vertical orientation is more challenging.

```
/*
 * word length histogram
 */

#include <stdio.h>

#define MAXLEN    80
#define FRQSCALE 24 /* scale to show on one line of screen */

/* inside/outside word */
```

```
#define INW    1
#define OUTW    0

int main(void)
{
    int freq[MAXLEN], c, cnt, state, maxfreq;
    double k;

    /* initialization */
    for (state = 0; state < MAXLEN; state++)
        freq[state] = 0;
    state = OUTW, cnt = 0;

    /* calculate length */
    while ((c = getchar()) != EOF)
        if (c == ' ' || c == '\t' || c == '\n') {
            if (state == INW) {
                freq[cnt - 1]++;
                cnt = 0;
            }
            state = OUTW;
        } else {
            state = INW;
            cnt++;
        }
    /* choose biggest frequency to have scale */
    for (maxfreq = cnt = 0; cnt < MAXLEN; cnt++)
        if (freq[cnt] > maxfreq)
            maxfreq = freq[cnt];
    /* show length with scale */
    k = (double) FRQSCALE / maxfreq;
    for (cnt = 0; cnt < MAXLEN; cnt++) {
        printf("%d: %d ", cnt + 1, freq[cnt]);
        freq[cnt] *= k;
        for (state = 0; state < freq[cnt]; state++)
            putchar('*');
        putchar('\n');
    }
    return 0;
}
```

A vertical orientation can be implemented by:

1. calculating the highest frequency after input is accomplished;
2. scaling all elements of frequency array to this value;
3. looping in reverse order with printing spaces or some predefined symbol if frequency of current element is respectively lower or not than current loop index.

```
/*
```

```
 * word length histogram
 */

#include <stdio.h>

#define MAXLEN    30
#define FRQSCALE 24 /* scale to show on one line of screen */

/* inside/outside word */
#define INW    1
#define OUTW    0

int main(void)
{
    int freq[MAXLEN], c, cnt, state, maxfreq;
    double k;

    /* initialization */
    for (state = 0; state < MAXLEN; state++)
        freq[state] = 0;
    state = OUTW, cnt = 0;

    /* calculate length */
    while ((c = getchar()) != EOF)
        if (c == ' ' || c == '\t' || c == '\n') {
            if (state == INW) {
                freq[cnt - 1]++;
                cnt = 0;
            }
            state = OUTW;
        } else {
            state = INW;
            cnt++;
        }
    /* choose biggest frequency to have scale */
    for (maxfreq = c = 0; c < MAXLEN; c++)
        if (freq[c] > maxfreq)
            maxfreq = freq[c];
    /* scale all */
    k = (double) FRQSCALE / maxfreq;
    for (c = 0; c < MAXLEN; c++)
        freq[c] *= k;
    /* show results */
    for (c = FRQSCALE; c > 0; c--) {
        printf("%d\t| ", c);
        for (cnt = 0; cnt < MAXLEN; cnt++)
            printf("%c ", (freq[cnt] < c) ? ' ' : '*');
        putchar('\n');
    }
    return 0;
```

```
}
```

Exercise 1-14: Write a program to print a histogram of the frequencies of different characters in its input.

```
/*
 * frequency historgram
 */

#include <stdio.h>
#include <stdlib.h>

/* basic counts */
#define NALPHA        26
#define NDIGIT         10

/* checking if belongs */
#define ISDIGIT(c)    ((c) >= '0' && (c) <= '9')
#define ISALPHAL(c)    ((c) >= 'a' && (c) <= 'z')
#define ISALPHAU(c)    ((c) >= 'A' && (c) <= 'Z')

int main(void)
{
    /* alphabet, digits, ' ', '\t', '\n' */
    int alpha_low[NALPHA], alpha_up[NALPHA],
        digits[NDIGIT], ns, nl, nt, unk;
    /* symbol to read */
    int c, i;

    /* clear all mem */
    for (c = 0; c < NALPHA; c++)
        alpha_low[c] = alpha_up[c] = 0;
    for (c = 0; c < NDIGIT; c++)
        digits[c] = 0;
    ns = nl = nt = unk = 0;

    /* count symbols */
    while ((c = getchar()) != EOF)
        if (ISDIGIT(c))
            digits[c - '0']++;
        else if (ISALPHAL(c))
            alpha_low[c - 'a']++;
        else if (ISALPHAU(c))
            alpha_up[c - 'A']++;
        else switch (c) {
            case ' ':    ns++;    break;
            case '\t':    nt++;    break;
            case '\n':    nl++;    break;
            default:    unk++;
        }

```

```
    /* create histogram and scale in parallel */
    /* a-z */
    for (c = 0; c < NALPHA; c++) {
        printf("%c: %d ", 'a' + c, alpha_low[c]);
        for (i = 0; i < alpha_low[c]; i++)
            putchar('*');
        putchar('\n');
    }
    /* A-Z */
    for (c = 0; c < NALPHA; c++) {
        printf("%c: %d ", 'A' + c, alpha_up[c]);
        for (i = 0; i < alpha_up[c]; i++)
            putchar('*');
        putchar('\n');
    }
    /* 0-9 */
    for (c = 0; c < NDIGIT; c++) {
        printf("%c: %d ", '0' + c, digits[c]);
        for (i = 0; i < digits[c]; i++)
            putchar('*');
        putchar('\n');
    }
    /* ' ', '\t', '\n' */
    printf(" : %d ", ns);
    for (i = 0; i < ns; i++)
        putchar('*');
    printf("\n\\t: %d ", nt);
    for (i = 0; i < nt; i++)
        putchar('*');
    printf("\n\\n: %d ", nl);
    for (i = 0; i < nl; i++)
        putchar('*');
    putchar('\n');
    return 0;
}
```

Exercise 1-15: Rewrite the temperature conversion program of Section 1.2 to use a function for conversion.

```
/*
 * fahrenheit program
 */

#include <stdio.h>

#define LOW    0
#define HIGH    300
#define INC    10

float f2c(int f)
{
```

```
    return (5.0f / 9) * (f - 32);
}

int main(void)
{
    int fahr;

    /* header */
    printf("  F | C\n-----------\n");
    /* main work */
    for (fahr = LOW; fahr <= HIGH; fahr += INC)
        printf("%3d | %5.1f\n", fahr, f2c(fahr));
    return 0;
}
```

Exercise 1-16: Revise the main routine of the longest line program so it will correctly print the length of arbitrarily long input lines, and as much as possible of the text.

This is reached just because of checking if the last symbol of line has been inputted is not equal to newline and continuing to read lines if so, with accumulating the length value. After this comparison between length recently got and the current maximal‘ length is performed, maximal length may be updated.
However, such approach doesn't help to output the longest line, only its length. To output also the line itself it must be stored in some additional buffer (and we don't know, if this line will be new longest, so we must prestore all buffers we got one after another, before the comparison with old maximal length), which itself must have its size, so we don't solve the problem fundamentally: there always will be some border for line size. However, most of practical cases don't require such functionality.

```
/*
 * finding the longest line
 */
#include <stdio.h>

#define MAXLINE        10

/*
 * read line from stdin to
 * buffer of lim size,
 * return length up to '\n'
 */
int getln(char *s, int lim)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (lim - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
```

```
        }
    *s = '\0';
    return s - b;
}

/* main work */
int main(void)
{
    char s[MAXLINE];
    int len, full_len, maxlen;

    maxlen = 0;
    while ((len = getln(s, MAXLINE)) > 0) {
        /* check for BIG string */
        full_len = 0;
        while (len == MAXLINE - 1 && s[len - 1] != '\n') {
            full_len += len;
            len = getln(s, MAXLINE);
        }
        full_len += len;
        /* choose max */
        if (full_len > maxlen)
            maxlen = full_len;
    }
    printf("maxlen = %d\n", maxlen);
    return 0;
}
```

Exercise 1-17: Write a program to print all input lines that are longer than 80 characters.

```
/*
 * output all lines with the length > 80
 */

#include <stdio.h>

#define MAXLINE     80

/*
 * get the line in buf[lim],
 * return its file length
 */
int getln(char *s, int lim)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (lim - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
```

```
            break;
        }
    *s = '\0';
    return s - b;
}

/* main work */
int main(int argc, char *argv[])
{
    char s[MAXLINE + 1];
    int l;

    /* while lines are present */
    while ((l = getln(s, MAXLINE + 1)) > 0)
        /* this line is too big:
         * | a | b | c | d | \0 |
         */
        if (l == MAXLINE && s[l - 1] != '\n') {
            printf("%s", s);
            /* show parts until:
             * - | a | b | \n | \0 | .. | or
             *   | a | b |  c | \n | \0 | -- EOL is found,
             *   then it's not important what's length
             * - | a | b | \0 | .. | .. | -- EOF is found,
             *   length is < lim - 1
             */
            do {
                l = getln(s, MAXLINE + 1);
                printf("%s", s);
            } while (s[l - 1] != '\n' && l == MAXLINE);
        }
    return 0;
}
```

Exercise 1-18: Write a program to remove trailing blanks and tabs from each line of input, and to delete entirely blank lines.

```
/*
 * truncate blank tails and empty lines
 */

#include <stdio.h>

#define MAXLINE    80

/* getline */
int getln(char *s, int max)
{
    register char *b;
    register int c;

```

```
    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';
    return s - b;
}

/*
 * trunc line:
 * " ab" -> " ab"
 * "ab " -> "ab"
 * "" -> (removed)
 * "abab  " -> "abab";
 *
 * line must be '\0'-terminated
 */
void trunc(char *s)
{
    register char *t;

    /* empty string */
    if (*s == '\n') {
        *s = '\0';
        return;
    }
    /* seek for tail */
    for (t = s; *t; t++) {
        /* potential blank tail: */
        if (*t == ' ' || *t == '\t') {
            for (s = t; *t != '\n' && *t != '\0'; t++)
                if (*t != ' ' && *t != '\t')
                    goto next; /* better than check outside */
            *s = *t;
            s[1] = '\0'; /* for the case *s == '\n' */
            return;
        }
    next:
        ;
    }
}

/* main work */
int main(void)
{
    char s[MAXLINE];

    while (getln(s, MAXLINE) > 0) {
```

```
        trunc(s);
        printf("%s", s);
    }
    return 0;
}
```

Exercise 1-19: Write a function reverse(s) that reverses the character string s. Use it to write a program that reverses its input a lint at a time.

```
/*
 * reverse input lines
 */

#include <stdio.h>

#define MAXLINE     80

/*
 * get line,
 * returns length up to '\n'
 */
int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';
    return s - b;
}

/*
 * reverse string with given
 * length (avoid recalc);
 * the end is '\0' or '\n'
 */
void srev(char *s, int l)
{
    register int i;
    register char c;

    /* check end to avoid reverse '\n' */
    if (s[l - 1] == '\n')
        l--;
    /* reverse */
    for (i = 0; i < (l >> 1); i++) {
```

```
        c = s[i];
        s[i] = s[l - i - 1];
        s[l - i - 1] = c;
    }
}

/* main work */
int main(void)
{
    char s[MAXLINE];
    register int l;

    while ((l = getln(s, MAXLINE)) > 0) {
        srev(s, l);
        printf("%s", s);
    }
    return 0;
}
```

Exercise 1-20: Write a program detab that replaces strings of blanks by the minimum number of tabs to space to the next tab stop. Assume a fixed set of tab stops, say evey n columns. Should n be a variable or a symbolic parameter?

```
/*
 * detab:
 * \t -> spaces
 */

#include <stdio.h>

#define TAB    8

int main(void)
{
    register int c, pos;

    for (pos = 0; (c = getchar()) != EOF; pos++)
        if (c == '\t') {
            /* reuse c */
            c = TAB - pos % TAB;    /* how many spaces need */
            pos += c - 1;        /* update to avoid i */
            while (c--)
                putchar(' ');
        } else {
            if (c == '\n')        /* update pos */
                pos = -1;
            putchar(c);
        }
    return 0;
}
```

Exercise 1-21: Write a program entab that replaces strings of blanks by the minimum number of tabs and blanks to achieve the same spacing. Use the same tab stops as for detab. When either a tab or a single blank would suffice to reach a tab stop, which should be given preference?

```
/*
 * entab;
 * replace ' '-only lines with '\t'
 * where possible
 */
#include <stdio.h>

#define MAXLINE    80
#define TAB    8

/* get line */
int getln(char *s, int max)
{
    register int c;
    register char *b;

    for (b = s; (s - b) < (max - 1) &&
        ((c = getchar()) != EOF); *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';
    return s - b;
}

/* check spaces */
int lnemp(const char *s)
{
    while (*s) {
        if (*s != ' ' && *s != '\n')
            return 0;
        s++;
    }
    return 1;
}

/* replace empty strings,
 * uses given line length
 */
int entab(char *s, int l)
{
    int tab, sp;
    char end;

    /* not count '\n' */
```

```
    end = s[l - 1];    /* preserve end */
    if (end == '\n')
        l--;
    /* calc */
    tab = l / TAB;
    sp = l % TAB;
    /* replace with tabs */
    while (tab--)
        *s++ = '\t';
    /* replace spaces by separating them
     * (they are already here)
     */
    s[sp] = end;
    /* preserved end */
    if (end == '\n')
        s[sp + 1] = '\0';
}

int main(void)
{
    register int c, l;
    char s[MAXLINE];

    while ((l = getln(s, MAXLINE)) > 0) {
        if (lnemp(s))
            entab(s, l);
        printf("%s", s);
    }
    return 0;
}
```

Exercise 1-22: Write a program to "fold" long input lines into two or more shorter lines after the last non-blank character that occurs before the n-th column of input. Make sure your program does something intelligent with very long lines, and if there are no blanks or tabs before the specified column.

```
/*
 * folding lines
 */

#include <stdio.h>

#define MAXLINE        10

/*
 * get line
 */
int getln(char *s, int max)
{
    register char *b;
    register int c;
```

```

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';
    return s - b;
}

/*
 * remove blank tail
 */
void trunc(char *s, int l)
{
    register int i;

    for (i = l - 1; i >= 0; i--)
        if (s[i] != ' ' && s[i] != '\t')
            break;
    /* cann't trunc -- fully blank */
    if (!i)
        return;
    /* trunc */
    s[i + 1] = '\n';
    s[i + 2] = '\0';
}

int main(void)
{
    char s[MAXLINE];
    register int l;

    /* read lines */
    while ((l = getln(s, MAXLINE)) > 0) {
        /* check if too big */
        if (l == MAXLINE - 1 && s[l - 1] != '\n')
            trunc(s, l);    /* rm blank tail */
        /* output */
        printf("%s", s);
    }
    return 0;
}
```

Exercise 1-23: Write a program to remove all comments from a C program. Don't forget to handle quoted strings and character constants properly. C comments do not nest.

```
/*
 * remove comments from the
 * C program
```

```
 */

#include <stdio.h>

#define MAXLINE        80

#define CHCKEOF(c)    {if((c) == EOF)return 0;}

/* main work */
int main(void)
{
    int br_small, br_big;    /* inside brackets */
    register int c;    /* read char */

    /*
     * init:
     * - outside of "", '';
     */
    br_small = br_big = 0;
    while ((c = getchar()) != EOF) {

        /* check '\' (may break brackets' checks
         * in cases like '\'' and '\"' (odd count))
         */
        if (c == '\\') {
            putchar(c);
            if (br_small) {
                c = getchar();    /* show without switching */
                CHCKEOF(c);
                putchar(c);
            }

        /* brackets check */
        } else if (c == '\'') {    /* '' */
            br_small ^= 1;        /* invert state */
            putchar(c);

        } else if (c == '\"') {    /* "" */
            br_big ^= 1;        /* invert state */
            putchar(c);

        /* (/ -> *) + outside == commentary */
        } else if (c == '/' && !(br_big + br_small)) {
            /* real commentary start */
            if ((c = getchar()) == '*') {
                do {
                    /* parse until end */
                    while ((c = getchar()) != '*')
                        CHCKEOF(c);
                    /* skip last *s */
```

```
                    while ((c = getchar()) == '*')
                        CHCKEOF(c);
                } while (c != '/');
            /* not commentary */
            } else {
                CHCKEOF(c);
                putchar(c);
            }

        /* other symbol */
        } else
            putchar(c);
    }
    printf("%d %d\n", br_small, br_big);
    return 0;
}
```

Exercise 1-24: Write a program to check a C program for rudimentary syntax errors like unbalanced parentheses, brackets and braces. Don't forget about quotes, both single and double, espace sequences, and comments. (This program is hard if you do it in full generality.)

```
/*
 * parse (), {}, []
 * in C-programs
 */
#include <stdio.h>

#define CHCKEOF(c)    {if((c) == EOF)return 0;}

/******************************
 * bracket stack,
 * is used for all types:
 * - parentheses; ()
 * - brackets; []
 * - figure br.; {}
 */

/* match state of stack */
#define OK    0    /* brackets match */
#define BAD    1    /* brackets don't match */

/* stack representation */
struct bst {
    int lev;    /* current stack depth */
    int state;    /* match state */
};
/******************************/

/* main work */
int main(void)
{
```

```
    register int c;
    int br_big, br_small;
    int p_st, b_st, f_st;

    /*
     * init: not in brackets,
     * 3 stacks zeroed
     */
    br_big = br_small = 0;
    p_st = b_st = f_st = 0;

    /* read and process */
    while ((c = getchar()) != EOF) {

        /*
         * (/ -> *) + outside == commentary:
         * ignore all
         */
        if (c == '/' && !(br_big + br_small)) {
            /* real commentary start */
            if ((c = getchar()) == '*') {
                do {
                    /* parse until end */
                    while ((c = getchar()) != '*')
                        CHCKEOF(c);
                    /* skip last *s */
                    while ((c = getchar()) == '*')
                        CHCKEOF(c);
                } while (c != '/');
            /* not commentary */
            } else
                CHCKEOF(c);

        /* check '\' (may break brackets' checks
         * in cases like '\'' and '\"' (odd count))
         */
        } else if (c == '\\') {
            if (br_small) {
                c = getchar();    /* skip without switching */
                CHCKEOF(c);
            }

        /* brackets check */
        } else if (c == '\'') {    /* '' */
            br_small ^= 1;        /* invert state */

        } else if (c == '\"') {    /* "" */
            br_big ^= 1;        /* invert state */

        /* finally, the main parser of brackets --
```

```
         * outside of comments and string literals */
        } else if (br_big + br_small == 0) {
            switch (c) {
                /* left */
                case '(': p_st++; break;
                case '[': b_st++; break;
                case '{': f_st++; break;
                /* right */
                case ')': p_st--; break;
                case ']': b_st--; break;
                case '}': f_st--;
            }
        }
    }

    /* check results */
    if (p_st | b_st | f_st)
        printf("mistakes detected\n");

    return 0;
}
```

Chapter 2

Types, Operators and Expressions

Exercise 2-1: Write a program to determine the ranges of `char`, `short`, `int`, and `long` variables, both `signed` and `unsigned`, by printing appropriate values from standart headers and by direct computation. Harder if you compute them: determine the ranges of the various floating-point types.

This can be solved just by writing a values into variables and incrementing them in loops until they reach lowest border after overflow.

Exercise 2-2: Write a loop equivalent to the for loop above without using `&&` or `||`.

This program is enough simple and doesn't require any additional commentaries.

Exercise 2-3: Write the function `htoi(s)`, which converts a string of hexadecimal digits (including an optional `0x` or `0X`) into its equivalent integer value. The allowable digits are 0 through 9, `a` through `f`, and `A` through `F`.

```
/*
 * hex string -> number
 */

#include <stdio.h>

/* line params */
#define MAXLINE        80

/* macros for hex */
#define isdigit(x)    ((x) >= '0' && (x) <= '9')
#define islalpha(x)    ((x) >= 'a' && (x) <= 'f')
#define isualpha(x)    ((x) >= 'A' && (x) <= 'F')

/* 16-digit -> 10-digit */
int x2d(int x)
{
    /* 0..9 */
    if (isdigit(x))
        return x - '0';
    /* a-f */
    if (islalpha(x))
        return x + 10 - 'a';
```

```
    /* A-F */
    if (isualpha(x))
        return x + 10 - 'A';
    /* some crap */
    return -1;
}

/* parse string and give num */
int htoi(const char *s)
{
    register int n, x;

    /* check prefix 0x or 0X */
    if (*s == '0' && (s[1] == 'x' || s[1] == 'X'))
        s += 2;
    /* parse */
    for (n = 0; *s; s++) {
        x = x2d(*s);    /* parse */
        if (x == -1)    /* incorrect */
            break;
        /* calc */
        n = (n << 4) + x;
    }
    return n;
}

/* get line */
int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

/*
 * main work:
 * 1. get line;
 * 2. transform it into hex;
 * 3. print it;
 */
int main(void)
```

```
{
    char s[MAXLINE];

    while (getln(s, MAXLINE) > 0)
        printf("= %d\n", htoi(s));
    return 0;
}
```

Exercise 2-4: Write an alternate version of `squeeze(s1,s2)` that deletes each character in `s1` that matches any character in the string `s2`.

```
/*
 * squeeze
 */

#include <stdio.h>

#define MAXLINE        80

/* check belonging, if s is empty --
 * no one belongs, even '\0' */
int bel(const char *s, char c)
{
    while (*s)
        if (*s++ == c)
            return 1;
    return 0;
}

/*
 * remove all symbols of s2
 * from s1;
 * O(N^2)
 */
void squeeze(char *s1, const char *s2)
{
    register int i, j;

    for (i = j = 0; s1[i]; i++)
        if (!(bel(s2, s1[i])))
            s1[j++] = s1[i];
    s1[j] = '\0';
}

/* getline */
int getln(char *s, int max)
{
    register int c;
    register char *b;

    for (b = s; (s - b) < (max - 1) &&
```

```
        (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';
    return s - b;
}

int main(void)
{
    char s1[MAXLINE], s2[MAXLINE];

    /* terminate the program with ^C or such */
    while (1) {
        getln(s1, MAXLINE);
        getln(s2, MAXLINE);
        squeeze(s1, s2);
        printf("> %s\n", s1);
    }
    return 0;
}
```

Exercise 2-5: Write the function `any(s1,s2)` which returns the first location in the string `s1` where any character from the string `s2` occurs, or -1 if `s1` contains no characters from `s2`. (The standart library function `strpbrk` does the same job but returns a pointer to the location.)

```
/*
 * any
 */

#include <stdio.h>

#define MAXLINE        80

/* check belonging, if s is empty --
 * no one belongs, even '\0' */
int bel(const char *s, char c)
{
    while (*s)
        if (*s++ == c)
            return 1;
    return 0;
}

/* getline,
 * but this time without '\n'
 * (to avoid clashing)
 */
int getln(char *s, int max)
{
```

```
    register int c;
    register char *b;

    for (b = s; (s - b) < (max - 1) &&
        (c = getchar()) != EOF && c != '\n'; *s++ = c)
        ;
    *s = '\0';
    return s - b;
}

/* any */
int any(const char *s1, const char *s2)
{
    const char *t;

    for (t = s1; *s1; s1++)
        if (bel(s2, *s1))
            return s1 - t;
    return -1;
}

int main(void)
{
    char s1[MAXLINE], s2[MAXLINE];
    int pos;

    /* terminate the program with ^C or such */
    while (1) {
        /* read strings */
        getln(s1, MAXLINE);
        getln(s2, MAXLINE);
        /* show pos */
        pos = any(s1, s2);
        if (pos == -1)
            printf("-\n");
        else
            printf("> %d\n", pos);
    }
    return 0;
}
```

Exercise 2-6: Write a function `setbits(x,p,n,y)` that returns `x` with the `n` bits that begin at position `p` set to the rightmost `n` bits of `y`, leaving the other bits unchanged.

```
/*
 * setbits
 */

#include <stdio.h>

/**************** SET BITS PART ****************/
```

```
/*
 * set bits:
 * y[0; n-1] -> x[p; p-n+1];
 */
unsigned setbits(unsigned x, int p, int n, unsigned y)
{
    /* move them in y */
    y <<= (p - n + 1);
    /* zero rest -> | */
    y &= ~(~0 << (p + 1));
    x |= y;
    /* one rest -> & */
    y |= (~(~0 << (p - n + 1)) | (~0 << (p + 1)));
    x &= y;
    return x;
}

/****************** STR -> NUM PART *************/

/* macros for hex */
#define isdigit(x)    ((x) >= '0' && (x) <= '9')
#define islalpha(x)    ((x) >= 'a' && (x) <= 'f')
#define isualpha(x)    ((x) >= 'A' && (x) <= 'F')

/* 16-digit -> 10-digit */
int x2d(int x)
{
    /* 0..9 */
    if (isdigit(x))
        return x - '0';
    /* a-f */
    if (islalpha(x))
        return x + 10 - 'a';
    /* A-F */
    if (isualpha(x))
        return x + 10 - 'A';
    /* some crap */
    return -1;
}

/* simple decimal fetch */
int atoi(const char *s)
{
    register int n;

    for (n = 0; *s && isdigit(*s); s++)
        n = 10 * n + *s - '0';
    return n;
}

```

```
/* hexademical fetch */
int htoi(const char *s)
{
    register int n, x;

    /* check prefix 0x or 0X */
    if (*s == '0' && (s[1] == 'x' || s[1] == 'X'))
        s += 2;
    /* parse */
    for (n = 0; *s; s++) {
        x = x2d(*s);    /* parse */
        if (x == -1)    /* incorrect */
            break;
        /* calc */
        n = (n << 4) + x;
    }
    return n;
}

/**************** GETLINE PART ***************/

#define MAXLINE        80

/* get line */
int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

/******************************/
/*
 * main work:
 * 1. read 2 strings -> transform to X and Y (hex);
 * 2. read 2 strings -> transform to p and n (decimal);
 * 3. calc X;
 * 4. print it;
 */
int main(void)
{
```

```
    char s[MAXLINE];
    unsigned x, y;
    int p, n;

    /* stop the program by ^C */
    while (1) {
        printf("x: ");        /* read x and y */
        getln(s, MAXLINE);
        x = htoi(s);
        printf("y: ");
        getln(s, MAXLINE);
        y = htoi(s);
        printf("p: ");        /* read p and n */
        getln(s, MAXLINE);
        p = atoi(s);
        printf("n: ");
        getln(s, MAXLINE);
        n = atoi(s);
        /* calc */
        x = setbits(x, p, n, y);
        printf("> %x\n", x);
    }
    return 0;
}
```

Exercise 2-7: Write a function `invert(x,p,n)` that returns `x` with the `n` bits that begin at position `p` inverted (i.e., 1 changed into 0 and vice versa), leaving the others unchanged.

```
/*
 * setbits
 */

#include <stdio.h>

/**************** INVERT BITS PART ****************/
/*
 * invert x[p; p-n+1]
 */
int invert(unsigned x, int p, int n)
{
    return x ^ (((1 << n) - 1) << (p - n + 1));
}

/****************** STR -> NUM PART *************/

/* macros for hex */
#define isdigit(x)    ((x) >= '0' && (x) <= '9')
#define islalpha(x)    ((x) >= 'a' && (x) <= 'f')
#define isualpha(x)    ((x) >= 'A' && (x) <= 'F')

/* 16-digit -> 10-digit */
```

```
int x2d(int x)
{
    /* 0..9 */
    if (isdigit(x))
        return x - '0';
    /* a-f */
    if (islalpha(x))
        return x + 10 - 'a';
    /* A-F */
    if (isualpha(x))
        return x + 10 - 'A';
    /* some crap */
    return -1;
}

/* simple decimal fetch */
int atoi(const char *s)
{
    register int n;

    for (n = 0; *s && isdigit(*s); s++)
        n = 10 * n + *s - '0';
    return n;
}

/* hexademical fetch */
int htoi(const char *s)
{
    register int n, x;

    /* check prefix 0x or 0X */
    if (*s == '0' && (s[1] == 'x' || s[1] == 'X'))
        s += 2;
    /* parse */
    for (n = 0; *s; s++) {
        x = x2d(*s);    /* parse */
        if (x == -1)    /* incorrect */
            break;
        /* calc */
        n = (n << 4) + x;
    }
    return n;
}

/***************** GETLINE PART ***************/

#define MAXLINE        80

/* get line */
int getln(char *s, int max)
```

```
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

/****************************/
/*
 * main work:
 * 1. read string -> transform to x (hex);
 * 2. read 2 strings -> transform to p and n (decimal);
 * 3. calc X;
 * 4. print it;
 */
int main(void)
{
    char s[MAXLINE];
    unsigned x;
    int p, n;

    /* stop the program by ^C */
    while (1) {
        printf("x: ");        /* read x */
        getln(s, MAXLINE);
        x = htoi(s);
        printf("p: ");        /* read p and n */
        getln(s, MAXLINE);
        p = atoi(s);
        printf("n: ");
        getln(s, MAXLINE);
        n = atoi(s);
        /* calc */
        x = invert(x, p, n);
        printf("> %x\n", x);
    }
    return 0;
}
```

Exercise 2-8: Write a function `rightrot(x,n)` that returns the value of the integer `x` rotated to the right by `n` bit positions.

```
/*
```

```
 * see rightrot(), the rest is additional
 */

#include <stdio.h>

/* macros for hex */
#define isdigit(x)    ((x) >= '0' && (x) <= '9')
#define islalpha(x)    ((x) >= 'a' && (x) <= 'f')
#define isualpha(x)    ((x) >= 'A' && (x) <= 'F')

/* 16-digit -> 10-digit */
int x2d(int x)
{
    /* 0..9 */
    if (isdigit(x))
        return x - '0';
    /* a-f */
    if (islalpha(x))
        return x + 10 - 'a';
    /* A-F */
    if (isualpha(x))
        return x + 10 - 'A';
    /* some crap */
    return -1;
}

/* parse string and give num */
int htoi(const char *s)
{
    register int n, x;

    /* check prefix 0x or 0X */
    if (*s == '0' && (s[1] == 'x' || s[1] == 'X'))
        s += 2;
    /* parse */
    for (n = 0; *s; s++) {
        x = x2d(*s);    /* parse */
        if (x == -1)    /* incorrect */
            break;
        /* calc */
        n = (n << 4) + x;
    }
    return n;
}

/* decimal parse */
int atoi(const char *s)
{
    register int n;

```

```
    for (n = 0; *s && isdigit(*s); s++)
        n = 10 * n + *s - '0';
    return n;
}

/* getline, as always */

#define MAXLINE     80

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';
    return s - b;
}

/*
 * bit count
 */
int bitcnt(unsigned x)
{
    register int c;

    for (c = 0, x |= ~0; x; x <<= 1, c++)
        ;
    return c;
}

/*
 * rotate argument
 * for n bit to right
 */
unsigned rightrot(unsigned x, int n)
{
    return (x << (bitcnt(x) - n)) | (x >> n);
}

/* main work */
int main(void)
{
    char s[MAXLINE];
    unsigned x;
```

```
    int n;

    /* ^C to terminate the program */
    while (1) {
        printf("x: ");        /* get num */
        getln(s, MAXLINE);
        x = htoi(s);
        printf("n: ");        /* get rotate count */
        getln(s, MAXLINE);
        n = atoi(s);
        /* process */
        x = rightrot(x, n);
        printf("> 0x%x\n\n", x);
    }
    return 0;
}
```

Exercise 2-9: In a two's complement number system, `x &= (x - 1)` deletes the rightmost 1-bit in `x`. Explain why. Use this observation to write a faster version of bitcount.

```
/*
 * see bitcount(), the rest is additional
 */

#include <stdio.h>

/* macros for hex */
#define isdigit(x)    ((x) >= '0' && (x) <= '9')
#define islalpha(x)    ((x) >= 'a' && (x) <= 'f')
#define isualpha(x)    ((x) >= 'A' && (x) <= 'F')

/* 16-digit -> 10-digit */
int x2d(int x)
{
    /* 0..9 */
    if (isdigit(x))
        return x - '0';
    /* a-f */
    if (islalpha(x))
        return x + 10 - 'a';
    /* A-F */
    if (isualpha(x))
        return x + 10 - 'A';
    /* some crap */
    return -1;
}

/* parse string and give num */
int htoi(const char *s)
{
    register int n, x;
```

```

    /* check prefix 0x or 0X */
    if (*s == '0' && (s[1] == 'x' || s[1] == 'X'))
        s += 2;
    /* parse */
    for (n = 0; *s; s++) {
        x = x2d(*s);    /* parse */
        if (x == -1)    /* incorrect */
            break;
        /* calc */
        n = (n << 4) + x;
    }
    return n;
}

/* getline, as always */

#define MAXLINE     80

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';
    return s - b;
}

/*
 * bit count
 */
int bitcount(unsigned x)
{
    register int n;

    for (n = 0; x; x &= (x - 1), n++)
        ;
    return n;
}

/* main work */
int main(void)
{
    char s[MAXLINE];
```

```
    unsigned x;

    /* ^C to terminate the program */
    while (1) {
        printf("x: ");        /* get num */
        getln(s, MAXLINE);
        x = htoi(s);
        /* process */
        printf("> %d\n\n", bitcount(x));
    }
    return 0;
}
```

Exercise 2-10: Rewrite the function `lower`, which converts upper case letters to lower case, with a conditional expression instead of `if-else`.

```
/*
 * lower with ternar
 * operator
 */

#include <stdio.h>

int lower(int c)
{
    return (c >= 'A' && c <= 'Z') ? c + 'a' - 'A' : c;
}

int main(void)
{
    register int c;

    while ((c = getchar()) != EOF)
        putchar(lower(c));
    return 0;
}
```

Chapter 3

Control Flow

Exercise 3-1: Our binary search makes two tests inside the loop, when one would suffice (at the price of more tests outside). Write a version with only one test inside the loop and measure the difference in run-time.

```
/*
 * testing 2 bsearch versions
 */

#include <stdio.h>
#include <stdlib.h>    /* rand() for tests */

/* classical bsearch */
int bsearch1(int *arr, int n, int x)
{
    register int l, m, h;

    l = 0, h = n - 1;
    while (l <= h) {
        m = (l + h) >> 1;
        if (arr[m] > x)    /* lower part */
            h = m - 1;
        else if (arr[m] < x)    /* upper part */
            l = m + 1;
        else    /* match */
            return m;
    }
    /* not found */
    return -1;
}

/*
 * updated version
 *
 * hint: it's the only correct,
 * version with if (arr[m] <= x) l = m;
 * etc. will loop forever where there are 2 el-s:
 *
 * 1. Initial:
```

```
 * | x | x |
 * ^   ^
 * l   h
 *
 * 2. Calc m:
 * m = (l + h) / 2 = 0;
 * | x | x |
 * ^   ^
 * l,m h
 *
 * 3. l = m -- the same, return to 1
 *
 * The reason is way of rounding.
 * So only if (arr[m] >= x) h = m; is correct way to do it.
 */
int bsearch2(int *arr, int n, int x)
{
    register int l, m, h;

    l = 0, h = n - 1;
    while (l < h) {    /* not strict for n == 1 case */
        m = (l + h) >> 1;
        if (arr[m] >= x) /* lower OR this */
            h = m;
        else        /* upper */
            l = m + 1;
    }
    /* check l or h, but not m (for 1 el.) */
    return (arr[l] == x) ? l : -1;
}

/* tests */
#define NARR(a)        (sizeof(a) / sizeof(a[0]))
#define SIZE        2000000

int icmp(const void *ia1, const void *ia2)
{
    return *(int *)ia1 - *(int *)ia2;
}

int main(void)
{
    int arr[SIZE], i;

    /* fill arr */
    for (i = 0; i < SIZE; i++)
        arr[i] = rand() % (SIZE * 3);

    /* sort it */
    qsort(arr, SIZE, sizeof(int), &icmp);
```

```

    /* try to find all el, change to bsearch2 here
     * to see the difference, on SIZE = 2000000
     * on Linux 4.4.39 on 2.8GHz x86 (32-bit) and 4Gb
     * it's ~108 ms */
    for (i = 0; i < SIZE * 3; i++)
        bsearch1(arr, SIZE, i);

    return 0;
}
```

Exercise 3-2: Write a function `escape(s,t)` that converts characters like newline and tab into visible escape sequences like `\n` and `\t` as it copies the string `t` to `s`. Use a switch. Write a function for the other direction as well, converting escape sequences into the real characters.

```
/*
 * escape(s, t) --
 * replace all \n from t with \\n
 */

#include <stdio.h>

#define MAXLINE        80

/* getline */
int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
        (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

/*
 * t -> s
 * with replacing of \n symbols
 */
void escape(char *s, const char *t)
{
    while (*t) {
        switch (*t) {
            case '\t':
                *s++ = '\\';
```

```
                *s++ = 't';
                break;
            case '\n':
                *s++ = '\\';
                *s++ = 'n';
                break;
            default:
                *s++ = *t;
        }
        t++;
    }
    /* finish string */
    *s = '\0';
}

/*
 * t -> s
 * with replacing to \n symbols
 */
void unescape(char *s, const char *t)
{
    while (*t) {
        if (*t == '\\') {
            t++;
            switch (*t) {
                case 'n': *s++ = '\n'; break;
                case 't': *s++ = '\t'; break;
                default:
                      *s++ = '\\';
                      *s++ = *t;
            }
        } else
            *s++ = *t;
        t++;
    }
    *s = '\0';
}

/* testing */
int main(void)
{
    char s[MAXLINE], t[MAXLINE];

    while (getln(t, MAXLINE) > 0) {
        unescape(s, t);
        printf("> %s", s);
    }
    return 0;
}
```

Exercise 3-3: Write a function `expand(s1,s2)` that expands shorthand notations like `a-z` in the string `s1` into the equivalent complete list `abc...xyz` in `s2`. Allow for letters of either case and digits, and be prepared to handle cases like `a-b-c` and `a-z0-9` and `-a-z`. Arrange that a leading or trailing `-` is taken literally.

```
/*
 * expand
 */

#include <stdio.h>

#define islalpha(c)    ((c) >= 'a' && (c) <= 'z')
#define isualpha(c)    ((c) >= 'A' && (c) <= 'Z')
#define isdigit(c)    ((c) >= '0' && (c) <= '9')

int range_ok(const char *s)
{
    return ((islalpha(*(s - 1)) && islalpha(*(s + 1))) ||
        (isualpha(*(s - 1)) && isualpha(*(s + 1))) ||
        (isdigit(*(s - 1)) && isdigit(*(s + 1)))) &&
            (*(s - 1) < *(s + 1));
}

/*
 * expand(s1, s2)
 */
void expand(const char *s1, char *s2)
{
    register int i, c;

    for (i = 0; s1[i]; i++) {
        /* it's easier to check '-' firstly */
        if (s1[i] == '-' && i && s1[i + 1]) {
            if (range_ok(&s1[i])) {    /* expand */
                for (c = s1[i - 1] + 1; c <= s1[i + 1]; c++)
                    *s2++ = c;
                i++;    /* skip '-' */
            } else
                *s2++ = '-';
        } else
            *s2++ = s1[i];
    }
    *s2 = '\0';
}

/* get line */

#define MAXLINE        80

int getln(char *s, int max)
```

```
{
    register int c;
    register const char *b;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

/* main work */
int main(void)
{
    char s1[MAXLINE], s2[MAXLINE * 10];

    /* stop the program with ^C */
    while (1) {
        printf("s1: ");
        getln(s1, MAXLINE);
        expand(s1, s2);
        printf("> %s", s2);
    }

    return 0;
}
```

Exercise 3-4: In a two's complement number representation, our version of itoa does not handle the largest negative number, that is, the value of `n` equal to $-2^{wordsize-1}$. Explain why not. Modify it to print that value correctly, regardless of the machine on which it runs.

```
/*
 * itoa
 */

#include <stdio.h>

int strlen(const char *s)
{
    register int n;

    for (n = 0; *s; s++)
        n++;
    return n;
}

void reverse(char *s)
```

```
{
    register int i;
    register char c;
    int l;

    for (l = strlen(s), i = 0; i < (l >> 1); i++) {
        c = s[i];
        s[i] = s[l-i-1];
        s[l-i-1] = c;
    }
}

void itoa(int n, char *s)
{
    int i, sign, t;

    if ((sign = n) < 0)
        n = -n;
    i = 0;
    do {
        t = n % 10;
        s[i++] = ((t > 0) ? t : -t) + '0';
    } while (n /= 10);
    if (sign < 0)
        s[i++] = '-';
    s[i] = '\0';
    reverse(s);
}

/* test */
int main(void)
{
    char s[100];
    int n;

    /* stop with ^C */
    while (1) {
        scanf("%d", &n);    /* num in */
        itoa(n, s);        /* calc */
        printf("> %s\n\n", s);    /* show */
    }

    return 0;
}
```

Exercise 3-5: Write the function `itob(s,n,b)` that converts the integer `n` into a base `b` character representation in the string `s`. In particular, `itob(n,s,16)` formats `n` as a hexademical integer in `s`.

```
/*
 * itob
 */
```

```

#include <stdio.h>

int strlen(const char *s)
{
    register int n;

    for (n = 0; *s; s++)
        n++;
    return n;
}

void reverse(char *s)
{
    register int i;
    register char c;
    int l;

    for (l = strlen(s), i = 0; i < (l >> 1); i++) {
        c = s[i];
        s[i] = s[l-i-1];
        s[l-i-1] = c;
    }
}

/* supports up to 26th numerical system (sic!) */
void itob(int n, char *s, int b)
{
    /* bases for digits, store them in data section instead of stack */
    static char bases[] = {
        '0', '1', '2', '3', '4', '5', '6', '7', '8', '9',
        'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j',
        'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't',
        'u', 'v', 'w', 'x', 'y', 'z'
    };
    int i, sign, t;

    if ((sign = n) < 0)    /* process sign */
        n = -n;
    i = 0;
    do {            /* choose right digit -- will be reversed */
        t = n % b;
        s[i++] = bases[t > 0 ? t : -t];
    } while (n /= b);
    if (sign < 0)        /* if negative, append - */
        s[i++] = '-';
    s[i] = '\0';
    reverse(s);
}

```

```
/* main work */
int main(void)
{
    int n, b;
    char s[100];

    /* stop with ^C */
    while (1) {
        printf("b: ");
        scanf("%d", &b);
        printf("n: ");
        scanf("%d", &n);
        itob(n, s, b);
        printf("> %s\n\n", s);
    }

    return 0;
}
```

Exercise 3-6: Write a version of `itoa` that accepts three arguments instead of two. The third argument is a minimum field width; the converted number must be padded with blanks on the left if necessary to make it wide enough.

```
/*
 * itoa with margin
 */

#include <stdio.h>

int strlen(const char *s)
{
    register int n;

    for (n = 0; *s; s++)
        n++;
    return n;
}

void reverse(char *s)
{
    register int i;
    register char c;
    int l;

    for (l = strlen(s), i = 0; i < (l >> 1); i++) {
        c = s[i];
        s[i] = s[l-i-1];
        s[l-i-1] = c;
    }
}

```

```
29  /* supports up to 26th numerical system (sic!) */
30  void itob(int n, char *s, int b, int m)
31  {
32      /* bases for digits, store them in data section instead of stack */
33      static char bases[] = {
34          '0', '1', '2', '3', '4', '5', '6', '7', '8', '9',
35          'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j',
36          'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't',
37          'u', 'v', 'w', 'x', 'y', 'z'
38      };
39      int i, sign, t;
40
41      if ((sign = n) < 0)    /* process sign */
42          n = -n;
43      i = 0;
44      do {            /* choose right digit -- will be reversed */
45          t = n % b;
46          s[i++] = bases[t > 0 ? t : -t];
47      } while (n /= b);
48      if (sign < 0)        /* if negative, append - */
49          s[i++] = '-';
50
51      /* check length and margin -- if too small,
52       * add spaces (will be reversed) */
53      t = i - m;        /* i is cur. length */
54      if (t < 0)
55          while (t++)
56              s[i++] = ' ';
57
58      /* finish and reverse */
59      s[i] = '\0';
60      reverse(s);
61  }
62
63  /* main work */
64  int main(void)
65  {
66      int n, b, m;
67      char s[100];
68
69      /* stop with ^C */
70      while (1) {
71          printf("b: ");        /* base */
72          scanf("%d", &b);
73          printf("n: ");        /* num */
74          scanf("%d", &n);
75          printf("m: ");        /* margin */
76          scanf("%d", &m);
77          itob(n, s, b, m);    /* process and out */
78          printf("> %s\n\n", s);
```

```
        }

        return 0;
    }
```

Chapter 4

Functions and Program Structure

Exercise 4-1: Write the function `strrindex(s,t)` which returns the position of the rightmost occurence of `t` in `s`, or -1 if there is none.

```
/*
 * strrindex(s,t);
 */

#include <stdio.h>

/* str length */
int strlen(const char *s)
{
    register int l;

    for (l = 0; *s; s++, l++)
        ;
    return l;
}

/* main core */
int strrindex(const char *s, const char *t)
{
    register int i, j, k, lt;

    /* check emptiness here to avoid loop checks */
    if (*t == '\0' || *s == '\0')
        return -1;

    /* main loop */
    lt = strlen(t) - 1;    /* remember for inner loop */
    for (i = strlen(s) - 1; i >= 0; i--) {
        /* start from the end of pattern, remember current */
        for (j = i, k = lt; k >= 0 && s[j] == t[k]; j--, k--)
            ;
        if (k == -1)    /* return START of pattern in main str */
            return i - lt;
    }
    return -1;
```

```
}

/* getline */

#define MAXLINE     80

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF && c != '\n'; *s++ = c)
        ;
    *s = '\0';

    return s - b;
}

/* test */
int main(void)
{
    char s[MAXLINE], t[MAXLINE];

    /* stop with ^C */
    while (1) {
        printf("s: ");
        getln(s, MAXLINE);
        printf("t: ");
        getln(t, MAXLINE);
        printf("> %d\n\n", strrindex(s, t));
    }

    return 0;
}
```

Exercise 4-2: Extend `atof` to handle scientific notation of the form `123.45e-6` where a floating-point number may be followed by `e` or `E` and an optionally signed exponent.

```
/*
 * atof with exp support
 */

#include <stdio.h>

#define isspace(c)    ((c) == ' ' || (c) == '\t')
#define isdigit(c)    ((c) >= '0' && (c) <= '9')

/* x^y */
int pow(int x, int y)
{
```

```
    register int p;

    for (p = 1; y--; p *= x)
        ;
    return p;
}

double atof(const char *s)
{
    double d, p;
    int sign, exp, exp_sign;

    /* skip spaces */
    while (isspace(*s))
        s++;

    /* check sign */
    sign = 1;
    if (*s == '-') {
        sign = -1;
        s++;
    } else if (*s == '+')
        s++;

    /* main loop */
    for (d = 0.0; isdigit(*s); s++)
        d = 10 * d + *s - '0';

    /* check . */
    p = 1.0;    /* here for correct end */
    if (*s == '.') {
        for (s++; isdigit(*s); s++) {
            d = 10 * d + *s - '0';
            p *= 10;
        }
    }

    /* check exp, independently of . */
    if (*s == 'e' || *s == 'E') {
        s++;

        /* get exponent sign */
        exp_sign = 1;
        if (*s == '-') {
            exp_sign = -1;
            s++;
        } else if (*s == '+')
            s++;

        /* accum exp (cann't call atoi -- since
```

```
         * cann't skip spaces etc. */
        for (exp = 0; isdigit(*s); s++)
            exp = 10 * exp + *s - '0';

        /* calc 10^exp */
        exp = pow(10, exp);
        d = (exp_sign == 1) ? (d * exp) : (d / exp);
    }

    /* res = f(d, sign, pow); */
    return sign * (double)d / p;
}

/* getline */

#define MAXLINE     80

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF && c != '\n'; *s++ = c)
        ;
    *s = '\0';

    return s - b;
}

/* test */
int main(void)
{
    char s[MAXLINE];

    while (getln(s, MAXLINE) > 0)
        printf("> %f\n", atof(s));

    return 0;
}

```

Exercise 4-3: Given the basic framework, it's straightforward to extend the calculator. Add the modulus (%) operator and provisions for negative numbers.

```
/*
 * calc
 * (if it's not exercise, I'll put into separate files);
 *
 * see "main skeleton" part for description
 */
```

```
7
8   #include <stdio.h>
9
10  /********************** declarations **********************/
11
12  /* common parsing */
13  int isdigit(int c);    /* shouldn't be macro: isdigit(c++) etc. will fail */
14  int isspace(int c);
15  double atof(const char *s);
16
17  /* stack */
18  #define STACK_MAX    100
19  void push(double d);
20  double pop(void);
21
22  /* process I/O stream */
23  #define BUF_MAX        100
24  int getch(void);
25  void ungetch(int c);
26
27  /* read next semantic sequence */
28  #define ARG_MAX        200
29  #define NUM        '0'    /* num type */
30  int getarg(char *s);
31
32  /********************** main skeleton **********************/
33  /*
34   * main algorithm is below
35   * (it doesn't use getline(), proceeding *semantical* constructs
36   * one by one instead):
37   *
38   * while ((new construct was read) not EOF):
39   * - if it's number -> place to stack;
40   * - if it's operation -> do it;
41   * - if it's \n -> get result from stack;
42   * - else -> error;
43   *
44   * calc output is after ">", error is after "!" (showing function sometimes)
45   */
46
47  int main(void)
48  {
49      char s[ARG_MAX];    /* arg sequence */
50      register int type;    /* arg type */
51      register int tmp;    /* temp for some ops */
52
53      while ((type = getarg(s)) != EOF) {
54          switch (type) {
55              case NUM:    /* number -> place to stack */
56                  push(atof(s));
```

```
                break;
            case '+':    /* a b + -> (a + b) */
                push(pop() + pop());
                break;
            case '-':    /* a b - -> (a - b), careful with order */
                tmp = pop();
                push(pop() - tmp);
                break;
            case '*':    /* a b * -> (a * b) */
                push(pop() * pop());
                break;
            case '/':    /* a b / -> (a / b) */
                tmp = pop();
                if (!tmp)    /* / 0 */
                    printf("! / 0\n");
                else
                    push(pop() / tmp);
                break;
            case '%':    /* a b % -> (a % b) */
                tmp = pop();
                push((int)pop() % (int)tmp);
                break;
            case '\n':    /* show result */
                printf("> %lf\n", pop());
                break;
            default:    /* some unexpected */
                printf("! \"%s\" is unknown\n", s);
        }
    }

    return 0;
}

/********************** stack of operands *****************/

double stack[STACK_MAX];
int sp = 0;        /* next FREE position */

void push(double d)
{
    if (sp < STACK_MAX)
        stack[sp++] = d;
    else
        printf("! push: stack is full\n");
}

double pop(void)
{
    if (sp > 0)
        return stack[--sp];
```

```
    printf("! pop: stack is empty\n");

    return 0.0;
}

/********************** I/O stream processing **************/

char io_buf[BUF_MAX];
int bp = 0;        /* next FREE position */

int getch(void)
{
    return (bp > 0) ? io_buf[--bp] : getchar();
}

void ungetch(int c)
{
    if (bp < BUF_MAX)
        io_buf[bp++] = c;
    else
        printf("! ungetch: io buf is full\n");
}

/********************** get semantic sequence *************/

int getarg(char *s)
{
    register int c, isfrac;

    /* skip empty chars */
    while (isspace(c = getch()))
        ;

    /* first non-empty -- check it:
     * it's ok for it to be:
     * - +-*%/ -- operations (after '-' must be not digit)
     * - 0-9 -- digit, also '-' can be before
     * - . -- fraction flag is set, next . will be ignored
     */

    if (!(isdigit(c) || c == '.' || c == '-'))
        return c;        /* first case -> ret op */

    /* not op */
    isfrac = 0;
    *s++ = c;    /* store first (even if it's not needed -- will be
                overwritten in next call with same s */

    /* special check for minus */
    if (c == '-') {
```

```
        c = getch();
        if (!isdigit(c)) {    /* operation, not negative num */
            ungetch(c);
            return '-';
        }
        /* if we here -- then negative num */
        *s++ = c;        /* add to stored - */

    /* also may be empty integer part */
    } else if (c == '.')
        isfrac = 1;

    /* accum all next digits */
    while (isdigit(c = getch()))
        *s++ = c;

    /* first non-digit is '.' and there wasn't '.' before --
     * accum fraction part */
    if (c == '.' && !isfrac) {
        *s++ = '.';
        while (isdigit(c = getch()))
            *s++ = c;
    }

    /* other case (either non-'.' or already had '.') --
     * return to buffer */
    *s = '\0';
    if (c != EOF)    /* buffer is chars, cann't store EOF */
        ungetch(c);

    return NUM;
}

/************************ common parsing ******************/

int isdigit(c)
{
    return c >= '0' && c <= '9';
}

int isspace(c)
{
    return c == ' ' || c == '\t';
}

int pow(int x, int y)
{
    register int p;

    for (p = 1; y--; p *= x)
```

```
        ;
    return p;
}

double atof(const char *s)
{
    double d, p;
    int sign, exp, exp_sign;

    /* skip spaces */
    while (isspace(*s))
        s++;

    /* check sign */
    sign = 1;
    if (*s == '-') {
        sign = -1;
        s++;
    } else if (*s == '+')
        s++;

    /* main loop */
    for (d = 0.0; isdigit(*s); s++)
        d = 10 * d + *s - '0';

    /* check . */
    p = 1.0;    /* here for correct end */
    if (*s == '.') {
        for (s++; isdigit(*s); s++) {
            d = 10 * d + *s - '0';
            p *= 10;
        }
    }

    /* check exp, independently of . */
    if (*s == 'e' || *s == 'E') {
        s++;

        /* get exponent sign */
        exp_sign = 1;
        if (*s == '-') {
            exp_sign = -1;
            s++;
        } else if (*s == '+')
            s++;

        /* accum exp (cann't call atoi -- since
         * cann't skip spaces etc. */
        for (exp = 0; isdigit(*s); s++)
            exp = 10 * exp + *s - '0';
```

```

        /* calc 10^exp */
        exp = pow(10, exp);
        d = (exp_sign == 1) ? (d * exp) : (d / exp);
    }

    /* res = f(d, sign, pow); */
    return sign * (double)d / p;
}

```

Exercise 4-4: Add commands to print the top element of the stack without popping, to duplicate it, and to swap the top two elements. Add a command to clear the stack.

```
/*
 * calc
 * (if it's not exercise, I'll put into separate files);
 *
 * see "main skeleton" part for description
 */

#include <stdio.h>

/********************** declarations **********************/

/* common parsing */
int isdigit(int c);    /* shouldn't be macro: isdigit(c++) etc. will fail */
int isspace(int c);
int islalpha(int c);
double atof(const char *s);

/* stack */
#define STACK_MAX    100
void push(double d);
double pop(void);
void clear_stack(void);

/* process I/O stream */
#define BUF_MAX        100
int getch(void);
void ungetch(int c);

/* read next semantic sequence */
#define ARG_MAX        200
#define NUM        '0'    /* num type */
#define CMD        '1'    /* command type */
int getarg(char *s);

/********************** main skeleton **********************/
/*
 * main algorithm is below
```

```
 * (it doesn't use getline(), proceeding *semantical* constructs
 * one by one instead):
 *
 * while ((new construct was read) not EOF):
 * - if it's number -> place to stack;
 * - if it's operation -> do it;
 * - if it's \n -> get result from stack;
 * - else -> error;
 *
 * calc output is after ">", error is after "!" (showing function sometimes)
 */

int main(void)
{
    char s[ARG_MAX];    /* arg sequence */
    register int type;    /* arg type */
    register double tmp, tmp2; /* temp for some ops */
    char *cmd_table[] = {
        "p",        /* print top of stack, not remove */
        "d",        /* dublicate top */
        "s",        /* swap two tops */
        "c"        /* clear all stack */
    };

    while ((type = getarg(s)) != EOF) {
        switch (type) {
            case CMD:    /* command -> find in table and execute */

                /* here is very stupid brute force,
                 * should be redone to binsearch, then calling
                 * a function pointer in a table */

                if (!strcmp(s, "p")) {        /* print top */
                    tmp = pop();
                    printf("> %lf\n", tmp);
                    push(tmp);
                } else if (!strcmp(s, "d")) {    /* dublicate top */
                    tmp = pop();
                    push(tmp);
                    push(tmp);
                } else if (!strcmp(s, "s")) {    /* swap two tops */
                    tmp = pop();
                    tmp2 = pop();
                    push(tmp);
                    push(tmp2);
                } else if (!strcmp(s, "c")) {    /* clear stack */
                    clear_stack();
                } else
                    printf("! unknown command %s\n", s);

```

```
                break;
            case NUM:    /* number -> place to stack */
                push(atof(s));
                break;
            case '+':    /* a b + -> (a + b) */
                push(pop() + pop());
                break;
            case '-':    /* a b - -> (a - b), careful with order */
                tmp = pop();
                push(pop() - tmp);
                break;
            case '*':    /* a b * -> (a * b) */
                push(pop() * pop());
                break;
            case '/':    /* a b / -> (a / b) */
                tmp = pop();
                if (!tmp)    /* / 0 */
                    printf("! / 0\n");
                else
                    push(pop() / tmp);
                break;
            case '%':    /* a b % -> (a % b) */
                tmp = pop();
                push((int)pop() % (int)tmp);
                break;
            case '\n':    /* show result */
                printf("> %lf\n", pop());
                break;
            default:    /* some unexpected */
                printf("! \"%s\" is unknown\n", s);
        }
    }

    return 0;
}

/********************** stack of operands ******************/

double stack[STACK_MAX];
int sp = 0;        /* next FREE position */

void push(double d)
{
    if (sp < STACK_MAX)
        stack[sp++] = d;
    else
        printf("! push: stack is full\n");
}

double pop(void)
```

```
{
    if (sp > 0)
        return stack[--sp];
    printf("! pop: stack is empty\n");

    return 0.0;
}

void clear_stack(void)
{
    sp = 0;
}

/********************** I/O stream processing **************/

char io_buf[BUF_MAX];
int bp = 0;        /* next FREE position */

int getch(void)
{
    return (bp > 0) ? io_buf[--bp] : getchar();
}

void ungetch(int c)
{
    if (bp < BUF_MAX)
        io_buf[bp++] = c;
    else
        printf("! ungetch: io buf is full\n");
}

/********************** get semantic sequence *************/

int getarg(char *s)
{
    register int c, isfrac;

    /* skip empty chars */
    while (isspace(c = getch()))
        ;

    /* first non-empty -- check it:
     * it's ok for it to be:
     * - +-*%/ -- operations (after '-' must be not digit)
     * - 0-9 -- digit, also '-' can be before
     * - . -- fraction flag is set, next . will be ignored
     * - a-z -- command start
     */

    if (!(isdigit(c) || c == '.' || c == '-' || islalpha(c)))
```

```
        return c;        /* first case -> ret op */

    /* check for command */
    if (islalpha(c)) {
        /* accum it, including current */
        do {
            *s++ = c;
        } while (islalpha(c = getch()));
        *s = '\0';

        /* return last to stream */
        if (c != EOF)
            ungetch(c);
        return CMD;        /* command type */
    }

    /* will be number */
    isfrac = 0;

    /* special check for minus */
    if (c == '-') {
        *s++ = c;
        c = getch();
        if (!isdigit(c)) {    /* operation, not negative num */
            if (c != EOF)
                ungetch(c);
            return '-';
        }
        /* if we here -- then negative num */
        *s++ = c;        /* add to stored - */

    /* also may be empty integer part */
    } else if (c == '.') {
        *s++ = '.';
        isfrac = 1;
    } else        /* digit */
        *s++ = c;

    /* accum all next digits */
    while (isdigit(c = getch()))
        *s++ = c;

    /* first non-digit is '.' and there wasn't '.' before --
     * accum fraction part */
    if (c == '.' && !isfrac) {
        *s++ = '.';
        while (isdigit(c = getch()))
            *s++ = c;
    }

```

```
    /* other case (either non-'.' or already had '.') --
     * return to buffer */
    *s = '\0';
    if (c != EOF)    /* buffer is chars, cann't store EOF */
        ungetch(c);

    return NUM;
}

/************************ common parsing ******************/

int isdigit(c)
{
    return c >= '0' && c <= '9';
}

int isspace(c)
{
    return c == ' ' || c == '\t';
}

int islalpha(c)
{
    return c >= 'a' && c <= 'z';
}

int pow(int x, int y)
{
    register int p;

    for (p = 1; y--; p *= x)
        ;
    return p;
}

double atof(const char *s)
{
    double d, p;
    int sign, exp, exp_sign;

    /* skip spaces */
    while (isspace(*s))
        s++;

    /* check sign */
    sign = 1;
    if (*s == '-') {
        sign = -1;
        s++;
    } else if (*s == '+')
```

```
        s++;

    /* main loop */
    for (d = 0.0; isdigit(*s); s++)
        d = 10 * d + *s - '0';

    /* check . */
    p = 1.0;    /* here for correct end */
    if (*s == '.') {
        for (s++; isdigit(*s); s++) {
            d = 10 * d + *s - '0';
            p *= 10;
        }
    }

    /* check exp, independently of . */
    if (*s == 'e' || *s == 'E') {
        s++;

        /* get exponent sign */
        exp_sign = 1;
        if (*s == '-') {
            exp_sign = -1;
            s++;
        } else if (*s == '+')
            s++;

        /* accum exp (cann't call atoi -- since
         * cann't skip spaces etc. */
        for (exp = 0; isdigit(*s); s++)
            exp = 10 * exp + *s - '0';

        /* calc 10^exp */
        exp = pow(10, exp);
        d = (exp_sign == 1) ? (d * exp) : (d / exp);
    }

    /* res = f(d, sign, pow); */
    return sign * (double)d / p;
}

/********************** string routines ************************/

int strcmp(const char *s0, const char *s1)
{
    for (; *s0 == *s1; s0++, s1++)
        if (*s0 == '\0')
            return 0;
    return *s0 - *s1;
}
```

Exercise 4-5: Add access to library functions like `sin`, `exp`, and `pow`.

```
/*
 * calc
 * (if it's not exercise, I'll put into separate files);
 *
 * see "main skeleton" part for description
 * (it's math version, for gcc to compile and link use
 * $ cc <file>.c -lm
 */

#include <stdio.h>
#include <math.h>

/********************** declarations **********************/

/* common parsing */
int isdigit(int c);    /* shouldn't be macro: isdigit(c++) etc. will fail */
int isspace(int c);
int islalpha(int c);
double atof(const char *s);

/* stack */
#define STACK_MAX    100
void push(double d);
double pop(void);
void clear_stack(void);

/* process I/O stream */
#define BUF_MAX        100
int getch(void);
void ungetch(int c);

/* read next semantic sequence */
#define ARG_MAX        200
#define NUM        '0'    /* num type */
#define CMD        '1'    /* command type */
int getarg(char *s);

/********************** main skeleton **********************/
/*
 * main algorithm is below
 * (it doesn't use getline(), proceeding *semantical* constructs
 * one by one instead):
 *
 * while ((new construct was read) not EOF):
 * - if it's number -> place to stack;
 * - if it's operation -> do it;
 * - if it's \n -> get result from stack;
```

```
 * - else -> error;
 *
 * calc output is after ">", error is after "!" (showing function sometimes)
 */

int main(void)
{
    char s[ARG_MAX];    /* arg sequence */
    register int type;    /* arg type */
    register double tmp, tmp2; /* temp for some ops */
    char *cmd_table[] = {
        "p",        /* print top of stack, not remove */
        "d",        /* dublicate top */
        "s",        /* swap two tops */
        "c",        /* clear all stack */
        "sin",        /* sin(x) */
        "cos",        /* cos(x) */
        "exp",        /* exp(x) */
        "pow"        /* pow(x,y) */
    };

    while ((type = getarg(s)) != EOF) {
        switch (type) {
            case CMD:    /* command -> find in table and execute */

                /* here is very stupid brute force,
                 * should be redone to binsearch, then calling
                 * a function pointer in a table */

                if (!strcmp(s, "p")) {        /* print top */
                    tmp = pop();
                    printf("> %lf\n", tmp);
                    push(tmp);
                } else if (!strcmp(s, "d")) {    /* dublicate top */
                    tmp = pop();
                    push(tmp);
                    push(tmp);
                } else if (!strcmp(s, "s")) {    /* swap two tops */
                    tmp = pop();
                    tmp2 = pop();
                    push(tmp);
                    push(tmp2);
                } else if (!strcmp(s, "c")) /* clear stack */
                    clear_stack();
                else if (!strcmp(s, "sin")) /* sin(x) */
                    push(sin(pop()));
                else if (!strcmp(s, "cos")) /* cos(x) */
                    push(cos(pop()));
                else if (!strcmp(s, "exp")) /* exp(x) */
                    push(exp(pop()));
```

```
                else if (!strcmp(s, "pow")) { /* pow(x,y) */
                    tmp = pop();
                    push(pow(pop(), tmp));
                } else
                    printf("! unknown command %s\n", s);
                break;
            case NUM:    /* number -> place to stack */
                push(atof(s));
                break;
            case '+':    /* a b + -> (a + b) */
                push(pop() + pop());
                break;
            case '-':    /* a b - -> (a - b), careful with order */
                tmp = pop();
                push(pop() - tmp);
                break;
            case '*':    /* a b * -> (a * b) */
                push(pop() * pop());
                break;
            case '/':    /* a b / -> (a / b) */
                tmp = pop();
                if (!tmp)    /* / 0 */
                    printf("! / 0\n");
                else
                    push(pop() / tmp);
                break;
            case '%':    /* a b % -> (a % b) */
                tmp = pop();
                push((int)pop() % (int)tmp);
                break;
            case '\n':    /* show result */
                printf("> %lf\n", pop());
                break;
            default:    /* some unexpected */
                printf("! \"%s\" is unknown\n", s);
        }
    }

    return 0;
}

/********************** stack of operands *****************/

double stack[STACK_MAX];
int sp = 0;        /* next FREE position */

void push(double d)
{
    if (sp < STACK_MAX)
        stack[sp++] = d;
```

```
    else
        printf("! push: stack is full\n");
}

double pop(void)
{
    if (sp > 0)
        return stack[--sp];
    printf("! pop: stack is empty\n");

    return 0.0;
}

void clear_stack(void)
{
    sp = 0;
}

/********************** I/O stream processing **************/

char io_buf[BUF_MAX];
int bp = 0;        /* next FREE position */

int getch(void)
{
    return (bp > 0) ? io_buf[--bp] : getchar();
}

void ungetch(int c)
{
    if (bp < BUF_MAX)
        io_buf[bp++] = c;
    else
        printf("! ungetch: io buf is full\n");
}

/********************** get semantic sequence *************/

int getarg(char *s)
{
    register int c, isfrac;

    /* skip empty chars */
    while (isspace(c = getch()))
        ;

    /* first non-empty -- check it:
     * it's ok for it to be:
     * - +-*%/ -- operations (after '-' must be not digit)
     * - 0-9 -- digit, also '-' can be before
```

```
 * - . -- fraction flag is set, next . will be ignored
 * - a-z -- command start
 */

if (!(isdigit(c) || c == '.' || c == '-' || islalpha(c)))
    return c;        /* first case -> ret op */

/* check for command */
if (islalpha(c)) {
    /* accum it, including current */
    do {
        *s++ = c;
    } while (islalpha(c = getch()));
    *s = '\0';

    /* return last to stream */
    if (c != EOF)
        ungetch(c);
    return CMD;        /* command type */
}

/* will be number */
isfrac = 0;

/* special check for minus */
if (c == '-') {
    *s++ = c;
    c = getch();
    if (!isdigit(c)) {    /* operation, not negative num */
        if (c != EOF)
            ungetch(c);
        return '-';
    }
    /* if we here -- then negative num */
    *s++ = c;        /* add to stored - */

/* also may be empty integer part */
} else if (c == '.') {
    *s++ = '.';
    isfrac = 1;
} else        /* digit */
    *s++ = c;

/* accum all next digits */
while (isdigit(c = getch()))
    *s++ = c;

/* first non-digit is '.' and there wasn't '.' before --
 * accum fraction part */
if (c == '.' && !isfrac) {
```

```
        *s++ = '.';
        while (isdigit(c = getch()))
            *s++ = c;
    }

    /* other case (either non-'.' or already had '.') --
     * return to buffer */
    *s = '\0';
    if (c != EOF)    /* buffer is chars, cann't store EOF */
        ungetch(c);

    return NUM;
}

/************************ common parsing ******************/

int isdigit(c)
{
    return c >= '0' && c <= '9';
}

int isspace(c)
{
    return c == ' ' || c == '\t';
}

int islalpha(c)
{
    return c >= 'a' && c <= 'z';
}

double atof(const char *s)
{
    double d, p;
    int sign, exp, exp_sign;

    /* skip spaces */
    while (isspace(*s))
        s++;

    /* check sign */
    sign = 1;
    if (*s == '-') {
        sign = -1;
        s++;
    } else if (*s == '+')
        s++;

    /* main loop */
    for (d = 0.0; isdigit(*s); s++)
```

```
        d = 10 * d + *s - '0';

    /* check . */
    p = 1.0;    /* here for correct end */
    if (*s == '.') {
        for (s++; isdigit(*s); s++) {
            d = 10 * d + *s - '0';
            p *= 10;
        }
    }

    /* check exp, independently of . */
    if (*s == 'e' || *s == 'E') {
        s++;

        /* get exponent sign */
        exp_sign = 1;
        if (*s == '-') {
            exp_sign = -1;
            s++;
        } else if (*s == '+')
            s++;

        /* accum exp (cann't call atoi -- since
         * cann't skip spaces etc. */
        for (exp = 0; isdigit(*s); s++)
            exp = 10 * exp + *s - '0';

        /* calc 10^exp */
        exp = pow(10, exp);
        d = (exp_sign == 1) ? (d * exp) : (d / exp);
    }

    /* res = f(d, sign, pow); */
    return sign * (double)d / p;
}

/********************** string routines ************************/

int strcmp(const char *s0, const char *s1)
{
    for (; *s0 == *s1; s0++, s1++)
        if (*s0 == '\0')
            return 0;
    return *s0 - *s1;
}
```

Exercise 4-6: Add commands for handling variables. (It's easy to provide twenty-six variables with single-letter names.) Add a variable for the most recently printed value.

```
/*
 * calc
 * (if it's not exercise, I'll put into separate files);
 *
 * see "main skeleton" part for description
 * (it's math version, for gcc to compile and link use
 * $ cc <file>.c -lm
 */

#include <stdio.h>
#include <math.h>

/********************** declarations **********************/

/* common parsing */
int isdigit(int c);    /* shouldn't be macro: isdigit(c++) etc. will fail */
int isspace(int c);
int islalpha(int c);
double atof(const char *s);

/* stack */
#define STACK_MAX    100
void push(double d);
double pop(void);
void clear_stack(void);

/* process I/O stream */
int getch(int *buf);

/* read next semantic sequence */
#define ARG_MAX        200
#define NUM        '0'    /* num type */
#define CMD        '1'    /* command type */
#define VAR        '2'    /* use variable */
#define VAR_PR    11    /* num of 'l' in alphabet */
int getarg(char *s);

/********************** main skeleton **********************/
/*
 * main algorithm is below
 * (it doesn't use getline(), proceeding *semantical* constructs
 * one by one instead):
 *
 * while ((new construct was read) not EOF):
 * - if it's number -> place to stack;
 * - if it's operation -> do it;
 * - if it's \n -> get result from stack;
 * - else -> error;
 *
 * calc output is after ">", error is after "!" (showing function sometimes)
```

```
 */

int main(void)
{
    char s[ARG_MAX];    /* arg sequence */
    register int type;    /* arg type */
    register double tmp, tmp2; /* temp for some ops */
    char *cmd_table[] = {
        "pr",        /* print top of stack, not remove */
        "du",        /* dublicate top */
        "sw",        /* swap two tops */
        "cl",        /* clear all stack */
        "sin",        /* sin(x) */
        "cos",        /* cos(x) */
        "exp",        /* exp(x) */
        "pow",        /* pow(x,y) */
        "set"        /* set var */
    };
    double var[26];    /* for a-z, p is special (last printed num) */

    while ((type = getarg(s)) != EOF) {
        switch (type) {
            case VAR:    /* variable -> place to stack */
                push(var[*s - 'a']);
                break;
            case CMD:    /* command -> find in table and execute */

                /* here is very stupid brute force,
                 * should be redone to binsearch, then calling
                 * a function pointer in a table */

                if (!strcmp(s, "pr")) {        /* print top */
                    tmp = pop();
                    printf("> %lf\n", tmp);
                    push(tmp);
                    var[VAR_PR] = tmp; /* update last printed */
                } else if (!strcmp(s, "du")) {    /* dublicate top */
                    tmp = pop();
                    push(tmp);
                    push(tmp);
                } else if (!strcmp(s, "sw")) {    /* swap two tops */
                    tmp = pop();
                    tmp2 = pop();
                    push(tmp);
                    push(tmp2);
                } else if (!strcmp(s, "cl")) /* clear stack */
                    clear_stack();
                else if (!strcmp(s, "sin")) /* sin(x) */
                    push(sin(pop()));
                else if (!strcmp(s, "cos")) /* cos(x) */
```

```
                    push(cos(pop()));
                else if (!strcmp(s, "exp")) /* exp(x) */
                    push(exp(pop()));
                else if (!strcmp(s, "pow")) { /* pow(x,y) */
                    tmp = pop();
                    push(pow(pop(), tmp));
                } else
                    printf("! unknown command %s\n", s);
                break;
            case NUM:    /* number -> place to stack */
                push(atof(s));
                break;
            case '=':    /* set top of stack -> var */
                tmp = pop();
                var[*s - 'a'] = tmp;
                push(tmp);
                break;
            case '+':    /* a b + -> (a + b) */
                push(pop() + pop());
                break;
            case '-':    /* a b - -> (a - b), careful with order */
                tmp = pop();
                push(pop() - tmp);
                break;
            case '*':    /* a b * -> (a * b) */
                push(pop() * pop());
                break;
            case '/':    /* a b / -> (a / b) */
                tmp = pop();
                if (!tmp)    /* / 0 */
                    printf("! / 0\n");
                else
                    push(pop() / tmp);
                break;
            case '%':    /* a b % -> (a % b) */
                tmp = pop();
                push((int)pop() % (int)tmp);
                break;
            case '\n':    /* show result */
                var[VAR_PR] = pop();    /* update last printed */
                printf("> %lf\n", var[VAR_PR]);
                break;
            default:    /* some unexpected */
                printf("! \"%s\" is unknown\n", s);
        }
    }

    return 0;
}
```

```
/********************** stack of operands *****************/

double stack[STACK_MAX];
int sp = 0;        /* next FREE position */

void push(double d)
{
    if (sp < STACK_MAX)
        stack[sp++] = d;
    else
        printf("! push: stack is full\n");
}

double pop(void)
{
    if (sp > 0)
        return stack[--sp];
    printf("! pop: stack is empty\n");

    return 0.0;
}

void clear_stack(void)
{
    sp = 0;
}

/********************** get semantic sequence *************/

int getch(int *buf)
{
    int c;

    /* buf is empty -- read stdin */
    if (*buf == 0)
        return getchar();

    /* buf isn't empty */
    c = *buf, *buf = 0;
    return c;
}

int getarg(char *s)
{
    register int c, isfrac;
    static int b = 0;

    /* skip empty chars */
    while (isspace(c = getch(&b)))
        ;
```

```
    /* first non-empty -- check it:
     * it's ok for it to be:
     * - +-*%/= -- operations (after '-' must be not digit)
     * - 0-9 -- digit, also '-' can be before
     * - . -- fraction flag is set, next . will be ignored
     * - a-z -- command start or variable, in the var case
     *   = may be before
     */

    if (!(isdigit(c) || c == '.' || c == '-' || c == '=' || islalpha(c)))
        return c;        /* first case -> ret op */

    /* command or variable */
    if (islalpha(c)) {
        *s++ = c;    /* write var/1st command letter anyway */
        c = getch(&b);
        if (!islalpha(c)) {    /* variable */
            b = c;
            *s = '\0';
            return VAR;
        } else {        /* command */
            /* accum it, including current */
            do {
                *s++ = c;
            } while (islalpha(c = getch(&b)));
            *s = '\0';
            /* return last to stream */
            b = c;
            return CMD;        /* command type */
        }
    }

    /* will be number */
    isfrac = 0;

    /* special check for minus */
    if (c == '-') {
        *s++ = c;
        c = getch(&b);
        if (!isdigit(c)) {    /* operation, not negative num */
            b = c;
            return '-';
        }
        /* if we here -- then negative num */
        *s++ = c;        /* add to stored - */

    /* also may be empty integer part */
    } else if (c == '.') {
        *s++ = '.';
```

```
        isfrac = 1;

    /* set variable */
    } else if (c == '=') {
        c = getch(&b);
        if (islalpha(c)) {    /* variable specified: a-z */
            *s++ = c;
        } else {        /* var isn't specified: a by default */
            *s++ = 'a';
            b = c;        /* return last symbol */
        }
        *s = '\0';        /* not needed, since 1 letter, but caller
                       may use strcmp() etc. */
        return '=';

    } else        /* digit */
        *s++ = c;

    /* accum all next digits */
    while (isdigit(c = getch(&b)))
        *s++ = c;

    /* first non-digit is '.' and there wasn't '.' before --
     * accum fraction part */
    if (c == '.' && !isfrac) {
        *s++ = '.';
        while (isdigit(c = getch(&b)))
            *s++ = c;
    }

    /* other case (either non-'.' or already had '.') --
     * return to bfer */
    *s = '\0';
    b = c;

    return NUM;
}

/************************ common parsing ******************/

int isdigit(c)
{
    return c >= '0' && c <= '9';
}

int isspace(c)
{
    return c == ' ' || c == '\t';
}

```

```
int islalpha(c)
{
    return c >= 'a' && c <= 'z';
}

double atof(const char *s)
{
    double d, p;
    int sign, exp, exp_sign;

    /* skip spaces */
    while (isspace(*s))
        s++;

    /* check sign */
    sign = 1;
    if (*s == '-') {
        sign = -1;
        s++;
    } else if (*s == '+')
        s++;

    /* main loop */
    for (d = 0.0; isdigit(*s); s++)
        d = 10 * d + *s - '0';

    /* check . */
    p = 1.0;    /* here for correct end */
    if (*s == '.') {
        for (s++; isdigit(*s); s++) {
            d = 10 * d + *s - '0';
            p *= 10;
        }
    }

    /* check exp, independently of . */
    if (*s == 'e' || *s == 'E') {
        s++;

        /* get exponent sign */
        exp_sign = 1;
        if (*s == '-') {
            exp_sign = -1;
            s++;
        } else if (*s == '+')
            s++;

        /* accum exp (cann't call atoi -- since
         * cann't skip spaces etc. */
        for (exp = 0; isdigit(*s); s++)
```

```
            exp = 10 * exp + *s - '0';

        /* calc 10^exp */
        exp = pow(10, exp);
        d = (exp_sign == 1) ? (d * exp) : (d / exp);
    }

    /* res = f(d, sign, pow); */
    return sign * (double)d / p;
}

/********************** string routines ***********************/

int strcmp(const char *s0, const char *s1)
{
    for (; *s0 == *s1; s0++, s1++)
        if (*s0 == '\0')
            return 0;
    return *s0 - *s1;
}

```

Exercise 4-7: Write a routine `ungets(s)` that will push back an entire string onto the input. Should ungets know about buf and bufp, or should it just use `ungetch`?

```
/*
 * calc
 * (if it's not exercise, I'll put into separate files);
 *
 * see "main skeleton" part for description
 * (it's math version, for gcc to compile and link use
 * $ cc <file>.c -lm
 */

#include <stdio.h>
#include <math.h>

/********************* declarations **********************/

/* common parsing */
int isdigit(int c);    /* shouldn't be macro: isdigit(c++) etc. will fail */
int isspace(int c);
int islalpha(int c);
double atof(const char *s);

/* stack */
#define STACK_MAX    100
void push(double d);
double pop(void);
void clear_stack(void);

```

```
/* process I/O stream */
#define BUF_MAX        100
int getch(void);
void ungetch(int c);
void ungets(const char *s);

/* read next semantic sequence */
#define ARG_MAX        200
#define NUM        '0'    /* num type */
#define CMD        '1'    /* command type */
int getarg(char *s);

/********************* main skeleton *********************/
/*
 * main algorithm is below
 * (it doesn't use getline(), proceeding *semantical* constructs
 * one by one instead):
 *
 * while ((new construct was read) not EOF):
 * - if it's number -> place to stack;
 * - if it's operation -> do it;
 * - if it's \n -> get result from stack;
 * - else -> error;
 *
 * calc output is after ">", error is after "!" (showing function sometimes)
 */

int main(void)
{
    char s[ARG_MAX];    /* arg sequence */
    register int type;    /* arg type */
    register double tmp, tmp2; /* temp for some ops */
    char *cmd_table[] = {
        "p",        /* print top of stack, not remove */
        "d",        /* dublicate top */
        "s",        /* swap two tops */
        "c",        /* clear all stack */
        "sin",        /* sin(x) */
        "cos",        /* cos(x) */
        "exp",        /* exp(x) */
        "pow",        /* pow(x,y) */
        "set"        /* set var */
    };
    double vars[26];    /* for a-z */

    while ((type = getarg(s)) != EOF) {
        switch (type) {
            case CMD:    /* command -> find in table and execute */

                /* here is very stupid brute force,
```

```
 * should be redone to binsearch, then calling
 * a function pointer in a table */

if (!strcmp(s, "p")) {        /* print top */
    tmp = pop();
    printf("> %lf\n", tmp);
    push(tmp);
} else if (!strcmp(s, "d")) {    /* dublicate top */
    tmp = pop();
    push(tmp);
    push(tmp);
} else if (!strcmp(s, "s")) {    /* swap two tops */
    tmp = pop();
    tmp2 = pop();
    push(tmp);
    push(tmp2);
} else if (!strcmp(s, "c")) /* clear stack */
    clear_stack();
else if (!strcmp(s, "sin")) /* sin(x) */
    push(sin(pop()));
else if (!strcmp(s, "cos")) /* cos(x) */
    push(cos(pop()));
else if (!strcmp(s, "exp")) /* exp(x) */
    push(exp(pop()));
else if (!strcmp(s, "pow")) { /* pow(x,y) */
    tmp = pop();
    push(pow(pop(), tmp));
} else
    printf("! unknown command %s\n", s);
break;
case NUM:    /* number -> place to stack */
    push(atof(s));
    break;
case '+':    /* a b + -> (a + b) */
    push(pop() + pop());
    break;
case '-':    /* a b - -> (a - b), careful with order */
    tmp = pop();
    push(pop() - tmp);
    break;
case '*':    /* a b * -> (a * b) */
    push(pop() * pop());
    break;
case '/':    /* a b / -> (a / b) */
    tmp = pop();
    if (!tmp)    /* / 0 */
        printf("! / 0\n");
    else
        push(pop() / tmp);
    break;
```

```
            case '%':    /* a b % -> (a % b) */
                tmp = pop();
                push((int)pop() % (int)tmp);
                break;
            case '\n':    /* show result */
                printf("> %lf\n", pop());
                break;
            default:    /* some unexpected */
                printf("! \"%s\" is unknown\n", s);
        }
    }

    return 0;
}

/********************** stack of operands *****************/

double stack[STACK_MAX];
int sp = 0;        /* next FREE position */

void push(double d)
{
    if (sp < STACK_MAX)
        stack[sp++] = d;
    else
        printf("! push: stack is full\n");
}

double pop(void)
{
    if (sp > 0)
        return stack[--sp];
    printf("! pop: stack is empty\n");

    return 0.0;
}

void clear_stack(void)
{
    sp = 0;
}

/********************** I/O stream processing **************/

char io_buf[BUF_MAX];
int bp = 0;        /* next FREE position */

int getch(void)
{
    return (bp > 0) ? io_buf[--bp] : getchar();
```

```
177   }
178
179   void ungetch(int c)
180   {
181       if (bp < BUF_MAX)
182           io_buf[bp++] = c;
183       else
184           printf("! ungetch: io buf is full\n");
185   }
186
187   void ungets(const char *s)
188   {
189       int l;
190
191       /* check length to know how many symbols can be placed */
192       l = strlen(s);
193       l = (l < BUF_MAX - bp) ? l : BUF_MAX - bp;
194       /* ungetch */
195       while (l--)
196           ungetch(*s++);
197   }
198
199   /********************** get semantic sequence *************/
200
201   int getarg(char *s)
202   {
203       register int c, isfrac;
204
205       /* skip empty chars */
206       while (isspace(c = getch()))
207           ;
208
209       /* first non-empty -- check it:
210        * it's ok for it to be:
211        * - +-*%/ -- operations (after '-' must be not digit)
212        * - 0-9 -- digit, also '-' can be before
213        * - . -- fraction flag is set, next . will be ignored
214        * - a-z -- command start
215        */
216
217       if (!(isdigit(c) || c == '.' || c == '-' || islalpha(c)))
218           return c;        /* first case -> ret op */
219
220       /* check for command */
221       if (islalpha(c)) {
222           /* accum it, including current */
223           do {
224               *s++ = c;
225           } while (islalpha(c = getch()));
226           *s = '\0';
```

```
        /* return last to stream */
        if (c != EOF)
            ungetch(c);
        return CMD;        /* command type */
    }

    /* will be number */
    isfrac = 0;

    /* special check for minus */
    if (c == '-') {
        *s++ = c;
        c = getch();
        if (!isdigit(c)) {    /* operation, not negative num */
            if (c != EOF)
                ungetch(c);
            return '-';
        }
        /* if we here -- then negative num */
        *s++ = c;        /* add to stored - */

    /* also may be empty integer part */
    } else if (c == '.') {
        *s++ = '.';
        isfrac = 1;
    } else        /* digit */
        *s++ = c;

    /* accum all next digits */
    while (isdigit(c = getch()))
        *s++ = c;

    /* first non-digit is '.' and there wasn't '.' before --
     * accum fraction part */
    if (c == '.' && !isfrac) {
        *s++ = '.';
        while (isdigit(c = getch()))
            *s++ = c;
    }

    /* other case (either non-'.' or already had '.') --
     * return to buffer */
    *s = '\0';
    if (c != EOF)    /* buffer is chars, cann't store EOF */
        ungetch(c);

    return NUM;
}
```

```
/*********************** common parsing ******************/

int isdigit(c)
{
    return c >= '0' && c <= '9';
}

int isspace(c)
{
    return c == ' ' || c == '\t';
}

int islalpha(c)
{
    return c >= 'a' && c <= 'z';
}

double atof(const char *s)
{
    double d, p;
    int sign, exp, exp_sign;

    /* skip spaces */
    while (isspace(*s))
        s++;

    /* check sign */
    sign = 1;
    if (*s == '-') {
        sign = -1;
        s++;
    } else if (*s == '+')
        s++;

    /* main loop */
    for (d = 0.0; isdigit(*s); s++)
        d = 10 * d + *s - '0';

    /* check . */
    p = 1.0;    /* here for correct end */
    if (*s == '.') {
        for (s++; isdigit(*s); s++) {
            d = 10 * d + *s - '0';
            p *= 10;
        }
    }

    /* check exp, independently of . */
    if (*s == 'e' || *s == 'E') {
        s++;
```

```

        /* get exponent sign */
        exp_sign = 1;
        if (*s == '-') {
            exp_sign = -1;
            s++;
        } else if (*s == '+')
            s++;

        /* accum exp (cann't call atoi -- since
         * cann't skip spaces etc. */
        for (exp = 0; isdigit(*s); s++)
            exp = 10 * exp + *s - '0';

        /* calc 10^exp */
        exp = pow(10, exp);
        d = (exp_sign == 1) ? (d * exp) : (d / exp);
    }

    /* res = f(d, sign, pow); */
    return sign * (double)d / p;
}

/********************** string routines ************************/

int strlen(const char *s)
{
    register int l;

    for (l = 0; *s; s++, l++)
        ;
    return l;
}

int strcmp(const char *s0, const char *s1)
{
    for (; *s0 == *s1; s0++, s1++)
        if (*s0 == '\0')
            return 0;
    return *s0 - *s1;
}

```

Exercise 4-8: Suppose that there will never be more than one character of pushback. Modify `getch` and `ungetch` accordingly.

```
/*
 * calc
 * (if it's not exercise, I'll put into separate files);
 *
 * see "main skeleton" part for description
```

```
 * (it's math version, for gcc to compile and link use
 * $ cc <file>.c -lm
 */

#include <stdio.h>
#include <math.h>

/********************** declarations **********************/

/* common parsing */
int isdigit(int c);    /* shouldn't be macro: isdigit(c++) etc. will fail */
int isspace(int c);
int islalpha(int c);
double atof(const char *s);

/* stack */
#define STACK_MAX    100
void push(double d);
double pop(void);
void clear_stack(void);

/* process I/O stream (in this version --
 * no more than 1 byte is returned to stream) */
int getch(void);
void ungetch(int c);

/* read next semantic sequence */
#define ARG_MAX        200
#define NUM        '0'    /* num type */
#define CMD        '1'    /* command type */
int getarg(char *s);

/********************** main skeleton **********************/
/*
 * main algorithm is below
 * (it doesn't use getline(), proceeding *semantical* constructs
 * one by one instead):
 *
 * while ((new construct was read) not EOF):
 * - if it's number -> place to stack;
 * - if it's operation -> do it;
 * - if it's \n -> get result from stack;
 * - else -> error;
 *
 * calc output is after ">", error is after "!" (showing function sometimes)
 */

int main(void)
{
    char s[ARG_MAX];    /* arg sequence */
```

```
    register int type;    /* arg type */
    register double tmp, tmp2; /* temp for some ops */
    char *cmd_table[] = {
        "p",        /* print top of stack, not remove */
        "d",        /* dublicate top */
        "s",        /* swap two tops */
        "c",        /* clear all stack */
        "sin",        /* sin(x) */
        "cos",        /* cos(x) */
        "exp",        /* exp(x) */
        "pow",        /* pow(x,y) */
        "set"        /* set var */
    };
    double vars[26];    /* for a-z */

    while ((type = getarg(s)) != EOF) {
        switch (type) {
            case CMD:    /* command -> find in table and execute */

                /* here is very stupid brute force,
                 * should be redone to binsearch, then calling
                 * a function pointer in a table */

                if (!strcmp(s, "p")) {        /* print top */
                    tmp = pop();
                    printf("> %lf\n", tmp);
                    push(tmp);
                } else if (!strcmp(s, "d")) {    /* dublicate top */
                    tmp = pop();
                    push(tmp);
                    push(tmp);
                } else if (!strcmp(s, "s")) {    /* swap two tops */
                    tmp = pop();
                    tmp2 = pop();
                    push(tmp);
                    push(tmp2);
                } else if (!strcmp(s, "c")) /* clear stack */
                    clear_stack();
                else if (!strcmp(s, "sin")) /* sin(x) */
                    push(sin(pop()));
                else if (!strcmp(s, "cos")) /* cos(x) */
                    push(cos(pop()));
                else if (!strcmp(s, "exp")) /* exp(x) */
                    push(exp(pop()));
                else if (!strcmp(s, "pow")) { /* pow(x,y) */
                    tmp = pop();
                    push(pow(pop(), tmp));
                } else
                    printf("! unknown command %s\n", s);
                break;
```

```
106             case NUM:    /* number -> place to stack */
107                 push(atof(s));
108                 break;
109             case '+':    /* a b + -> (a + b) */
110                 push(pop() + pop());
111                 break;
112             case '-':    /* a b - -> (a - b), careful with order */
113                 tmp = pop();
114                 push(pop() - tmp);
115                 break;
116             case '*':    /* a b * -> (a * b) */
117                 push(pop() * pop());
118                 break;
119             case '/':    /* a b / -> (a / b) */
120                 tmp = pop();
121                 if (!tmp)    /* / 0 */
122                     printf("! / 0\n");
123                 else
124                     push(pop() / tmp);
125                 break;
126             case '%':    /* a b % -> (a % b) */
127                 tmp = pop();
128                 push((int)pop() % (int)tmp);
129                 break;
130             case '\n':    /* show result */
131                 printf("> %lf\n", pop());
132                 break;
133             default:    /* some unexpected */
134                 printf("! \"%s\" is unknown\n", s);
135         }
136     }
137
138     return 0;
139 }
140
141 /********************** stack of operands *****************/
142
143 double stack[STACK_MAX];
144 int sp = 0;        /* next FREE position */
145
146 void push(double d)
147 {
148     if (sp < STACK_MAX)
149         stack[sp++] = d;
150     else
151         printf("! push: stack is full\n");
152 }
153
154 double pop(void)
155 {
```

```
    if (sp > 0)
        return stack[--sp];
    printf("! pop: stack is empty\n");

    return 0.0;
}

void clear_stack(void)
{
    sp = 0;
}

/********************** I/O stream processing **************/

char io_cp;        /* stored byte */
char is_cp = 0;        /* is something stored (copied) */

int getch(void)
{
    /* something was in "buffer" */
    if (is_cp) {
        is_cp = 0;
        return io_cp;
    }
    /* empty */
    return getchar();
}

void ungetch(int c)
{
    if (!is_cp) {
        is_cp = 1;
        io_cp = c;
    } else
        printf("! buffer is full\n");
}

/************************ get semantic sequence *************/

int getarg(char *s)
{
    register int c, isfrac;

    /* skip empty chars */
    while (isspace(c = getch()))
        ;

    /* first non-empty -- check it:
     * it's ok for it to be:
     * - +-*%/ -- operations (after '-' must be not digit)
```

```
 * - 0-9 -- digit, also '-' can be before
 * - . -- fraction flag is set, next . will be ignored
 * - a-z -- command start
 */

if (!(isdigit(c) || c == '.' || c == '-' || islalpha(c)))
    return c;        /* first case -> ret op */

/* check for command */
if (islalpha(c)) {
    /* accum it, including current */
    do {
        *s++ = c;
    } while (islalpha(c = getch()));
    *s = '\0';

    /* return last to stream */
    if (c != EOF)
        ungetch(c);
    return CMD;        /* command type */
}

/* will be number */
isfrac = 0;

/* special check for minus */
if (c == '-') {
    *s++ = c;
    c = getch();
    if (!isdigit(c)) {    /* operation, not negative num */
        if (c != EOF)
            ungetch(c);
        return '-';
    }
    /* if we here -- then negative num */
    *s++ = c;        /* add to stored - */

/* also may be empty integer part */
} else if (c == '.') {
    *s++ = '.';
    isfrac = 1;
} else        /* digit */
    *s++ = c;

/* accum all next digits */
while (isdigit(c = getch()))
    *s++ = c;

/* first non-digit is '.' and there wasn't '.' before --
 * accum fraction part */
```

```
    if (c == '.' && !isfrac) {
        *s++ = '.';
        while (isdigit(c = getch()))
            *s++ = c;
    }

    /* other case (either non-'.' or already had '.') --
     * return to buffer */
    *s = '\0';
    if (c != EOF)    /* buffer is chars, cann't store EOF */
        ungetch(c);

    return NUM;
}

/*********************** common parsing ******************/

int isdigit(c)
{
    return c >= '0' && c <= '9';
}

int isspace(c)
{
    return c == ' ' || c == '\t';
}

int islalpha(c)
{
    return c >= 'a' && c <= 'z';
}

double atof(const char *s)
{
    double d, p;
    int sign, exp, exp_sign;

    /* skip spaces */
    while (isspace(*s))
        s++;

    /* check sign */
    sign = 1;
    if (*s == '-') {
        sign = -1;
        s++;
    } else if (*s == '+')
        s++;

    /* main loop */
```

```
    for (d = 0.0; isdigit(*s); s++)
        d = 10 * d + *s - '0';

    /* check . */
    p = 1.0;    /* here for correct end */
    if (*s == '.') {
        for (s++; isdigit(*s); s++) {
            d = 10 * d + *s - '0';
            p *= 10;
        }
    }

    /* check exp, independently of . */
    if (*s == 'e' || *s == 'E') {
        s++;

        /* get exponent sign */
        exp_sign = 1;
        if (*s == '-') {
            exp_sign = -1;
            s++;
        } else if (*s == '+')
            s++;

        /* accum exp (cann't call atoi -- since
         * cann't skip spaces etc. */
        for (exp = 0; isdigit(*s); s++)
            exp = 10 * exp + *s - '0';

        /* calc 10^exp */
        exp = pow(10, exp);
        d = (exp_sign == 1) ? (d * exp) : (d / exp);
    }

    /* res = f(d, sign, pow); */
    return sign * (double)d / p;
}

/********************** string routines ************************/

int strlen(const char *s)
{
    register int l;

    for (l = 0; *s; s++, l++)
        ;
    return l;
}

int strcmp(const char *s0, const char *s1)
```

```
{
    for (; *s0 == *s1; s0++, s1++)
        if (*s0 == '\0')
            return 0;
    return *s0 - *s1;
}

```

Exercise 4-9: Our `getch` and `ungetch` do not handle a pushed-back `EOF` correctly. Decide what their properties ought to be if an `EOF` is pushed back, then implement your design.

```
/*
 * calc
 * (if it's not exercise, I'll put into separate files);
 *
 * see "main skeleton" part for description
 * (it's math version, for gcc to compile and link use
 * $ cc <file>.c -lm
 */

#include <stdio.h>
#include <math.h>

/********************** declarations **********************/

/* common parsing */
int isdigit(int c);    /* shouldn't be macro: isdigit(c++) etc. will fail */
int isspace(int c);
int islalpha(int c);
double atof(const char *s);

/* stack */
#define STACK_MAX    100
void push(double d);
double pop(void);
void clear_stack(void);

/* process I/O stream */
#define BUF_MAX        100
int getch(void);
void ungetch(int c);

/* read next semantic sequence */
#define ARG_MAX        200
#define NUM        '0'    /* num type */
#define CMD        '1'    /* command type */
int getarg(char *s);

/********************** main skeleton **********************/
/*
 * main algorithm is below
```

```
41    * (it doesn't use getline(), proceeding *semantical* constructs
42    * one by one instead):
43    *
44    * while ((new construct was read) not EOF):
45    * - if it's number -> place to stack;
46    * - if it's operation -> do it;
47    * - if it's \n -> get result from stack;
48    * - else -> error;
49    *
50    * calc output is after ">", error is after "!" (showing function sometimes)
51    */
52
53   int main(void)
54   {
55       char s[ARG_MAX];    /* arg sequence */
56       register int type;    /* arg type */
57       register double tmp, tmp2; /* temp for some ops */
58       char *cmd_table[] = {
59           "p",        /* print top of stack, not remove */
60           "d",        /* dublicate top */
61           "s",        /* swap two tops */
62           "c",        /* clear all stack */
63           "sin",        /* sin(x) */
64           "cos",        /* cos(x) */
65           "exp",        /* exp(x) */
66           "pow",        /* pow(x,y) */
67           "set"        /* set var */
68       };
69       double vars[26];    /* for a-z */
70
71       while ((type = getarg(s)) != EOF) {
72           switch (type) {
73               case CMD:    /* command -> find in table and execute */
74
75                   /* here is very stupid brute force,
76                    * should be redone to binsearch, then calling
77                    * a function pointer in a table */
78
79                   if (!strcmp(s, "p")) {        /* print top */
80                       tmp = pop();
81                       printf("> %lf\n", tmp);
82                       push(tmp);
83                   } else if (!strcmp(s, "d")) {    /* dublicate top */
84                       tmp = pop();
85                       push(tmp);
86                       push(tmp);
87                   } else if (!strcmp(s, "s")) {    /* swap two tops */
88                       tmp = pop();
89                       tmp2 = pop();
90                       push(tmp);
```

```
                    push(tmp2);
                } else if (!strcmp(s, "c")) /* clear stack */
                    clear_stack();
                else if (!strcmp(s, "sin")) /* sin(x) */
                    push(sin(pop()));
                else if (!strcmp(s, "cos")) /* cos(x) */
                    push(cos(pop()));
                else if (!strcmp(s, "exp")) /* exp(x) */
                    push(exp(pop()));
                else if (!strcmp(s, "pow")) { /* pow(x,y) */
                    tmp = pop();
                    push(pow(pop(), tmp));
                } else
                    printf("! unknown command %s\n", s);
                break;
            case NUM:    /* number -> place to stack */
                push(atof(s));
                break;
            case '+':    /* a b + -> (a + b) */
                push(pop() + pop());
                break;
            case '-':    /* a b - -> (a - b), careful with order */
                tmp = pop();
                push(pop() - tmp);
                break;
            case '*':    /* a b * -> (a * b) */
                push(pop() * pop());
                break;
            case '/':    /* a b / -> (a / b) */
                tmp = pop();
                if (!tmp)    /* / 0 */
                    printf("! / 0\n");
                else
                    push(pop() / tmp);
                break;
            case '%':    /* a b % -> (a % b) */
                tmp = pop();
                push((int)pop() % (int)tmp);
                break;
            case '\n':    /* show result */
                printf("> %lf\n", pop());
                break;
            default:    /* some unexpected */
                printf("! \"%s\" is unknown\n", s);
        }
    }

    return 0;
}
```

```
/********************** stack of operands *****************/

double stack[STACK_MAX];
int sp = 0;        /* next FREE position */

void push(double d)
{
    if (sp < STACK_MAX)
        stack[sp++] = d;
    else
        printf("! push: stack is full\n");
}

double pop(void)
{
    if (sp > 0)
        return stack[--sp];
    printf("! pop: stack is empty\n");

    return 0.0;
}

void clear_stack(void)
{
    sp = 0;
}

/********************** I/O stream processing **************/

int io_buf[BUF_MAX];
int bp = 0;        /* next FREE position */

int getch(void)
{
    return (bp > 0) ? io_buf[--bp] : getchar();
}

void ungetch(int c)
{
    if (bp < BUF_MAX)
        io_buf[bp++] = c;
    else
        printf("! ungetch: io buf is full\n");
}

/********************** get semantic sequence *************/

int getarg(char *s)
{
    register int c, isfrac;
```

```
    /* skip empty chars */
    while (isspace(c = getch()))
        ;

    /* first non-empty -- check it:
     * it's ok for it to be:
     * - +-*%/ -- operations (after '-' must be not digit)
     * - 0-9 -- digit, also '-' can be before
     * - . -- fraction flag is set, next . will be ignored
     * - a-z -- command start
     */

    if (!(isdigit(c) || c == '.' || c == '-' || islalpha(c)))
        return c;        /* first case -> ret op */

    /* check for command */
    if (islalpha(c)) {
        /* accum it, including current */
        do {
            *s++ = c;
        } while (islalpha(c = getch()));
        *s = '\0';

        /* return last to stream */
        ungetch(c);
        return CMD;        /* command type */
    }

    /* will be number */
    isfrac = 0;

    /* special check for minus */
    if (c == '-') {
        *s++ = c;
        c = getch();
        if (!isdigit(c)) {    /* operation, not negative num */
            ungetch(c);
            return '-';
        }
        /* if we here -- then negative num */
        *s++ = c;        /* add to stored - */

    /* also may be empty integer part */
    } else if (c == '.') {
        *s++ = '.';
        isfrac = 1;
    } else        /* digit */
        *s++ = c;

```

```
    /* accum all next digits */
    while (isdigit(c = getch()))
        *s++ = c;

    /* first non-digit is '.' and there wasn't '.' before --
     * accum fraction part */
    if (c == '.' && !isfrac) {
        *s++ = '.';
        while (isdigit(c = getch()))
            *s++ = c;
    }

    /* other case (either non-'.' or already had '.') --
     * return to buffer */
    *s = '\0';
    ungetch(c);

    return NUM;
}

/*********************** common parsing ******************/

int isdigit(c)
{
    return c >= '0' && c <= '9';
}

int isspace(c)
{
    return c == ' ' || c == '\t';
}

int islalpha(c)
{
    return c >= 'a' && c <= 'z';
}

double atof(const char *s)
{
    double d, p;
    int sign, exp, exp_sign;

    /* skip spaces */
    while (isspace(*s))
        s++;

    /* check sign */
    sign = 1;
    if (*s == '-') {
        sign = -1;
```

```
291         s++;
292     } else if (*s == '+')
293         s++;
294
295     /* main loop */
296     for (d = 0.0; isdigit(*s); s++)
297         d = 10 * d + *s - '0';
298
299     /* check . */
300     p = 1.0;    /* here for correct end */
301     if (*s == '.') {
302         for (s++; isdigit(*s); s++) {
303             d = 10 * d + *s - '0';
304             p *= 10;
305         }
306     }
307
308     /* check exp, independently of . */
309     if (*s == 'e' || *s == 'E') {
310         s++;
311
312         /* get exponent sign */
313         exp_sign = 1;
314         if (*s == '-') {
315             exp_sign = -1;
316             s++;
317         } else if (*s == '+')
318             s++;
319
320         /* accum exp (cann't call atoi -- since
321          * cann't skip spaces etc. */
322         for (exp = 0; isdigit(*s); s++)
323             exp = 10 * exp + *s - '0';
324
325         /* calc 10^exp */
326         exp = pow(10, exp);
327         d = (exp_sign == 1) ? (d * exp) : (d / exp);
328     }
329
330     /* res = f(d, sign, pow); */
331     return sign * (double)d / p;
332 }
333
334 /********************** string routines ************************/
335
336 int strcmp(const char *s0, const char *s1)
337 {
338     for (; *s0 == *s1; s0++, s1++)
339         if (*s0 == '\0')
340             return 0;
```

```
    return *s0 - *s1;
}

```

Exercise 4-10: An alternate organization uses `getline` to read an entire input line; this makes `getch` and `ungetch` unnecessary. Revise the calculator to use this approach.

```
/*
 * see slv implementation
 */
```

Exercise 4-11: Modify `getop` so that it doesn't need to use `ungetch`. Hint: use an internal static variable.

```
/*
 * calc
 * (if it's not exercise, I'll put into separate files);
 *
 * see "main skeleton" part for description
 * (it's math version, for gcc to compile and link use
 * $ cc <file>.c -lm
 */

#include <stdio.h>
#include <math.h>

/********************** declarations ***********************/

/* common parsing */
int isdigit(int c);    /* shouldn't be macro: isdigit(c++) etc. will fail */
int isspace(int c);
int islalpha(int c);
double atof(const char *s);

/* stack */
#define STACK_MAX    100
void push(double d);
double pop(void);
void clear_stack(void);

/* process I/O stream */
int getch(int *buf);

/* read next semantic sequence */
#define ARG_MAX        200
#define NUM        '0'    /* num type */
#define CMD        '1'    /* command type */
int getarg(char *s);

/********************** main skeleton **********************/
/*
```

```
 * main algorithm is below
 * (it doesn't use getline(), proceeding *semantical* constructs
 * one by one instead):
 *
 * while ((new construct was read) not EOF):
 * - if it's number -> place to stack;
 * - if it's operation -> do it;
 * - if it's \n -> get result from stack;
 * - else -> error;
 *
 * calc output is after ">", error is after "!" (showing function sometimes)
 */

int main(void)
{
    char s[ARG_MAX];    /* arg sequence */
    register int type;    /* arg type */
    register double tmp, tmp2; /* temp for some ops */
    char *cmd_table[] = {
        "p",        /* print top of stack, not remove */
        "d",        /* dublicate top */
        "s",        /* swap two tops */
        "c",        /* clear all stack */
        "sin",        /* sin(x) */
        "cos",        /* cos(x) */
        "exp",        /* exp(x) */
        "pow",        /* pow(x,y) */
        "set"        /* set var */
    };
    double vars[26];    /* for a-z */

    while ((type = getarg(s)) != EOF) {
        switch (type) {
            case CMD:    /* command -> find in table and execute */

                /* here is very stupid brute force,
                 * should be redone to binsearch, then calling
                 * a function pointer in a table */

                if (!strcmp(s, "p")) {        /* print top */
                    tmp = pop();
                    printf("> %lf\n", tmp);
                    push(tmp);
                } else if (!strcmp(s, "d")) {    /* dublicate top */
                    tmp = pop();
                    push(tmp);
                    push(tmp);
                } else if (!strcmp(s, "s")) {    /* swap two tops */
                    tmp = pop();
                    tmp2 = pop();
```

```
                    push(tmp);
                    push(tmp2);
                } else if (!strcmp(s, "c")) /* clear stack */
                    clear_stack();
                else if (!strcmp(s, "sin")) /* sin(x) */
                    push(sin(pop()));
                else if (!strcmp(s, "cos")) /* cos(x) */
                    push(cos(pop()));
                else if (!strcmp(s, "exp")) /* exp(x) */
                    push(exp(pop()));
                else if (!strcmp(s, "pow")) { /* pow(x,y) */
                    tmp = pop();
                    push(pow(pop(), tmp));
                } else
                    printf("! unknown command %s\n", s);
                break;
            case NUM:    /* number -> place to stack */
                push(atof(s));
                break;
            case '+':    /* a b + -> (a + b) */
                push(pop() + pop());
                break;
            case '-':    /* a b - -> (a - b), careful with order */
                tmp = pop();
                push(pop() - tmp);
                break;
            case '*':    /* a b * -> (a * b) */
                push(pop() * pop());
                break;
            case '/':    /* a b / -> (a / b) */
                tmp = pop();
                if (!tmp)    /* / 0 */
                    printf("! / 0\n");
                else
                    push(pop() / tmp);
                break;
            case '%':    /* a b % -> (a % b) */
                tmp = pop();
                push((int)pop() % (int)tmp);
                break;
            case '\n':    /* show result */
                printf("> %lf\n", pop());
                break;
            default:    /* some unexpected */
                printf("! \"%s\" is unknown\n", s);
        }
    }

    return 0;
}
```

```
138
139 /********************** stack of operands *****************/
140
141 double stack[STACK_MAX];
142 int sp = 0;        /* next FREE position */
143
144 void push(double d)
145 {
146     if (sp < STACK_MAX)
147         stack[sp++] = d;
148     else
149         printf("! push: stack is full\n");
150 }
151
152 double pop(void)
153 {
154     if (sp > 0)
155         return stack[--sp];
156     printf("! pop: stack is empty\n");
157
158     return 0.0;
159 }
160
161 void clear_stack(void)
162 {
163     sp = 0;
164 }
165
166 /********************** get semantic sequence *************/
167
168 int getch(int *buf)
169 {
170     int c;
171
172     /* buf is empty -- read stdin */
173     if (*buf == 0)
174         return getchar();
175
176     /* buf isn't empty */
177     c = *buf, *buf = 0;
178     return c;
179 }
180
181 int getarg(char *s)
182 {
183     register int c, isfrac;
184     static int b = 0;
185
186     /* skip empty chars */
187     while (isspace(c = getch(&b)))
```

```
        ;

    /* first non-empty -- check it:
     * it's ok for it to be:
     * - +-*%/ -- operations (after '-' must be not digit)
     * - 0-9 -- digit, also '-' can be before
     * - . -- fraction flag is set, next . will be ignored
     * - a-z -- command start
     */

    if (!(isdigit(c) || c == '.' || c == '-' || islalpha(c)))
        return c;        /* first case -> ret op */

    /* check for command */
    if (islalpha(c)) {
        /* accum it, including current */
        do {
            *s++ = c;
        } while (islalpha(c = getch(&b)));
        *s = '\0';

        /* return last to stream */
        b = c;
        return CMD;        /* command type */
    }

    /* will be number */
    isfrac = 0;

    /* special check for minus */
    if (c == '-') {
        *s++ = c;
        c = getch(&b);
        if (!isdigit(c)) {    /* operation, not negative num */
            b = c;
            return '-';
        }
        /* if we here -- then negative num */
        *s++ = c;        /* add to stored - */

    /* also may be empty integer part */
    } else if (c == '.') {
        *s++ = '.';
        isfrac = 1;
    } else        /* digit */
        *s++ = c;

    /* accum all next digits */
    while (isdigit(c = getch(&b)))
        *s++ = c;
```

```
    /* first non-digit is '.' and there wasn't '.' before --
     * accum fraction part */
    if (c == '.' && !isfrac) {
        *s++ = '.';
        while (isdigit(c = getch(&b)))
            *s++ = c;
    }

    /* other case (either non-'.' or already had '.') --
     * return to bfer */
    *s = '\0';
    b = c;

    return NUM;
}

/************************ common parsing *******************/

int isdigit(c)
{
    return c >= '0' && c <= '9';
}

int isspace(c)
{
    return c == ' ' || c == '\t';
}

int islalpha(c)
{
    return c >= 'a' && c <= 'z';
}

double atof(const char *s)
{
    double d, p;
    int sign, exp, exp_sign;

    /* skip spaces */
    while (isspace(*s))
        s++;

    /* check sign */
    sign = 1;
    if (*s == '-') {
        sign = -1;
        s++;
    } else if (*s == '+')
        s++;
```

```
    /* main loop */
    for (d = 0.0; isdigit(*s); s++)
        d = 10 * d + *s - '0';

    /* check . */
    p = 1.0;    /* here for correct end */
    if (*s == '.') {
        for (s++; isdigit(*s); s++) {
            d = 10 * d + *s - '0';
            p *= 10;
        }
    }

    /* check exp, independently of . */
    if (*s == 'e' || *s == 'E') {
        s++;

        /* get exponent sign */
        exp_sign = 1;
        if (*s == '-') {
            exp_sign = -1;
            s++;
        } else if (*s == '+')
            s++;

        /* accum exp (cann't call atoi -- since
         * cann't skip spaces etc. */
        for (exp = 0; isdigit(*s); s++)
            exp = 10 * exp + *s - '0';

        /* calc 10^exp */
        exp = pow(10, exp);
        d = (exp_sign == 1) ? (d * exp) : (d / exp);
    }

    /* res = f(d, sign, pow); */
    return sign * (double)d / p;
}

/********************** string routines ***********************/

int strcmp(const char *s0, const char *s1)
{
    for (; *s0 == *s1; s0++, s1++)
        if (*s0 == '\0')
            return 0;
    return *s0 - *s1;
}

```

Exercise 4-12: Adapt the ideas of printd to write a recursive version of `itoa`; that is, convert an integer into a string by calling a recursive routine.

```
/*
 * recursive itoa
 */

#include <stdio.h>

/*
 * f is hack for caller to pass always 1,
 * than it's the highest recursion level (others will have
 * 0), so i will be resetted
 */
void itoa(char *s, int n, int f)
{
    static int i = 0;

    if (n < 0) {    /* called only once */
        s[i++] = '-';
        n = -n;
    }
    if (n / 10)
        itoa(s, n / 10, 0);
    s[i++] = n % 10 + '0';
    s[i] = '\0';
    /* reset i if it was the first time */
    if (f == 1)
        i = 0;
}

/* test */
int main(void)
{
    int n;
    char s[100];

    /* terminate with ^C */
    while (1) {
        printf("n: ");
        scanf("%d", &n);
        itoa(s, n, 1);
        printf("s: %s\n", s);
    }

    return 0;
}

```

Exercise 4-13: Write a recursive version of the function `reverse(s)`, which reverses the string `s` in place.

```
/*
 * recursive reverse
 */

#include <stdio.h>

int strlen(const char *s)
{
    register int l;

    for (l = 0; *s; s++, l++)
        ;
    return l;
}

void reverse(char *s)
{
    char c;
    int pos;

    /* check if > 1 symbol */
    pos = strlen(s) - 1;
    if (pos > 0) {    /* need recursion */
        c = s[pos];
        s[pos] = '\0';
        reverse(s + 1);
        /* swap finally */
        s[pos] = s[0];
        s[0] = c;
    }
}

#define MAXLINE     100

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF && c != '\n'; *s++ = c)
        ;
    *s = '\0';

    return s - b;
}

/* test */
int main(void)
{
```

```
    char s[MAXLINE];

    while (getln(s, MAXLINE) > 0) {
        reverse(s);
        printf("> %s\n", s);
    }

    return 0;
}
```

Exercise 4-14: Define a macro `swap(t,x,y)` that interchanges two arguments of type `t`. (Block structure will help.)

```
/*
 * macros
 */
#include <stdio.h>

#define swap(t, x, y)    {    \
        t tmp;        \
        tmp = x;    \
        x = y;        \
        y = tmp;    \
    }

/* test */
int main(void)
{
    int x, y;

    x = 12L, y = 0xff;
    swap(int, x, y);
    printf("%d %d\n", x, y);

    struct s {
        int a;
        int b;
    } s1 = { 12, 45 }, s2 = { 0xdcUL, 12L };
    swap(struct s, s1, s2);
    printf("%d %d, %d %d\n", s1.a, s1.b, s2.a, s2.b);

    return 0;
}
```

Chapter 5

Pointers and Arrays

Exercise 5-1: As written, `getint` treats a `+` or `-` not followed by a digit as a valid representation of zero. Fix it to push such a character back on the input.

```
/*
 * getint/getfloat
 */

#include <stdio.h>

/* getch/ungetch */

#define MAX_BUF        200

int buf[MAX_BUF];
int bp = 0;        /* next FREE pos */

int getch(void)
{
    return (bp <= 0) ? getchar() : buf[--bp];
}

void ungetch(int c)
{
    if (bp < MAX_BUF)
        buf[bp++] = c;
    else
        printf("ungetch: buffer fill\n");
}

/* main work */
int getint(int *pn)
{
    int c, sign, t;

    while (isspace(c = getch()))    /* skip spaces */
        ;
    if (!isdigit(c) && c != '+' && c != '-' && c != EOF) {
        ungetch(c);
```

```
        return 0;
    }
    sign = (c == '-') ? -1 : 1;
    if (c == '+' || c == '-') {
        t = c;
        c = getch();
        if (!isdigit(c)) {    /* not [+-][0-9] */
            ungetch(c);
            ungetch(t);
            return 0;
        }
    }

    for (*pn = 0; isdigit(c); c = getch())
        *pn = *pn * 10 + c - '0';
    *pn *= sign;
    ungetch(c);

    return c;
}

/* test */
int main(void)
{
    int n;

    while (getint(&n) > 0)
        printf("> %d\n", n);
    return 0;
}
```

Exercise 5-2: Write `getfloat`, the floating-point analog of `getint`. What type does `getfloat` return as its function value?

```
/*
 * getint/getfloat
 */

#include <stdio.h>

/* pow */

int pow(int x, int y)
{
    register int p;

    for (p = 1; y--; p *= x)
        ;
    return p;
}

```

```
/* getch/ungetch */

#define MAX_BUF        200

int buf[MAX_BUF];
int bp = 0;        /* next FREE pos */

int getch(void)
{
    return (bp <= 0) ? getchar() : buf[--bp];
}

void ungetch(int c)
{
    if (bp < MAX_BUF)
        buf[bp++] = c;
    else
        printf("ungetch: buffer fill\n");
}

/* main work */
int getfloat(float *pn)
{
    int c, sign, t, p, exp_sign, exp_pow;

    while (isspace(c = getch()))    /* skip spaces */
        ;

    /* non digit */
    if (!isdigit(c) && c != '+' && c != '-' && c != EOF && c != '.') {
        ungetch(c);
        return 0;
    }

    sign = (c == '-') ? -1 : 1;
    if (c == '+' || c == '-') {    /* potential num's sign */
        t = c;
        c = getch();
        if (!isdigit(c) && c != '.') {    /* not [+-][.0-9] */
            ungetch(c);
            ungetch(t);
            return 0;
        }
    }

    /* integer part */
    for (*pn = 0; isdigit(c); c = getch())
        *pn = *pn * 10 + c - '0';
    /* check fraction */
    if (c == '.')
```

```
        c = getch();
    /* fraction part */
    for (p = 1; isdigit(c); c = getch()) {
        p *= 10;
        *pn = *pn * 10 + c - '0';
    }

    *pn = (sign * *pn) / p;

    /* exponent */
    if (c == 'e' || c == 'E') {
        t = c;
        c = getch();
        /* [eE][^0-9+-] -- incorrect */
        if (c != '+' && c != '-' && !isdigit(c)) {
            ungetch(c);        /* [^0-9+-] -> stream */
            ungetch(t);        /* [eE] -> stream */

            return 1;
        }
        /* check '+'/'-' */
        exp_sign = (c == '-') ? -1 : 1;
        if (c == '+' || c == '-') {
            p = c;      /* temp */
            c = getch();
            if (!isdigit(c)) {    /* [eE][+-][^0-9] -- incorrect */
                ungetch(c);    /* [^0-9] -> stream */
                ungetch(p);    /* [+-] -> stream */
                ungetch(t);    /* [eE] -> stream */

                return 1;
            }
        }
        /* calc pow */
        for (exp_pow = 0; isdigit(c); c = getch())
            exp_pow = 10 * exp_pow + c - '0';
        /* finish */
        if (exp_sign > 0)
            *pn = *pn * pow(10, exp_pow);
        else
            *pn = *pn / pow(10, exp_pow);
    }
    ungetch(c);

    return c;
}

/* test */
int main(void)
{
```

```
    float f;

    while (getfloat(&f) > 0)
        printf("> %f\n", f);
    return 0;
}
```

Exercise 5-3: Write a pointer version of the function `strcat` that we showed in Chapter 2: `strcat(s,t)` copies the string `t` to the end of `s`.

```
/*
 * strcat
 */
#include <stdio.h>

void strcat(char *s, const char *t)
{
    /* find the end */
    while (*s)
        s++;
    /* t -> s */
    while (*s++ = *t++)
        ;
}

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF && c != '\n'; *s++ = c)
        ;
    *s = '\0';

    return s - b;
}

#define MAX_LINE    100

/* test */
int main(void)
{
    char s[MAX_LINE], t[MAX_LINE];

    /* stop with ^C */
    while (1) {
        printf("t: ");
        getln(t, MAX_LINE);
        printf("s: ");
        getln(s, MAX_LINE);
```

```
        strcat(s, t);
        printf("> %s\n", s);
    }

    return 0;
}
```

Exercise 5-4: Write the function `strend(s,t)` which returns 1 if the string `t` occurs at the end of the string `s`, and zero otherwise.

```
/*
 * strend
 */
#include <stdio.h>

int strlen(const char *s)
{
    register int l;

    for (l = 0; *s; s++, l++)
        ;
    return l;
}

int strend(const char *s, const char *t)
{
    register const char *rs, *rt;

    /* move backward */
    rs = s, rt = t;
    rs += strlen(rs), rt += strlen(rt);

    /* loop until either start or first mismatch */
    while (rs != s && rt != t && *--rs == *--rt)
        ;

    return rt == t ? 1 : 0;
}

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF && c != '\n'; *s++ = c)
        ;
    *s = '\0';

    return s - b;
}
```

```

#define MAX_LINE    100


/* test */

int main(void)
{
    char s[MAX_LINE], t[MAX_LINE];

    while (1) {    /* stop with ^C */
        printf("s: ");
        getln(s, MAX_LINE);
        printf("t: ");
        getln(t, MAX_LINE);
        printf("> %d\n", strend(s, t));
    }

    return 0;
}


```

Exercise 5-5: Write versions of the library functions `strncpy`, `strncat`, and `strncmp`, which operate on at most the first `n` characters of their argument strings. For example, `strncpy(s,t,n)` copies at most `n` characters of `t` to `s`.

```
/*
 * strn* functions
 */

#include <stdio.h>

size_t strnlen(const char *s, size_t n)
{
    register size_t l;

    for (l = 0; l < n && *s; s++, l++)
        ;
    return l;
}

void strncpy(char *d, const char *s, size_t n)
{
    while (n-- > 0 && (*d++ = *s++))
        ;
}

void strncat(char *d, const char *s, size_t n)
{
    while (*d)
```

```
25          d++;
26      while (n-- > 0 && (*d++ = *s++))
27          ;
28  }
29
30  int strncmp(const char *s0, const char *s1, size_t n)
31  {
32      for (; n > 0 && *s0 == *s1; s0++, s1++, n--)
33          if (*s0 == '\0' || n <= 0)
34              return 0;
35      return *s0 - *s1;
36  }
37
38  int getln(char *s, int max)
39  {
40      register char *b;
41      register int c;
42
43      for (b = s; (s - b) < (max - 1) &&
44              (c = getchar()) != EOF && c != '\n'; *s++ = c)
45          ;
46      *s = '\0';
47
48      return s - b;
49  }
50
51  #define MAX_LINE    100
52
53
54  /* test */
55  int main(void)
56  {
57      char s0[MAX_LINE], s1[MAX_LINE], tmp[MAX_LINE];
58      int n;
59
60      while (1) {    /* stop with ^C */
61          printf("s0: ");
62          getln(s0, MAX_LINE);
63          printf("s1: ");
64          getln(s1, MAX_LINE);
65          printf("n: ");
66          scanf("%u", &n);
67          getchar();
68          strcpy(tmp, s0);    /* backup: s0 will be modified */
69          /* perform all with originals */
70          printf("strnlen: %u %u\n", strnlen(s0, n), strnlen(s1, n));
71          printf("strncmp: %d\n", strncmp(s0, s1, n));
72          strncpy(s0, s1, n);
73          printf("strncpy: %s\n", s0);
74          strncat(tmp, s1, n);
```

```
        printf("strncat: %s\n\n", tmp);
    }

    return 0;
}


```

Exercise 5-6: Rewrite appropriate programs from earlier chapters and exercises with pointers instead of array indexing. Good possibilities include `getline` (Chapters 1 and 4), `atoi`, `itoa`, and their variants (Chapters 2, 3 and 4), `reverse` (Chapter 3), and `strindex` and `getop` (Chapter 4).

See previous solutions, I've implemented them using pointers as well.

Exercise 5-7: Rewrite `readlines` to store lines in an array supplied by `main`, rather than calling `alloc` to maintain storage. How much faster is the program?

```
/*
 * sort program
 */

#include <stdio.h>
#include <string.h>
#include <stdlib.h>    /* malloc */

/******** one string input **********/

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

#define MAXLINE        100

/********** array of strings **********/

#define MAXLINES    1000000

int readlines(char *lineptr[], int maxlines)
{
```

```
    int lcnt;
    char s[MAXLINE], *sp;

    /* read new line to buffer */
    lcnt = 0;
    while (getln(s, MAXLINE) > 0) {
        /* store it into array */
        if (lcnt >= maxlines)        /* too many */
            return -1;
        sp = malloc(strlen(s) + 1);
        if (sp == NULL)            /* alloc fail */
            return -1;
        strcpy(sp, s);
        lineptr[lcnt++] = sp;
    }

    return lcnt;
}

/****** compare strings ********/

int scmp(const void *s0, const void *s1)
{
    return strcmp(*(char **)s0, *(char **)s1);
}

/****** show them ********/
void prlines(char *lineptr[], int maxlines)
{
    while (maxlines--)
        printf("%s", *lineptr++);
}

/******* free them *******/
void freelines(char *lineptr[], int maxlines)
{
    while (maxlines--)
        free(*lineptr++);
}

/* main test */
int main(void)
{
    int lcnt;
    char *lineptr[MAXLINES];

    if ((lcnt = readlines(lineptr, MAXLINES)) > 0) { /* input ok, may sort */
        qsort(lineptr, lcnt, sizeof(char *), scmp);
        printf("> qsort:\n");
        prlines(lineptr, lcnt);
```

```
        /* free mem */
        freelines(lineptr, lcnt);    /* redundant, not needed */

        return 0;
    } else {
        printf("input fail\n");

        return -1;
    }
}

```

```
/*
 * sort program
 */

#include <stdio.h>
#include <string.h>

/******** one string input **********/

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

#define MAXLINE        100

/********** array of strings **********/

#define MAXLINES    1000000

int readlines(char lineptr[][MAXLINE], int maxlines)
{
    int lcnt;

    /* read new line to buffer */
    lcnt = 0;
    for (lcnt = 0; getln(lineptr[lcnt], MAXLINE) > 0; lcnt++)
```

```
        if (lcnt >= maxlines)        /* too many */
            return -1;

    return lcnt;
}

/****** show them ********/
void prlines(char lineptr[][MAXLINE], int maxlines)
{
    while (maxlines--)
        printf("%s", *lineptr++);
}

/* placed into data section to avoid stack smashes on big counts,
 * since it's two dimensional array, memory-consumpting */
char lineptr[MAXLINES][MAXLINE];

/* main test */
int main(void)
{
    int lcnt;

    if ((lcnt = readlines(lineptr, MAXLINES)) > 0) { /* input ok, may sort */
        qsort(lineptr, lcnt, MAXLINE, strcmp);
        printf("> qsort:\n");
        prlines(lineptr, lcnt);

        return 0;
    } else {
        printf("input fail\n");

        return -1;
    }
}

```

Exercise 5-8: There is no error checking in `day_of_year` or `month_day`. Remedy this defect.

```
/*
 * day_of_year, month_day
 */

#include <stdio.h>

static char daytab[2][13] = {
    {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31},
    {0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}
};

/* (year; month; day of month) -> day of year */
int day_of_year(int year, int month, int day)
```

```
14  {
15      int i, leap;
16
17      /* error check 1 -- monthes */
18      if (month < 1 || month > 12)
19          return -1;
20
21      /* is year leap */
22      leap = (!(year % 4) && (year % 100)) || !(year % 400);
23
24      /* error check 2 -- day */
25      if (day < 1 || day > daytab[leap][month])
26          return -1;
27
28      /* calc */
29      for (i = 1; i < month; i++)
30          day += daytab[leap][i];
31
32      return day;
33  }
34
35  /* (year; day of year) -> (month; day of month) */
36  int month_day(int year, int yearday, int *pmonth, int *pday)
37  {
38      int i, leap;
39
40      leap = (!(year % 4) && (year % 100)) || !(year % 400);
41
42      /* error check -- yearday */
43      if (yearday < 1 || yearday > (leap ? 366 : 365))
44          return -1;
45
46      /* calc */
47      for (i = 1; yearday > daytab[leap][i]; i++)
48          yearday -= daytab[leap][i];
49      *pmonth = i;
50      *pday = yearday;
51
52      return 0;
53  }
54
55  /* test */
56  int main(void)
57  {
58      int d, y, m, yd, res;
59
60      while (1) {    /* ^C to end */
61          /* (y; m; d) -> yd */
62          printf("(y; m; d): ");
63          scanf("%d %d %d", &y, &m, &d);
```

```
        res = day_of_year(y, m, d);
        if (res == -1)
            printf("> incorrect input\n");
        else
            printf("> %d\n", res);
        /* (y; yd) -> (m; d) */
        printf("(y; yd): ");
        scanf("%d %d", &y, &yd);
        if (month_day(y, yd, &m, &d) == -1)
            printf("> incorrect input\n");
        else
            printf("> (%d; %d)\n\n", m, d);
    }

    return 0;
}
```

Exercise 5-9: Rewrite the routines `day_of_year` and `month_day` with pointers instead of indexing.

```
/*
 * day_of_year, month_day
 */

#include <stdio.h>

static char daytab[2][13] = {
    {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31},
    {0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}
};

/* (year; month; day of month) -> day of year */
int day_of_year(int year, int month, int day)
{
    int leap;

    /* error check 1 -- monthes */
    if (month < 1 || month > 12)
        return -1;

    /* is year leap */
    leap = (!(year % 4) && (year % 100)) || !(year % 400);

    /* error check 2 -- day */
    if (day < 1 || day > daytab[leap][month])
        return -1;

    /* calc */
    while (--month)
        day += *(daytab[leap] + month);

    return day;
```

```
}

/* (year; day of year) -> (month; day of month) */
int month_day(int year, int yearday, int *pmonth, int *pday)
{
    int i, leap;

    leap = (!(year % 4) && (year % 100)) || !(year % 400);

    /* error check -- yearday */
    if (yearday < 1 || yearday > (leap ? 366 : 365))
        return -1;

    /* calc */
    for (i = 1; yearday > *(*(daytab + leap) + i); i++)
        yearday -= *(*(daytab + leap) + i);
    *pmonth = i;
    *pday = yearday;

    return 0;
}

/* test */
int main(void)
{
    int d, y, m, yd, res;

    while (1) {    /* ^C to end */
        /* (y; m; d) -> yd */
        printf("(y; m; d): ");
        scanf("%d %d %d", &y, &m, &d);
        res = day_of_year(y, m, d);
        if (res == -1)
            printf("> incorrect input\n");
        else
            printf("> %d\n", res);
        /* (y; yd) -> (m; d) */
        printf("(y; yd): ");
        scanf("%d %d", &y, &yd);
        if (month_day(y, yd, &m, &d) == -1)
            printf("> incorrect input\n");
        else
            printf("> (%d; %d)\n\n", m, d);
    }

    return 0;
}
```

Exercise 5-10: Write the program `expr`, which evaluates a reverse Polish expression from the command line, where each operator or operand is a separate argument.

```
/*
 * see getlex.c in slv
 */

```

Exercise 5-11: Modify the programs `entab` and `detab` (written as exercises in Chapter 1) to accept a list of tab stops as arguments. Use the default tab settings if there are no arguments.

```
/*
 * detab, place tabulations until tab borders
 * in the positions given as list of CLI parameters
 */

#include <stdio.h>

#define TAB        8
#define MAXPOS        100

int tab_default[] = {
    0, 5, 10
};

#define NARR(arr)    (sizeof(arr) / sizeof((arr)[0]))

int main(int argc, char *argv[])
{
    register int c, pos;
    int tabs[MAXPOS], top, i;

    if (argc == 1)
        for (top = 0; top < NARR(tab_default); top++)
            tabs[top] = tab_default[top];
    /* read CLI:
     * $ detab <pos1> [pos2] [pos3] ... [posN]
     */
    else
        for (top = 0; --argc && *++argv; )
            tabs[top++] = atoi(*argv);

    /* read input */
    for (pos = i = 0; (c = getchar()) != EOF; pos++)
        if (pos == tabs[i] && i < top) {
            putchar(c);
            /* reuse c */
            c = TAB - pos % TAB;    /* how many spaces need */
            pos += c - 1;        /* update to avoid i */
            while (c--)
                putchar(' ');
            i++;
        } else {
            if (c == '\n') {    /* update pos */
```

```
                    pos = -1;
                    i = 0;
                }
                putchar(c);
            }
        return 0;
    }
```

Exercise 5-12: Extend `entab` and `detab` to accept the shortland `entab -m +n` to mean tab stops every `n` columns, starting at column `m`. Choose convenient (for the user) default behaviour.

```
/*
 * detab, place tabulations until tab borders
 * in the positions given as list of CLI parameters
 */

#include <stdio.h>

#define TAB        8
#define MAXPOS        100

int tab_default[] = {
    0, 5, 10
};

#define NARR(arr)    (sizeof(arr) / sizeof((arr)[0]))

int main(int argc, char *argv[])
{
    register int c, pos;
    int tabs[MAXPOS], top, i, m, n;

    if (argc == 1)
        for (top = 0; top < NARR(tab_default); top++)
            tabs[top] = tab_default[top];
    /* read CLI:
     * $ detab <pos1> [pos2] [pos3] ... [posN]
     */
    else
        for (top = 0; --argc && *++argv; top++) {
            if (**argv == '-') {
                /* check full form */
                if (**(argv + 1) != '+') {
                    printf("usage: detab [-<m> +<n>]"
                        " or detab <pos1> [pos2] ... [posN]\n");
                    return -1;
                }
                m = atoi((*argv++) + 1);
                n = atoi(*argv + 1);
                argc--;

```

```
                /* write an array */
                tabs[0] = m;
                for (top = 1; top < MAXPOS; top++)
                    tabs[top] = m + n * top;
                break;
            }
            tabs[top] = atoi(*argv);
        }

    /* read input */
    for (pos = i = 0; (c = getchar()) != EOF; pos++)
        if (pos == tabs[i] && i < top) {
            putchar(c);
            /* reuse c */
            c = TAB - pos % TAB;    /* how many spaces need */
            pos += c - 1;        /* update to avoid i */
            while (c--)
                putchar(' ');
            i++;
        } else {
            if (c == '\n') {    /* update pos */
                pos = -1;
                i = 0;
            }
            putchar(c);
        }
    return 0;
}
```

Exercise 5-13: Write the program `tail`, which prints the last `n` lines of its input. By default, `n` is 10, let us say, but it can be changed by an optional argument, so that `tail -n` prints the last `n` lines. The program should behave rationally no matter how unreasonable the input or the value of `n`. Write the program so it makes the best use of available storage; lines hould be stored as in the sorting program of Section 5.6, not in a two-dimensional array of fixed size.

The implementation of `tail` is based on the usage of dynamically allocated array of size 10 or given with `-n` option, and each array elements points to dynamically allocated line. The index of newly allocated line is the number of current line modulus array size, in the case where its placed is used by old line (when number of current line is bigger or equal the size of array), previous line is `free`'ed.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAXLINE        100    /* one line length */

int getln(char *s, int max)
{
    register char *b;
```

```
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF && c != '\n'; *s++ = c)
        ;
    *s = '\0';

    return s - b;
}

int readlines(char *lines[], int maxlines)
{
    char buf[MAXLINE], *alloc;
    int pos, top;

    for (top = 0; getln(buf, MAXLINE) > 0; ) {
        /* prepare dynamic buf */
        alloc = malloc(strlen(buf) + 1);
        if (alloc == NULL)
            return -1;
        strcpy(alloc, buf);

        /* get pos */
        pos = top++ % maxlines;
        if (top > maxlines)
            free(lines[pos]);

        /* place */
        lines[pos] = alloc;
    }
    return top;
}

void freelines(char *lines[], int maxlines, int top)
{
    int pos;

    /* free lines themselves */
    if (top <= maxlines)
        for (pos = 0; pos < top; pos++)
            free(lines[pos]);
    else {
        for (pos = top % maxlines; pos < maxlines; pos++)
            free(lines[pos]);
        for (pos = 0; pos < top % maxlines; pos++)
            free(lines[pos]);
    }

    /* free array */
    free(lines);
```

```
60  }
61
62  int main(int argc, char *argv[])
63  {
64      int maxlines, top, pos;
65      char **lines;
66
67      maxlines = 10;        /* default */
68      while (--argc && *++argv)    /* parse CLI */
69          while (*++*argv)
70              switch (**argv) {
71              case 'n':
72                  if (*++argv == NULL) {
73                      printf("-n argument isn't specified\n");
74                      return 1;
75                  }
76                  maxlines = atoi(*argv);
77                  if (maxlines <= 0) {
78                      printf("n must be >0\n");
79                      return 1;
80                  }
81                  break;
82              case 'h':
83                  printf("usage: tail [-n <maxlines>]\n");
84                  return 0;
85              default:
86                  printf("unknown option: %c\n", **argv);
87                  return 1;
88              }
89
90      /* alloc buf for lines */
91      lines = malloc(sizeof(char *) * maxlines);
92      if (lines == NULL) {
93          printf("fail alloc lines\n");
94          return -1;
95      }
96
97      /* process input and fill lines */
98      if ((top = readlines(lines, maxlines)) != -1) {
99          /* show the lines */
100         if (top <= maxlines)
101             for (pos = 0; pos < top; pos++)
102                 printf("%s\n", lines[pos]);
103         else {
104             for (pos = top % maxlines; pos < maxlines; pos++)
105                 printf("%s\n", lines[pos]);
106             for (pos = 0; pos < top % maxlines; pos++)
107                 printf("%s\n", lines[pos]);
108         }
109         return 0;
```

```
    } else {
        printf("input fail\n");
        return -1;
    }

    /* memory will be free'd anyway, but
     * here is for demonstration */
    freelines(lines, maxlines, top);

    return 0;
}
```

Exercise 5-14: Modify the `sort` program to handle a `-r` flag, which indicates sorting in reverse (decreasing) order. Be sure that `-r` works with `-n`.

```
/*
 * see 5.16.c
 */
```

Exercise 5-15: Add the option `-f` to fold upper and lower case together, so that case distinctions are not made during sorting; for example, `a` and `A` compare equal.

```
/*
 * see 5.16.c
 */
```

Exercise 5-16: Add the `-d` ("directory order") option, which makes comparisons only on letters, numbers and blanks. Make sure it works in conjunction with `-f`.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* normal string comparator */
int strcmp(const char *s0, const char *s1)
{
    for (; *s0 == *s1; s0++, s1++)
        if (*s0 == '\0')
            return 0;
    return *s0 - *s1;
}

/* ignore case */
int stricmp(const char *s0, const char *s1)
{
    char c0, c1;

    for (; *s0 == *s1; s0++, s1++)
        if (*s0 == '\0')
            return 0;
    c0 = *s0, c1 = *s1;
    if (c0 >= 'A' && c0 <= 'Z')
```

```
        c0 += 'a' - 'A';
    if (c1 >= 'A' && c1 <= 'Z')
        c1 += 'a' - 'A';
    return c0 - c1;
}

/* directory order */
int strdcmp(const char *s0, const char *s1)
{
    for (; *s0 == *s1; s0++, s1++)
        if (*s0 == '\0' || !isalnum(*s0))
            return 0;
    return *s0 - *s1;
}

/* directory order with case ignoring */
int strdicmp(const char *s0, const char *s1)
{
    char c0, c1;

    for (; *s0 == *s1; s0++, s1++)
        if (*s0 == '\0' || !isalnum(*s0))
            return 0;
    c0 = *s0, c1 = *s1;
    if (c0 >= 'A' && c0 <= 'Z')
        c0 += 'a' - 'A';
    if (c1 >= 'A' && c1 <= 'Z')
        c1 += 'a' - 'A';
    return c0 - c1;
}

/* getline */
#define MAXLINE        100

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

```

```
/* getlines */
#define MAXLINES    1000

int readlines(char *lines[], int maxlines)
{
    char buf[MAXLINE];
    int lineno;

    for (lineno = 0; getln(buf, MAXLINE) > 0; lineno++)
        /* too big count or alloc error */
        if (lineno >= maxlines || ((lines[lineno] = strdup(buf)) == NULL))
            return -1;

    return lineno;        /* line count */
}

/* qsort */
int scmp(const void *sp0, const void *sp1)
{
    return strcmp(*(const char **)sp0, *(const char **)sp1);
}

int scmp_rev(const void *sp0, const void *sp1)
{
    return strcmp(*(const char **)sp1, *(const char **)sp0);
}

int sicmp(const void *sp0, const void *sp1)
{
    return stricmp(*(const char **)sp0, *(const char **)sp1);
}

int sicmp_rev(const void *sp0, const void *sp1)
{
    return stricmp(*(const char **)sp1, *(const char **)sp0);
}

int sdcmp(const void *sp0, const void *sp1)
{
    return strdcmp(*(const char **)sp0, *(const char **)sp1);
}

int sdcmp_rev(const void *sp0, const void *sp1)
{
    return strdcmp(*(const char **)sp1, *(const char **)sp0);
}

int sdicmp(const void *sp0, const void *sp1)
{
    return strdicmp(*(const char **)sp0, *(const char **)sp1);
```

```
124 }
125
126 int sdicmp_rev(const void *sp0, const void *sp1)
127 {
128     return strdicmp(*(const char **)sp1, *(const char **)sp0);
129 }
130
131 int ncmp(const void *np0, const void *np1)
132 {
133     int i0, i1;
134
135     i0 = atof(*(const char **)np0);
136     i1 = atof(*(const char **)np1);
137
138     return (i0 < i1) ? -1 : (i0 > i1) ? 1 : 0;
139 }
140
141 int ncmp_rev(const void *np0, const void *np1)
142 {
143     int i0, i1;
144
145     i0 = atof(*(const char **)np0);
146     i1 = atof(*(const char **)np1);
147
148     return (i1 < i0) ? -1 : (i1 > i0) ? 1 : 0;
149 }
150
151 /* print lines */
152 void prlines(char *lines[], int maxlines)
153 {
154     while (maxlines--)
155         fputs(*lines++, stdout);
156 }
157
158 int main(int argc, char *argv[])
159 {
160     int r, n, f, d;        /* CLI */
161     char *lines[MAXLINES];
162     int linecnt;
163     /* duplications to avoid any checking */
164     int (*cmp[])(const void *, const void *) = {
165         scmp, scmp_rev, sicmp, sicmp_rev, sdcmp, sdcmp_rev, sdicmp, sdicmp_rev,
166         ncmp, ncmp_rev, ncmp, ncmp_rev, ncmp, ncmp_rev, ncmp, ncmp_rev
167     };
168
169     /* read CLI */
170     for (d = f = r = n = 0; --argc && *++argv; )
171         while (*+++argv)
172             switch (**argv) {
173                 case 'n': n = 1; break;    /* numerical */
```

```
                case 'r': r = 1; break;    /* reverse */
                case 'f': f = 1; break;    /* ignore case */
                case 'd': d = 1; break;    /* directory order */
                default:
                      printf("unknown option: %c\n", **argv);
                      return 2;
            }

    /* read lines */
    linecnt = readlines(lines, MAXLINES);
    if (linecnt == -1) {
        fprintf(stderr, "input error\n");
        return 1;
    }

    /* sort them */
    qsort(lines, linecnt, sizeof(char *), cmp[n * 8 + d * 4 + f * 2 + r]);

    /* show */
    prlines(lines, linecnt);
    return 0;
}
```

Exercise 5-17: Add a field-handling capability, so sorting may be done on fields within lines, each field sorted according to an independent set of options. (The index for this book was sorted with `-df` for the index category and `-n` for the page numbers.)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* getline part */
#define MAXLINE        100
int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF && c != '\n'; *s++ = c)
        ;
    *s = '\0';

    return s - b;
}

/* readlines */
#define MAXLINES    2000
int readlines(char *lines[], int maxlines)
{
    int lineno;
```

```
25      char buf[MAXLINE];
26
27      for (lineno = 0; getln(buf, maxlines) > 0; lineno++)
28          if (lineno >= maxlines || (lines[lineno] = strdup(buf)) == NULL)
29              return -1;
30      return lineno;
31  }
32
33  void prlines(char *lines[], int linecnt)
34  {
35      while (linecnt--)
36          printf("%s\n", *lines++);
37  }
38
39  /* CLI options */
40  #define    OPT_N    (1 << 0)    /* numeric comparison */
41  #define    OPT_R    (1 << 1)    /* reverse comparison */
42  #define    OPT_F    (1 << 2)    /* ignore case while comparing */
43
44  /* line fields */
45  #define MAXFIELDS    5
46
47  int flds[MAXFIELDS];    /* each field has it's own comparison rule */
48  char sep = ' ';        /* separator, default val */
49
50  /* compare strings according to fields;
51   * all is done in the lines themselves, no buffers */
52  int linecmp(const void *sp0, const void *sp1)
53  {
54      register char *t0, *t1, *s0, *s1;
55      register int fld, cmp;
56
57      /* get concrete */
58      s0 = *(char **)sp0, s1 = *(char **)sp1;
59
60      /* walk */
61      for (fld = 0, t0 = s0, t1 = s1; ; fld++) {
62          while (*s0 != '\0') {    /* next field in first */
63              if (*s0 == sep)
64                  break;
65              s0++;
66          }
67          while (*s1 != '\0') {    /* next field in second */
68              if (*s1 == sep)
69                  break;
70              s1++;
71          }
72          if ((unsigned)(*s0) ^ (unsigned)(*s1))    { /* not equal --
73                                  some missymetry */
74              fprintf(stderr, "err, not symmetrical fields\n");
```

```
            exit(1);
        }

        if (*s0 == '\0')    /* both finished */
            return fldcmp(fld, t0, t1);    /* CMP HERE */
                /* not finished, cut the next token */
        *s0 = *s1 = '\0';
        cmp = fldcmp(fld, t0, t1);        /* CMP HERE */
        *s0 = *s1 = sep;    /* restore separator */
        if (cmp)    /* already know what line is "bigger" */
            return cmp;
        while (*++s0 == sep)
            ;
        while (*++s1 == sep)
            ;
        t0 = s0, t1 = s1;
    }
}

int fldcmp(int fld, const char *s0, const char *s1)
{
    int i0, i1;
    char c0, c1;

    if (flds[fld] & OPT_N) {    /* numerical comparison */
        i0 = atoi(s0), i1 = atoi(s1);    /* return i0 - i1 will fail on big */
        if (flds[fld] & OPT_R)
            return i1 < i0 ? -1 : i1 > i0 ? 1 : 0;
        return i0 < i1 ? -1 : i0 > i1 ? 1 : 0;
    }
    if (!(flds[fld] & OPT_F))
        return (flds[fld] & OPT_R) ? strcmp(s1, s0) : strcmp(s0, s1);
    return (flds[fld] & OPT_R) ? strcasecmp(s1, s0) : strcasecmp(s0, s1);
}

/* free lines */
void frlines(char *lines[], int linecnt)
{
    while (linecnt--)
        free(*lines++);
}

/* some error checks are skipped */
int main(int argc, char **argv)
{
    int opt;    /* global CLI */
    char *lines[MAXLINES];
    int linecnt;
    int fld;

```

```
    /* process CLI */
    for (opt = 0, sep = ' '; --argc && **++argv == '-'; ) {
        while (*++*argv)
            switch (**argv) {
                case 't':    /* field separator */
                    sep = *++*argv;
                    break;
                case 'k':
                    fld = *++*argv - '0';
                    while (*++*argv)
                        switch (**argv) {
                        case 'n':
                            flds[fld] |= OPT_N;
                            break;
                        case 'f':
                            flds[fld] |= OPT_F;
                            break;
                        case 'r':
                            flds[fld] |= OPT_R;
                            break;
                        default:
                            printf("unknown option %c\n",
                                    **argv);
                            return 1;
                        }
                    goto next;

                    /* global params
                     * (substituted by -k for each field) */
                case 'n': opt |= OPT_N; break;    /* numerical */
                case 'f': opt |= OPT_F; break;    /* ignoring case */
                case 'r': opt |= OPT_R; break;    /* reverse */
                default:
                      printf("unknown option %c\n", **argv);
                      return 2;
            }
next:        ;
    }

    /* store all options */
    for (fld = 0; fld < MAXFIELDS; fld++)
        flds[fld] |= opt;

    /* read lines */
    if ((linecnt = readlines(lines, MAXLINES)) == -1) {
        fprintf(stderr, "input err\n");
        return 1;
    }
    /* sort */
    qsort(lines, linecnt, sizeof(char *), linecmp);
```

```
    /* show */
    prlines(lines, linecnt);
    /* free (not needed, but just as demonstration */
    frlines(lines, linecnt);

    return 0;
}
```

Exercise 5-18: Make `dcl` recover from input errors.

```
/*
 * dcl program,
 * fixed:
 * - int arr[f f f f f]
 * - [] fun()
 * - clearing all after any errors
 */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* getch/ungetch */
#define    MAXBUF        1000
static char buf[MAXBUF];
static int bp = 0;
int getch(void)
{
    return bp > 0 ? buf[--bp] : getchar();
}
void ungetch(int c)
{
    if (bp >= MAXBUF)
        printf("ungetch: buf is fill\n");
    else
        buf[bp++] = c;
}

/* main work */
#define MAXTOKEN    100
enum { NAME, PARENS, BRACKETS };
void dcl(void);
void dirdcl(void);
void undcl(void);

/* global vars */
int tokentype;
char token[MAXTOKEN];
char name[MAXTOKEN];
char datatype[MAXTOKEN];
char out[1000];

```

```
/* error occured, clean all state */
void error(const char *msg)
{
    fprintf(stderr, msg);
    tokentype = 0;
    *token = *name = *datatype = *out = '\0';
    main();        /* restart */
}

int gettoken(void)
{
    register int c;
    register char *p;

    p = token;
    while ((c = getch()) == ' ' || c == '\t')    /* skip spaces */
        ;
    if (c == '(') {
        if ((c = getch()) == ')') {
            strcpy(token, "()");
            return tokentype = PARENS;
        } else {
            ungetch(c);
            return tokentype = '(';
        }
    } else if (c == '[') {
        for (*p++ = c; (*p = getch()) != ']'; p++)
            if (!isalnum(*p))
                error("array size isn't var/num\n");
        *++p = '\0';
        return tokentype = BRACKETS;
    } else if (isalpha(c)) {
        for (*p++ = c; isalnum(c = getch()); )
            *p++ = c;
        *p = '\0';
        ungetch(c);
        return tokentype = NAME;
    } else
        return tokentype = c;
}

/*
 * dcl:
 * [****...***]<direct>
 *
 * C -> english, e.g.:
 * int (*ptr[])() -> ptr: array of pointer to function returning int
 */
void dcl(void)
{
```

```
    int ns;

    for (ns = 0; gettoken() == '*'; )
        ns++;
    dirdcl();
    while (ns-- > 0)
        strcat(out, " pointer to");
}

/* only direct declarations */
void dirdcl(void)
{
    int type;

    if (tokentype == '(') {
        dcl();
        if (tokentype != ')')
            error("error: missing )\n");
    } else if (tokentype == NAME)
        strcpy(name, token);
    else
        error("error: expected name or (dcl)\n");
    while ((type = gettoken()) == PARENS || type == BRACKETS)
        if (type == PARENS)
            strcat(out, " function returning");
        else {
            strcat(out, " array");
            strcat(out, token);
            strcat(out, " of");
        }
}

int istype(char *s)
{
    if (!isalpha(*s))
        return 0;
    while (!isspace(*s) && *s) {
        if (!isalnum(*s))
            return 0;
        s++;
    }
    return 1;
}

/*********** main**********/
int main(void)
{
    while (gettoken() != EOF) {
        if (!istype(token))
            error("first lexem is not a type\n");
```

```
        strcpy(datatype, token);
        *out = '\0';
        dcl();
        if (tokentype != '\n')
            error("syntax error\n");
        printf("%s: %s %s\n", name, out, datatype);
    }
    return 0;
}
```

Exercise 5-19: Modify `undcl` so that it does not add redundant parentheses to declarations.

```
/*
 * dcl program,
 * fixed:
 * - int arr[f f f f f]
 * - [] fun()
 * - clearing all after any errors
 */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* getch/ungetch */
#define    MAXBUF        1000
static char buf[MAXBUF];
static int bp = 0;
int getch(void)
{
    return bp > 0 ? buf[--bp] : getchar();
}
void ungetch(int c)
{
    if (bp >= MAXBUF)
        printf("ungetch: buf is fill\n");
    else
        buf[bp++] = c;
}

/* main work */
#define MAXTOKEN    100
enum { NAME, PARENS, BRACKETS };
void dcl(void);
void dirdcl(void);
void undcl(void);

/* global vars */
int tokentype;
char token[MAXTOKEN];
char name[MAXTOKEN];
char datatype[MAXTOKEN];
```

```
40  char out[1000];
41
42  /* error occured, clean all state */
43  void error(const char *msg)
44  {
45      fprintf(stderr, msg);
46      tokentype = 0;
47      *token = *name = *datatype = *out = '\0';
48      main();        /* restart */
49  }
50
51  int gettoken(void)
52  {
53      register int c;
54      register char *p;
55
56      p = token;
57      while ((c = getch()) == ' ' || c == '\t')    /* skip spaces */
58          ;
59      if (c == '(') {
60          if ((c = getch()) == ')') {
61              strcpy(token, "()");
62              return tokentype = PARENS;
63          } else {
64              ungetch(c);
65              return tokentype = '(';
66          }
67      } else if (c == '[') {
68          for (*p++ = c; (*p = getch()) != ']'; p++)
69              if (!isalnum(*p))
70                  error("array size isn't var/num\n");
71          *++p = '\0';
72          return tokentype = BRACKETS;
73      } else if (isalpha(c)) {
74          for (*p++ = c; isalnum(c = getch()); )
75              *p++ = c;
76          *p = '\0';
77          ungetch(c);
78          return tokentype = NAME;
79      } else
80          return tokentype = c;
81  }
82
83  /*
84   * undcl:
85   * english (modified) -> C, e.g.:
86   * x [] * () int ->  int (*x[])()
87   */
88  int main(void)
89  {
```

```
    int type;
    char temp[MAXTOKEN];

    while (gettoken() != EOF) {
        strcpy(out, token);
        while ((type = gettoken()) != '\n')
            if (type == BRACKETS || type == PARENS) {
                if (*out == '*') {
                    strcpy(temp, "(");
                    strcat(temp, out);
                    strcat(temp, ")");
                    strcpy(out, temp);
                }
                strcat(out, token);
            } else if (type == '*') {
                strcpy(temp, "*");
                strcat(temp, out);
                strcpy(out, temp);
            } else if (type == NAME) {
                strcpy(temp, token);
                strcat(temp, " ");
                strcat(temp, out);
                strcpy(out, temp);
            } else
                error("invalid input\n");
        printf("%s\n", out);
    }
    return 0;
}

int istype(char *s)
{
    if (!isalpha(*s))
        return 0;
    while (!isspace(*s) && *s) {
        if (!isalnum(*s))
            return 0;
        s++;
    }
    return 1;
}
```

Exercise 5-20: Expand `dcl` to handle declarations with function argument types, qualifiers like `const`, and so on.

```
/*
 * dcl program,
 * fixed:
 * - int arr[f f f f f]
 * - [] fun()
 * - clearing all after any errors
```

```
 */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* getch/ungetch */
#define    MAXBUF        1000
static char buf[MAXBUF];
static int bp = 0;
int getch(void)
{
    return bp > 0 ? buf[--bp] : getchar();
}
void ungetch(int c)
{
    if (bp >= MAXBUF)
        printf("ungetch: buf is fill\n");
    else
        buf[bp++] = c;
}

/* main work */
#define MAXTOKEN    100
enum { NAME, PARENS, ARGS, BRACKETS };
void dcl(void);
void dirdcl(void);
void undcl(void);

/* global vars */
int tokentype;
char token[MAXTOKEN];
char name[MAXTOKEN];
char datatype[MAXTOKEN];
char out[1000];

/* error occured, clean all state */
void error(const char *msg)
{
    fprintf(stderr, msg);
    tokentype = 0;
    *token = *name = *datatype = *out = '\0';
    main();        /* restart */
}

int ismodifier(char *s)
{
    static char *mods[] = {
        "const",
        "register",
        "volatile",
```

```
        "restrict",
        NULL
    };
    register char **p;

    for (p = mods; *p; p++)
        if (!strcmp(*p, s))
            return 1;
    return 0;
}

int istype(char *s)
{
    if (!isalpha(*s))
        return 0;
    while (!isspace(*s) && *s) {
        if (!isalnum(*s))
            return 0;
        s++;
    }
    return 1;
}

/* after "fun(", e.g.:
 * "const int, int, char)"
 */
int getargs(void)
{
    register int c;
    register char *p, *t;

    p = token, c = '(';
    while (1) {
        do {
            *p++ = c;
            while (isspace(c = getch()))
                ;
            if (!isalpha(c))
                error("modifier/type must start from letter\n");
            t = p, *p++ = c;
            while (isalnum(c = getch()))
                *p++ = c;
            if (c != ' ' && c != ')' && c != ',')
                error("unknown char in args\n");
            *p = '\0';
        } while (ismodifier(t));
        if (c == ')') {
            *p++ = c;
            break;
        }
```

```
        if (c == ' ')
            *(p - 1) = ',';
    }
    *p = '\0';
}

int gettoken(void)
{
    register int c;
    register char *p;

    p = token;
    while ((c = getch()) == ' ' || c == '\t')    /* skip spaces */
        ;
    if (c == '(') {
        if ((c = getch()) == ')') {
            strcpy(token, "()");
            return tokentype = PARENS;
        } else {
            /* function can be after (dcl) or name */
            if (tokentype == NAME || tokentype == ')') {
                ungetch(c);
                getargs();
                return tokentype = ARGS;
            } else {
                ungetch(c);
                return tokentype = '(';
            }
        }
    } else if (c == '[') {
        for (*p++ = c; (*p = getch()) != ']'; p++)
            if (!isalnum(*p))
                error("array size isn't var/num\n");
        *++p = '\0';
        return tokentype = BRACKETS;
    } else if (isalpha(c)) {
        for (*p++ = c; isalnum(c = getch()); )
            *p++ = c;
        *p = '\0';
        ungetch(c);
        return tokentype = NAME;
    } else
        return tokentype = c;
}

/*
 * dcl:
 * [****...***]<direct>
 *
 * C -> english, e.g.:
```

```
 * int (*ptr[])() -> ptr: array of pointer to function returning int
 */
void dcl(void)
{
    int ns;

    for (ns = 0; gettoken() == '*'; )
        ns++;
    dirdcl();
    while (ns-- > 0)
        strcat(out, " pointer to");
}

/* only direct declarations */
void dirdcl(void)
{
    int type;

    if (tokentype == '(') {
        dcl();
        if (tokentype != ')')
            error("error: missing )\n");
    } else if (tokentype == NAME)
        strcpy(name, token);
    else
        error("error: expected name or (dcl)\n");
    while ((type = gettoken()) == PARENS || type == BRACKETS || type == ARGS)
        if (type == PARENS)
            strcat(out, " function returning");
        else if (type == ARGS) {
            strcat(out, " function receiving ");
            strcat(out, token);
            strcat(out, " returning");
        } else {
            strcat(out, " array");
            strcat(out, token);
            strcat(out, " of");
        }
}


/*********** main**********/
int main(void)
{
    while (gettoken() != EOF) {
        tokentype = -1;        /* no type yet */
        if (!istype(token))
            error("first lexem is not a type\n");
        strcpy(datatype, token);
        *out = '\0';
```

```
        dcl();
        if (tokentype != '\n')
            error("syntax error\n");
        printf("%s: %s %s\n", name, out, datatype);
    }
    return 0;
}
```

Chapter 6

Structures

Exercise 6-1: Our version of `getword` does not properly handle underscores, string constants, comments, or preprocessor control lines. Write a better version.

```
/* some tests: _auto, #auto, auto_, char
 * char char auto auto auto*/
#include <stdio.h>
#include <stdlib.h>

/* getch/ungetch */
#define MAXBUF        100
static char buf[MAXBUF];
static int bp = 0;

int getch(void)
{
    return bp > 0 ? buf[--bp] : getchar();
}

void ungetch(int c)
{
    if (bp >= MAXBUF)
        printf("ungetch: buf is fill\n");
    else
        buf[bp++] = c;
}

/* check commentaries and strings */
static int iscom = 0;
static int isstr = 0;

/* getword: read the words */

#define MAXWORD        100

int getword(char *word, int lim)
{
    int c;
    char *w;
```

```

    w = word;
    while (isspace(c = getch()))
        ;
    if (c == EOF)
        return EOF;


    /* inside comments -- immediatly finish if not the end of comment */
    if (iscom) {
        if (c == '*') {
            if ((c = getch()) == '/')
                iscom = 0;
            else
                ungetch(c);
        }
        return *word = '\0';
    }

    /* inside string literal -- immediatly finish if not the end of it */
    if (isstr) {
        if (c == '\\')
            getch();    /* skip possible \" */
        else if (c == '\"')    /* end of pars */
            isstr = 0;
        return *word = '\0';
    }

    /* process input */
    for (ungetch(c); --lim > 0; w++) {
        /* keyword is separated in the end -- normal case, count it */
        if (isspace(*w = getch()) || *w == '\n' || *w == '(' ||
                *w == ',' || *w == ';' || *w == ':')
            break;
        if (*w == '_')        /* not keyword, but name */
            return *word = '\0';
        if (*w == '/') {    /* check commentaries */
            if ((c = getch()) == '*') {
                iscom = 1;
            } else
                ungetch(c);
            return *word = '\0';
        }
        if (*w == '\\')        /* check brackets */
            getch();    /* skip possible \" */
        else if (*w == '\"') {
            isstr = 1;
            return *word = '\0';
        }
    }
```

```
    *w = '\0';
    return *word;
}

/* key words of C */

struct key {
    char *word;
    int cnt;
} keytab[] = {
    { "auto", 0 },
    { "break", 0 },
    { "case", 0 },
    { "char", 0 },
    { "const", 0 },
    { "continue", 0 },
    { "default", 0 }
};

#define NKEYS    (sizeof keytab / sizeof(struct key))

/* comparator */
int keycmp(const void *key, const void *rec)
{
    return strcmp((const char *)key, ((struct key *)rec)->word);
}

int main(int argc, char *argv[])
{
    auto struct key *keyp;
    char word[MAXWORD];
    /* test this: auto, char */
    int _auto;

    /* read new word */
    while (getword(word, MAXWORD) != EOF)
        if (isalpha(*word))    /* keyword */
            /* find and inc freq */
            if ((keyp = bsearch(word, keytab, NKEYS, sizeof(struct key),
                                keycmp)) != NULL)
                keyp->cnt++;

    /* show all */
    for (keyp = keytab; keyp < &keytab[NKEYS]; keyp++)
        if (keyp->cnt)
            printf("%s: %d\n", keyp->word, keyp->cnt);
    return 0;
}
```

Exercise 6-2: Write a program that reads a C program and prints in alphabetical order each group

of variable names that are identical in the first 6 characters, but different somewhere thereafter. Don't count words within strings and comments. Make 6 a parameter that can be set from the command line.

There are 2 points in implementation of the utility:

- names must be stored in the binary tree, and comparison in `add_tree` should be implemented with `strncmp` instead of `strcmp`;
- to avoid storing words from comments and string literals parser must check the current state (maintaining variables `iscom` and `isstr` respectively):
  - if `/*` is reached and `iscom` is off, it indicates that the comment is started;
  - if `*/` is reached and `iscom` is on, it indicates that the comment is ended;
  - if \ is reached, next `"` is skipped;
  - if `"` is reached, isstr is inversed.

  Only those names are stored which occure outside of comments and literals.

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <string.h>
4
5   /* tree of var names */
6   struct tnode {
7       char *name;
8       struct tnode *l;
9       struct tnode *r;
10  };
11
12  static int nvarchars = 6;
13
14  struct tnode *add_tnode(struct tnode *node, char *name)
15  {
16      int c;
17
18      if (node == NULL) {
19          /* alloc */
20          node = malloc(sizeof(struct tnode));
21          if (node == NULL) {        /* node itself */
22              fprintf(stderr, "error alloc node for %s\n", name);
23              exit(1);
24          }
25          node->name = strndup(name, nvarchars);
26          if (node->name == NULL) {    /* name */
27              free(node);
28              fprintf(stderr, "error alloc string for %s\n", name);
29              exit(1);
30          }
31          /* links */
```

```
32          node->l = node->r = NULL;
33      } else {
34          c = strncmp(name, node->name, nvarchars);
35          if (c < 0)
36              node->l = add_tnode(node->l, name);
37          else if (c > 0)
38              node->r = add_tnode(node->r, name);
39      }
40
41      return node;
42  }
43
44  void treeprint(struct tnode *root)
45  {
46      if (root != NULL) {
47          treeprint(root->l);
48          printf("%s\n", root->name);
49          treeprint(root->r);
50      }
51  }
52
53  /* word-related stuff */
54  #define MAXWORD        100
55
56  int issep(int c)
57  {
58      return c == '(' || c == ')' || c == ';' || c == ':';
59  }
60
61  int getword(char *w, int max)
62  {
63      register char *b;
64      register int c;
65
66      for (b = w; (w - b) < (max - 1) && (c = getchar()) != '\n'
67              && !isspace(c) && !issep(c); *w++ = c)
68          if (c == EOF) {
69              *w = '\0';
70              return EOF;
71          }
72      *w = '\0';
73
74      return *b;
75  }
76
77  /* check if word is variable in C program part */
78
79  static int iscom = 0;        /* in commentary */
80  static int isstr = 0;        /* in string literal */
81
```

```
82  static char *keys[] = {
83      "auto", "break", "case",
84      "char", "const", "continue",
85      "default", "do", "double",
86      "else", "enum", "extern",
87      "float", "for", "goto",
88      "if", "int", "long",
89      "register", "return", "short",
90      "signed", "sizeof", "static",
91      "struct", "switch", "typedef",
92      "union", "unsigned", "void",
93      "volatile", "while"
94  };
95
96  #define NKEYS    (sizeof keys / sizeof(keys[0]))
97
98  int scmp(const void *name, const void *keyp)
99  {
100     return strcmp((const char *)name, *(const char **)keyp);
101 }
102
103 int isvar(char *word)
104 {
105     register char *w;
106
107     /* [a-zA-Z][0-9a-zA-Z]*, not a keyword */
108     if (!isalpha(*word) || bsearch(word, keys, NKEYS, sizeof(char *), scmp)
109             != NULL)
110         return 0;
111     for (w = word + 1; *w; w++)
112         if (!isalnum(*w))
113             return 0;
114
115     /* not in commentaries nor in brackets */
116     return !(iscom || isstr);
117 }
118
119 /* check for commentaries or " */
120 int parse_word(const char *w)
121 {
122     for (; *w; w++)
123         if (*w == '/') {
124             if (*++w == '*' && !isstr)
125                 iscom = 1;
126             else
127                 w--;
128         } else if (*w == '*') {
129             if (*++w == '/' && !isstr)
130                 iscom = 0;
131             else
```

```
                w--;
        } else if (*w == '\\')
            w++;    /* skip possible \" */
        else if (*w == '\"' && !iscom)
            isstr ^= 1;
}


int main(int argc, char *argv[])
{
    struct tnode *root;
    char word[MAXWORD];

    if (argc > 1)
        nvarchars = atoi(*++argv);

    /* read and place */
    for (root = NULL; getword(word, MAXWORD) != EOF; ) {
        parse_word(word);    /* switch comment modes if needed */
        if (isvar(word) && !iscom && !isstr)
            root = add_tnode(root, word);
    }

    /* show words */
    printf("Tree is:\n");
    treeprint(root);

    return 0;
}
```

Exercise 6-3: Write a cross-referencer that prints a list of all words in a document, and, for each word, a list of the line numbers on which it occurs. Remove noise words like "the", "and", and so on.

Main algorithm is based on the binary tree where each node stores pointer to word string and pointers to head and tail of the linked list storing line numbers where it occured, head is needed for printing line numbers in correct order in the end, while tail is where new line number node will be placed.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* list of linenos */
struct lnode {
    unsigned lineno;
    struct lnode *nxt;
};

/* tree */
```

```
12  struct tnode {
13      char *word;
14      struct lnode *head;
15      struct lnode *tail;
16      struct tnode *l;
17      struct tnode *r;
18  };
19
20  struct tnode *add_tree(struct tnode *tnode, const char *word,
21          unsigned lineno)
22  {
23      struct lnode *lnode;
24      int cmp;
25
26      if (tnode == NULL) {
27          tnode = malloc(sizeof(*tnode));
28          if (tnode == NULL)
29              goto err;
30          tnode->word = strdup(word);
31          if (tnode->word == NULL) {
32              free(tnode);
33              goto err;
34          }
35          lnode = malloc(sizeof(*lnode));
36          if (lnode == NULL) {
37              free(tnode->word);
38              free(tnode);
39              goto err;
40          }
41          lnode->nxt = NULL, lnode->lineno = lineno;
42          tnode->head = tnode->tail = lnode;
43          tnode->l = tnode->r = NULL;
44      } else {
45          cmp = strcmp(tnode->word, word);
46          if (cmp < 0)
47              tnode->r = add_tree(tnode->r, word, lineno);
48          else if (cmp > 0)
49              tnode->l = add_tree(tnode->l, word, lineno);
50          else if (lineno != tnode->tail->lineno) {
51              lnode = malloc(sizeof(*lnode));
52              if (lnode == NULL)
53                  goto err;
54              lnode->nxt = NULL, lnode->lineno = lineno;
55              tnode->tail->nxt = lnode;
56              tnode->tail = lnode;
57          }
58      }
59
60      return tnode;
61
```

```
62  err:
63      fprintf(stderr, "error %s %d\n", word, lineno);
64      exit(1);
65  }
66
67  void print_tree(struct tnode *tnode)
68  {
69      register struct lnode *lp;
70
71      if (tnode != NULL) {
72          print_tree(tnode->l);
73
74          /* print */
75          printf("%s: ", tnode->word);
76          for (lp = tnode->head; lp != NULL; lp = lp->nxt)
77              printf("%d ", lp->lineno);
78          putchar('\n');
79
80          print_tree(tnode->r);
81      }
82  }
83
84  /* getword part */
85  #define MAXWORD        100
86
87  /* word is written,
88   * first incorrect char is returned */
89  int getword(char *word, int max)
90  {
91      register char *w;
92      register int c;
93
94      for (w = word; (w - word) < (max - 1) &&
95              isalpha(c = getchar()); *w++ = c)
96          ;
97      *w = '\0';
98
99      return c;
100 }
101
102 /* main work */
103 int main(void)
104 {
105     char word[MAXWORD];
106     struct tnode *root;
107     register unsigned lineno;
108     register int c;
109
110     /* read and place */
111     for (root = NULL, lineno = 1; (c = getword(word, MAXWORD)) != EOF; ) {
```

```
        if (isalpha(*word))
            root = add_tree(root, word, lineno);
        if (c == '\n')
            lineno++;
    }

    /* show */
    print_tree(root);
    return 0;
}

```

Exercise 6-4: Write a program that prints the distinct words in its input sorted into decreasing order of frequency of occurence. Precede each word by its count.

There are 2 main ways to implement this program:

- using two binary tree, one for word-frequency pair, and another is built after first in the way similar to tree from the cross-referencer program but with frequency as direct node data and the list of words to which it points;
- using one binary tree of pairs and an array which is filled with data from the tree and sorted.

```
/*
 * error checks are skipped
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* tree with words */
struct tnode {
    char *word;
    int frq;
    struct tnode *l, *r;
};

struct tnode *add_tree(struct tnode *root, char *word)
{
    int cmp;

    if (root == NULL) {
        root = malloc(sizeof(*root));
        root->word = strdup(word);
        root->l = root->r = NULL;
        root->frq = 1;
    } else {
        cmp = strcmp(word, root->word);
        if (cmp < 0)
            root->l = add_tree(root->l, word);
        else if (cmp > 0)
```

```
            root->r = add_tree(root->r, word);
        else
            root->frq++;
    }
    return root;
}

/* tree with frequencies */
struct lnode {
    char *word;
    struct lnode *nxt;
};

struct tnode2 {
    int frq;
    struct lnode *head, *tail;
    struct tnode2 *l, *r;
};

struct tnode2 *add_tree2(struct tnode2 *root, char *word, int frq)
{
    struct lnode *lnode;

    if (root == NULL) {
        root = malloc(sizeof(*root));
        root->frq = frq;
        root->l = root->r = NULL;
        lnode = malloc(sizeof(*lnode));
        lnode->word = strdup(word);
        lnode->nxt = NULL;
        root->head = root->tail = lnode;
    } else {
        if (root->frq > frq)
            root->l = add_tree2(root->l, word, frq);
        else if (root->frq < frq)
            root->r = add_tree2(root->r, word, frq);
        else {
            lnode = malloc(sizeof(*lnode));
            lnode->word = strdup(word);
            lnode->nxt = NULL;
            root->tail->nxt = lnode;
            root->tail = lnode;
        }
    }
    return root;
}

/* tr1 -> tr2 */
void tr1tr2(struct tnode *root, struct tnode2 **root2)
{
```

```
    if (root != NULL) {
        tr1tr2(root->r, root2);
        *root2 = add_tree2(*root2, root->word, root->frq);
        tr1tr2(root->l, root2);
    }
}

/* print tr1 */
void prtr(struct tnode *root)
{
    if (root != NULL) {
        prtr(root->l);
        printf("%s: %d\n", root->word, root->frq);
        prtr(root->r);
    }
}

/* print tr2 */
void prtr2(struct tnode2 *root)
{
    register struct lnode *lp;

    if (root != NULL) {
        prtr2(root->r);
        printf("%d: ", root->frq);
        for (lp = root->head; lp != NULL; lp = lp->nxt)
            printf("%s ", lp->word);
        putchar('\n');
        prtr2(root->l);
    }
}

/* getword */

#define MAXWORD        100

int getword(char *word, int max)
{
    register char *w;
    register int c;

    for (w = word; (w - word) < (max - 1) &&
            isalpha(c = getchar()); *w++ = c)
        ;
    *w = '\0';

    return c;
}

int main(void)
```

```
129 {
130     char word[MAXWORD];
131     struct tnode *root;
132     struct tnode2 *root2;
133 
134     root = (struct tnode *)(root2 = NULL);
135     /* calc freqs */
136     while (getword(word, MAXWORD) != EOF)
137         if (isalpha(*word))
138             root = add_tree(root, word);
139 
140     /* create the freq tree */
141     tr1tr2(root, &root2);
142 
143     /* show */
144     prtr2(root2);
145 
146     return 0;
147 }
```

```
 1 /*
 2  * error checks are skipped
 3  */
 4 #include <stdio.h>
 5 #include <stdlib.h>
 6 #include <string.h>
 7 
 8 /* count */
 9 static int el_cnt = 0;
10 
11 /* tree with words */
12 struct tnode {
13     char *word;
14     int frq;
15     struct tnode *l, *r;
16 };
17 
18 struct tnode *add_tree(struct tnode *root, char *word)
19 {
20     int cmp;
21 
22     el_cnt++;
23     if (root == NULL) {
24         root = malloc(sizeof(*root));
25         root->word = strdup(word);
26         root->l = root->r = NULL;
27         root->frq = 1;
28     } else {
29         cmp = strcmp(word, root->word);
30         if (cmp < 0)
```

```
31              root->l = add_tree(root->l, word);
32          else if (cmp > 0)
33              root->r = add_tree(root->r, word);
34          else
35              root->frq++;
36      }
37      return root;
38  }
39
40  /* array with frequencies */
41
42  struct el {
43      char *word;
44      int frq;
45  };
46
47  static int top = 0;
48
49  /* fill the array */
50  void tr2arr(struct tnode *root, struct el *arr)
51  {
52      if (root != NULL) {
53          tr2arr(root->l, arr);
54          arr[top].word = strdup(root->word);
55          arr[top].frq = root->frq;
56          top++;
57          tr2arr(root->r, arr);
58      }
59  }
60
61  /* comparator */
62  int elcmp(const void *el1, const void *el2)
63  {
64      return ((struct el*)el1)->frq < ((struct el*)el2)->frq;
65  }
66
67  /* getword */
68
69  #define MAXWORD        100
70
71  int getword(char *word, int max)
72  {
73      register char *w;
74      register int c;
75
76      for (w = word; (w - word) < (max - 1) &&
77              isalpha(c = getchar()); *w++ = c)
78          ;
79      *w = '\0';
80
```

```
    return c;
}

int main(void)
{
    char word[MAXWORD];
    struct tnode *root;
    struct el *arr;

    /* calc freqs */
    root = NULL;
    while (getword(word, MAXWORD) != EOF)
        if (isalpha(*word))
            root = add_tree(root, word);

    /* arr with freqs */
    arr = malloc(sizeof(*arr) * el_cnt);

    /* fill it */
    tr2arr(root, arr);

    /* sort it */
    qsort(arr, top, sizeof(struct el), elcmp);

    /* show arr */
    while (top--) {
        printf("%d %s\n", arr->frq, arr->word);
        arr++;
    }

    return 0;
}
```

Exercise 6-5: Write a function `undef` that will remove a name and definition from the table maintaned by `lookup` and `install`.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* hash table */

struct node {
    char *macro;
    char *subst;
    struct node *nxt;
};

#define NHASHTAB    2    /* make this 101 or some prime big num for real
                        cases */

```

```
16  static struct node *hashtab[NHASHTAB];
17
18  unsigned hash(const char *s)
19  {
20      register unsigned h;
21
22      for (h = 0; *s; s++)
23          h = 31 * h + *s;
24      return h % NHASHTAB;
25  }
26
27  struct node *lookup(const char *macro)
28  {
29      register struct node *np;
30
31      for (np = hashtab[hash(macro)]; np != NULL; np = np->nxt)
32          if (!strcmp(np->macro, macro))
33              return np;
34      return NULL;
35  }
36
37  struct node *install(const char *macro, const char *subst)
38  {
39      struct node *np;
40      unsigned h;
41
42      np = lookup(macro);
43      if (np == NULL) {
44          np = malloc(sizeof(*np));
45          if (np == NULL)
46              return NULL;
47          if ((np->macro = strdup(macro)) == NULL) {
48              free(np);
49              return NULL;
50          }
51          h = hash(macro);
52          np->nxt = hashtab[h];
53          hashtab[h] = np;
54      } else
55          free(np->subst);
56      if ((np->subst = strdup(subst)) == NULL)
57          return NULL;
58
59      return np;
60  }
61
62  int undef(const char *macro)
63  {
64      register struct node **prv, *cur;
65
```

```
    /* cann't use lookup() since it'll
     * return pointer to node without previous pointer
     */
    prv = &hashtab[hash(macro)];
    for (cur = *prv; cur != NULL; prv = &(cur->nxt), cur = cur->nxt)
        if (!strcmp(cur->macro, macro)) {
            *prv = cur->nxt;
            free(cur->macro);
            free(cur->subst);
            free(cur);
            return 1;
        }
    return 0;
}

void printtab(void)
{
    register struct node **hp, *np;

    for (hp = hashtab; hp < &hashtab[NHASHTAB]; hp++) {
        printf("queue: ");
        for (np = *hp; np != NULL; np = np->nxt)
            printf("(%s; %s) ", np->macro, np->subst);
        putchar('\n');
    }
}

/* getline part */

#define MAXLINE        1000

int getln(char *s, int max)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getchar()) != EOF && c != '\n'; *s++ = c)
        ;
    *s = '\0';

    return s - b;
}

int main(void)
{
    install("pi", "3.1415");
    install("ex", "2.718281828");
    install("q", "1234.5");
    install("tt", "56456.7");
```

```
    install("tr", "12555");
    undef("tytyt");
    undef("tt");
    undef("tr");
    undef("ex");
    undef("q");
    undef("pi");
    printtab();
    return 0;
}
```

Exercise 6-6: Implement a simple version of the `#define` processor (i.e., no arguments) suitable for use with C programs, based on the routines of this section. You may also find `getch` and `ungetch` helpful.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* hash table */

struct node {
    char *macro;
    char *subst;
    struct node *nxt;
};

#define NHASHTAB    101

static struct node *hashtab[NHASHTAB];

unsigned hash(const char *s)
{
    register unsigned h;

    for (h = 0; *s; s++)
        h = 31 * h + *s;
    return h % NHASHTAB;
}

struct node *lookup(const char *macro)
{
    register struct node *np;

    for (np = hashtab[hash(macro)]; np != NULL; np = np->nxt)
        if (!strcmp(np->macro, macro))
            return np;
    return NULL;
}

struct node *install(const char *macro, const char *subst)
```

```
37  {
38      struct node *np;
39      unsigned h;
40
41      np = lookup(macro);
42      if (np == NULL) {
43          np = malloc(sizeof(*np));
44          if (np == NULL)
45              return NULL;
46          if ((np->macro = strdup(macro)) == NULL) {
47              free(np);
48              return NULL;
49          }
50          h = hash(macro);
51          np->nxt = hashtab[h];
52          hashtab[h] = np;
53      } else
54          free(np->subst);
55      if ((np->subst = strdup(subst)) == NULL)
56          return NULL;
57
58      return np;
59  }
60
61  int undef(const char *macro)
62  {
63      register struct node **prv, *cur;
64
65      /* cann't use lookup() since it'll
66       * return pointer to node without previous pointer
67       */
68      prv = &hashtab[hash(macro)];
69      for (cur = *prv; cur != NULL; prv = &(cur->nxt), cur = cur->nxt)
70          if (!strcmp(cur->macro, macro)) {
71              *prv = cur->nxt;
72              free(cur->macro);
73              free(cur->subst);
74              free(cur);
75              return 1;
76          }
77      return 0;
78  }
79
80  void printtab(void)
81  {
82      register struct node **hp, *np;
83
84      for (hp = hashtab; hp < &hashtab[NHASHTAB]; hp++) {
85          if (*hp == NULL)
86              continue;
```

```
        printf("queue: ");
        for (np = *hp; np != NULL; np = np->nxt)
            printf("(%s; %s) ", np->macro, np->subst);
        putchar('\n');
    }
}

/* getch/ungetch */

#define MAXBUF        1000

static char buf[MAXBUF];
static int bp = 0;

int getch(void)
{
    return bp > 0 ? buf[--bp] : getchar();
}

void ungetch(int c)
{
    if (bp < MAXBUF)
        buf[bp++] = c;
    else
        fprintf(stderr, "ungetch: buf fill\n");
}

/* word */

#define MAXWORD        100

int getword(char *word, int max)
{
    register char *w;
    register int c;

    for (w = word; (w - word) < (max - 1) &&
            isalpha(c = getch()); *w++ = c)
        ;
    *w = '\0';

    return c;
}

int main(void)
{
    register int c;
    int iscom, isstr;
    char w[MAXWORD], s[MAXWORD], *p;

```

```
    iscom = isstr = 0;
    while ((c = getch()) != EOF) {
        switch (c) {
            case '#':
                if (iscom || isstr){    /* comment or string */
                    printf("uyes\n");
                    break;
                }
                getword(w, MAXWORD);
                if (!strcmp(w, "define")) {
                    /* skip spaces */
                    while (isspace(c = getch()))
                        ;
                    ungetch(c);
                    if (!(isalpha(c) || c == '_'))
                        break;

                    /* read macro */
                    p = w;
                    while (isalnum(c = getch()) || c == '_')
                        *p++ = c;
                    ungetch(c);
                    *p = '\0';

                    /* skip spaces */
                    while (isspace(c = getch()))
                        ;
                    ungetch(c);

                    /* read subst */
                    p = s;
                    while (!isspace(c = getch()))
                        *p++ = c;
                    ungetch(c);
                    *p = '\0';

                    /* install to table */
                    install(w, s);

                } else {        /* not define */
                    for (p = &w[strlen(w) - 1]; p >= w; p--)
                        ungetch(*p);
                }
                break;
            case '/':
                if ((c = getch()) == '*' && !iscom && !isstr)
                    iscom = 1;
                else
                    ungetch(c);
                break;
```

```
            case '*':
                if ((c = getch()) == '/' && iscom && !isstr)
                    iscom = 0;
                else
                    ungetch(c);
                break;
            case '\\':
                if ((c = getch()) != '\"')
                    ungetch(c);
                break;
            case '\"':
                if (!iscom)
                    isstr ^= 1;
                break;
        }
    }

    /* show table */
    printtab();

    return 0;
}
```

Chapter 7

Input and Output

Exercise 7-1: Write a program that converts upper case to lower or lower case to upper, depending on the name it is invoked with, as found in `argv[0]`.

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

int check_name(char *argv)
{
    int big;

    while (!isalpha(*argv))
        argv++;
    for (big = isupper(*argv++); *argv; argv++)
        if (isalpha(*argv) && (isupper(*argv) != big))
            return -1;
    return !!big;
}


int main(int argc, char *argv[])
{
    int big, c, (*fp[])(int) = { tolower, toupper };

    if ((big = check_name(*argv)) == -1) {
        fprintf(stderr, "name must be written in one case\n");
        exit(1);
    }
    while ((c = getchar()) != EOF)
        putchar((*fp[big])(c));
    return 0;
}
```

Exercise 7-2: Write a program that will print arbitrary input in a sensible way. As a minimum, it should print non-graphic characters in octal or hexademical according to local custom, and break long text lines.

```
/*
```

```
 * prints each symbol as it is for printable ones,
 * or as hex code in other case;
 * it's actually strange behaviour for '\n' symbols, although it's demonstrative
 */
#include <stdio.h>

#define DEF_MAX_POS    80

int main(int argc, char *argv[])
{
    register int c, pos, maxpos;

    maxpos = DEF_MAX_POS;    /* default */

    if (argc > 1)        /* error check is skipped */
        maxpos = atoi(*(argv + 1));

    for (pos = 0; (c = getchar()) != EOF; pos++) {
        printf(!isprint(c) ? "0x%x" : "%c", c);
        if (pos == maxpos - 1) {
            printf("0x%x", '\n');
            pos = -1;
        }
    }
    return 0;
}

```

Exercise 7-3: Revise `minprintf` to handle more of the other facilities of `printf`.

```
#include <stdio.h>
#include <stdarg.h>

int minprintf(const char *fmt, ...)
{
    va_list argp;
    int argc;
    char *s;

    for (va_start(argp, fmt), argc = 0; *fmt; fmt++) {
        if (*fmt != '%')
            putchar(*fmt);
        else
            switch (*++fmt) {
            case 'd':
            case 'i':
                printf("%d", va_arg(argp, int));
                argc++;
                break;
            case 'u':
                printf("%u", va_arg(argp, unsigned));
```

```
                argc++;
                break;
            case 'o':
                printf("%o", va_arg(argp, unsigned));
                argc++;
                break;
            case 'x':
            case 'X':
                printf("%x", va_arg(argp, unsigned));
                argc++;
                break;
            case 's':
                for (s = va_arg(argp, char *); *s; s++)
                    putchar(*s);
                argc++;
                break;
            case 'f':
                printf("%f", va_arg(argp, double));
                argc++;
                break;
            case 'e':
            case 'E':
                argc++;
                printf("%e", va_arg(argp, double));
                break;
            case 'p':
                argc++;
                printf("%p", va_arg(argp, void *));
                break;
            case '%':
            default:
                putchar(*fmt);
        }
    }
    va_end(argp);

    return argc;
}

#define NARR(arr)    (sizeof (arr) / sizeof((arr)[0]))

int main(void)
{
    int arr[] = {
        minprintf("Cool Lady"),
        minprintf("Cool Lady %s", "Ayrinus\n"),
        minprintf("Cool Lady %s\n", "Ayrinus"),
        minprintf("Cool Lady %s\n", "Ayrinus\n"),
        minprintf("List of %d, %d, %u\n", 12, -12, 12U),
        minprintf("Now float: %f\n", 12e2)
```

```
    }, i;

    printf("arr size:\n");
    for (i = 0; i < NARR(arr); i++)
        printf("%d ", arr[i]);
    putchar('\n');

    return 0;
}
```

Exercise 7-4: Write a private version of `scanf` analogous to `minprintf` from the previous section.

```
#include <stdio.h>
#include <ctype.h>
#include <stdarg.h>

int minscanf(const char *fmt, ...)
{
    va_list argp;
    int argc;

    for (argc = 0, va_start(argp, fmt); *fmt; fmt++) {
        if (isblank(*fmt))
            continue;
        if (*fmt == '%')
            switch (*++fmt) {
                case 'd':
                case 'i':
                    argc++;
                    scanf("%d", va_arg(argp, int *));
                    continue;
                case 'o':
                    argc++;
                    scanf("%o", va_arg(argp, unsigned *));
                    continue;
                case 'u':
                    argc++;
                    scanf("%d", va_arg(argp, unsigned *));
                    continue;
                case 'x':
                    argc++;
                    scanf("%x", va_arg(argp, unsigned *));
                    continue;
                case 'c':
                    argc++;
                    scanf("%c", va_arg(argp, char *));
                    continue;
                case 's':
                    argc++;
                    scanf("%s", va_arg(argp, char *));
                    continue;
```

```
                case 'e':
                    argc++;
                    scanf("%lf", va_arg(argp, double *));
                    continue;
                case 'f':
                    argc++;
                    scanf("%f", va_arg(argp, float *));
                    continue;
                /* including %% */
            }
        if (getchar() != *fmt)
            break;
    }

    return argc;

}

int main(void)
{
    int a, b, c, cnt;

    a = b = c = 0;
    cnt = minscanf("%d %d %d", &a, &b, &c);
    printf("a is %d\nb is %d\nc is %d\n", a, b, c);
    printf("cnt is %d\n", cnt);

    return 0;
}

```

Exercise 7-5: Rewrite the postfix calculator of Chapter 4 to use `scanf` and/or `sscanf` to do the input and number conversion.

```
/*
 * calc
 * (if it's not exercise, I'll put into separate files);
 *
 * see "main skeleton" part for description
 */

#include <stdio.h>

/********************** declarations ***********************/

/* stack */
#define STACK_MAX    100
void push(double d);
double pop(void);

/********************** main skeleton **********************/
```

```
18  /*
19   * main algorithm is below
20   * (it doesn't use getline(), proceeding *semantical* constructs
21   * one by one instead):
22   *
23   * while ((new construct was read) not EOF):
24   * - if it's number -> place to stack;
25   * - if it's operation -> do it;
26   * - if it's \n -> get result from stack;
27   * - else -> error;
28   *
29   * calc output is after ">", error is after "!" (showing function sometimes)
30   */
31
32  int main(void)
33  {
34      double d1, d2, tmp;
35      char op;
36
37      while (1) {
38          if (scanf("%lf %lf %c", &d1, &d2, &op) != 3)
39              printf("! some err\n");
40          else {
41              push(d1);        /* prepare operands */
42              push(d2);
43              switch (op) {        /* prepare operation */
44              case '+':    /* a b + -> (a + b) */
45                  push(pop() + pop());
46                  break;
47              case '-':    /* a b - -> (a - b), careful with order */
48                  tmp = pop();
49                  push(pop() - tmp);
50                  break;
51              case '*':    /* a b * -> (a * b) */
52                  push(pop() * pop());
53                  break;
54              case '/':    /* a b / -> (a / b) */
55                  tmp = pop();
56                  if (!tmp)    /* / 0 */
57                      printf("! / 0\n");
58                  else
59                      push(pop() / tmp);
60                  break;
61              case '%':    /* a b % -> (a % b) */
62                  tmp = pop();
63                  push((int)pop() % (int)tmp);
64                  break;
65              default:    /* some unexpected */
66                  printf("! %c is unknown\n", op);
67              }
```

```

            printf("> %lf\n", pop());
        }
    }

    return 0;
}

/********************** stack of operands *****************/

double stack[STACK_MAX];
int sp = 0;        /* next FREE position */

void push(double d)
{
    if (sp < STACK_MAX)
        stack[sp++] = d;
    else
        printf("! push: stack is full\n");
}

double pop(void)
{
    if (sp > 0)
        return stack[--sp];
    printf("! pop: stack is empty\n");

    return 0.0;
}
```

Exercise 7-6: Write a program to compare two files, printing the first line where they differ.

```
/* simple version of diff */

#include <stdio.h>
#include <stdlib.h>

int fgetln(char *s, int max, FILE *fp)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getc(fp)) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
```

```
20  }
21
22  int fputln(char *s, FILE *fp)
23  {
24      for (; *s; s++)
25          putc(*s, fp);
26      return ferror(fp);
27  }
28
29  #define MAXLINE        1000
30
31  int main(int argc, char *argv[])
32  {
33      FILE *fp1, *fp2;
34      char s1[MAXLINE], s2[MAXLINE];
35      register int l1, l2, ln1, ln2;
36
37      /* check CLI */
38      if (argc != 3) {
39          fprintf(stderr, "usage: diff <f1> <f2>\n");
40          exit(1);
41      }
42
43      /* open files */
44      fp1 = fopen(argv[1], "r");
45      if (fp1 == NULL) {
46          fprintf(stderr, "error opening %s\n", argv[1]);
47          exit(1);
48      }
49      fp2 = fopen(argv[2], "r");
50      if (fp2 == NULL) {
51          fprintf(stderr, "error opening %s\n", argv[2]);
52          exit(1);
53      }
54
55      /* read in loop */
56      for (ln1 = ln2 = 1; ; ln1++, ln2++) {
57          l1 = fgetln(s1, MAXLINE, fp1);
58          l2 = fgetln(s2, MAXLINE, fp2);
59
60          /* some of files are longer;
61           * may be avoided by always
62           * if (strcmp(s1, s2)) ...,
63           * but may show empty line for some file
64           */
65          if (l1 && !l2)
66              printf("> %s %d: %s", argv[1], ln1, s1);
67          else if (!l1 && l2)
68              printf("> %s %d: %s", argv[2], ln2, s2);
69          else if (strcmp(s1, s2))
```

```
            printf("> %s %d: %s\n> %s %d: %s\n",
                    argv[1], ln1, s1, argv[2], ln2, s2);
        else if (!l1 && !l2)    /* end of stream */
            break;
        else
            continue;
    }

    exit(0);
}

```

Exercise 7-7: Modify the pattern finding program of Chapter 5 to take its input from a set of named files or, if no files are named as arguments, from the standard input. Should the file name be printed when a matching line is found?

```
/* grep without regexs, features:
 * - supports both stdin/file arg modes;
 * - options are (must be before arguments)
 *   -i
 *   -v
 *   -c
 *   -n
 */

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* I/O */

#define MAXLINE        1000

int fgetln(char *s, int max, FILE *fp)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getc(fp)) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

int fputln(char *s, FILE *fp)
{
```

```
    for (; *s; s++)
        putc(*s, fp);
    return ferror(fp);
}

/* grep CLI options */

static int v = 0;        /* except */
static int c = 0;        /* only count */
static int n = 0;        /* linenos */
/* ignoring the case */
extern char *strcasestr(const char *, const char *);
static char *(*find)(const char *, const char *) = strstr;
static int multiple = 0;    /* > 1 file to read, need prefix name in output */

/* grep output */
static int lineno, cnt;        /* line num, match cnt */

void grep(const char *pattern, const char *fname)
{
    FILE *fp;
    char s[MAXLINE];

    if (fname == NULL)        /* open a file part */
        fp = stdin;
    else {
        fp = fopen(fname, "r");
        if (fp == NULL) {
            fprintf(stderr, "grep: error opening %s\n", fname);
            return;
        }
    }

    /* main loop */
    for (lineno = 1, cnt = 0; fgetln(s, MAXLINE, fp) > 0; lineno++)
        if (((*find)(s, pattern) != NULL) != v) {
            cnt++;
            if (c)
                continue;
            if (multiple)
                printf("%s:", fname);
            if (n)
                printf("%d:", lineno);
            fputln(s, stdout);
        }
    fclose(fp);
    if (c) {
        if (multiple)
            printf("%s:", fname);
        printf("%d\n", cnt);
```

```
    }
}

int main(int argc, char *argv[])
{
    char *pattern;

    while (--argc && **++argv == '-')
        while (*++*argv)
            switch (**argv) {
                case 'c': c = 1; break;
                case 'i': find = strcasestr; break;
                case 'v': v = 1; break;
                case 'n': n = 1; break;
                case 'h':
                      printf("usage: grep [-v] [-i] [-c] [-n]"
                        " <pattern> [file 1] [file 2] [...]"
                        " [file n]\n");
                      exit(0);
                default:
                      fprintf(stderr, "grep: unknown option: %c\n"
                              "-h for help\n",
                              **argv);
                      exit(2);
            }
    if (!argc) {
        fprintf(stderr, "grep: missed pattern\n-h for help\n");
        exit(2);
    }
    if (argc <= 1)
        grep(*argv, NULL);        /* main work */
    else {
        if (argc > 2)
            multiple = 1;
        for (pattern = *argv++; --argc; argv++)
            grep(pattern, *argv);    /* main work */
    }

    exit(0);
}
```

Exercise 7-8: Write a program to print a set of files, starting each new one on a new page, with a title and a running page count for each file.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* some routines */

char *str2upper(char *s)
```

```
8   {
9       register char *t;
10
11      for (t = s; *t; t++)
12          *t = toupper(*t);
13
14      return s;
15  }
16
17  /* I/O part */
18
19  #define MAXLINE        1000
20
21  int fgetln(char *s, int max, FILE *fp)
22  {
23      register char *b;
24      register int c;
25
26      for (b = s; (s - b) < (max - 1) &&
27              (c = getc(fp)) != EOF; *s++ = c)
28          if (c == '\n') {
29              *s++ = c;
30              break;
31          }
32      *s = '\0';
33
34      return s - b;
35  }
36
37  int fputln(const char *s, FILE *fp)
38  {
39      for (; *s; s++)
40          putc(*s, fp);
41      return ferror(fp);
42  }
43
44  /* numeration of pages, running through all files */
45  static int pageno = 1;
46  static int lineno = 1;
47  static int pagesize = 25;
48
49  /* pcat */
50  void cat(const char *fname)
51  {
52      FILE *fp;
53      char s[MAXLINE];
54      register int lineno;
55
56      if (fname != NULL) {
57          fp = fopen(fname, "r");
```

```
        if (fp == NULL) {
            fprintf(stderr, "pcat: error opening: %s\n", fname);
            return;
        }
    } else
        fp = stdin;
    if (fname != NULL)
        printf("============== %s ============\n",
                str2upper((char *)fname));
    while (fgetln(s, MAXLINE, fp) > 0) {
        if (++lineno > pagesize) {
            lineno = 1;
            printf("================= %d ===============\n", pageno++);
        }
        fputln(s, stdout);
    }
    fclose(fp);
}

int main(int argc, char *argv[])
{
    while (--argc && **++argv == '-') {
        while (*++*argv)
            switch (**argv) {
                case 'n':
                    pagesize = atoi(*++argv);
                    argc--;
                    goto next;
                default:
                    fprintf(stderr, "pcat: unknown option: %c\n",
                              **argv);
                    exit(2);
            }
    next:    ;
    }
    if (!argc)
        cat(NULL);
    else
        while (argc--)
            cat(*argv++);
    return 0;
}
```

Exercise 7-9: Functions like `isupper` can be implemented to save space or to save time. Explore both possibilities.

```
#include <stdio.h>

int toupper1(int c)
{
    if (c >= 'a' && c <= 'z')
```

```
6           c -= ('a' - 'A');
7       return c;
8   }
9
10  static char arr[256];    /* ascii */
11
12  int init_toupper2(void)
13  {
14      register int i;
15
16      for (i = 0; i < 97; i++)
17          arr[i] = i;
18      for (i = 97; i < 122; i++)
19          arr[i] = i - ('a' - 'A');
20      for (i = 123; i < 256; i++)
21          arr[i] = i;
22  }
23
24  int toupper2(int c)
25  {
26      return arr[c];
27  }
28
29  static int (*fun)(int) = toupper1;
30
31  void pr(FILE *fp)
32  {
33      register int c;
34
35      while ((c = getc(fp)) != EOF)
36          putc((*fun)(c), stdout);
37  }
38
39  int main(int argc, char *argv[])
40  {
41      FILE *fp;
42
43      if (argc > 1 && **(argv + 1) == '-') {
44          argv++;
45          fun = toupper2;
46          argc--;
47          init_toupper2();
48      }
49
50      if (argc == 1)
51          pr(stdin);
52      else {
53          while (--argc && *++argv) {
54              fp = fopen(*argv, "r");
55              if (fp == NULL) {
```

```
56                  fprintf(stderr, "error fopen() %s\n", *argv);
57                  continue;
58              }
59              pr(fp);
60              fclose(fp);
61          }
62      }
63
64      return 0;
65  }
66
67
```

Chapter 8

The UNIX System Interface

Exercise 8-1: Rewrite the program `cat` from Chapter 7 using `read`, `write`, `open` and `close` instead of their standart library equivalents. Perform experiments to determine the relative speeds of the two versions.

It's interesting and useful to compare 3 versions of `cat`:

1. C library version with reading the hole lines with `fgets` or its analog;
2. C library version with reading file per byte using `getc` or its analog;
3. UNIX system calls version using `read` and `write`.

The sources of all versions are provided below.
Version 1:

```
/*
 * cat based on the libc, version
 * with reading hole lines (meaningless, but to compare
 * with byte reader)
 */

#include <stdio.h>
#include <stdlib.h>

/* fgets/fputs */
#define MAXLINE        1000

int fgetln(char *s, int max, FILE *fp)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getc(fp)) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';
```

```

    return s - b;
}

int fputs(const char *s, FILE *fp)
{
    while (*s)
        putc(*s++, fp);
    return ferror(fp);
}

/* main work */
int main(int argc, char *argv[])
{
    FILE *fp;
    extern void cat(FILE *fp);

    if (argc == 1)
        cat(stdin);
    else
        while (--argc && *++argv) {
            fp = fopen(*argv, "r");
            if (fp == NULL) {
                fprintf(stderr, "error opening %s\n", *argv);
                exit(1);
            }
            cat(fp);
            fclose(fp);
        }
    exit(0);
}

void cat(FILE *fp)
{
    char s[MAXLINE];

    while (fgets(s, MAXLINE, fp) > 0)
        if (fputs(s, stdout)) {
            fprintf(stderr, "error while writing\n");
            return;
        }
}
```

Version 2:

```
/*
 * cat -- libc version with reading for bytes
 */
#include <stdio.h>
#include <stdlib.h>

```

```
void cat(FILE *fp)
{
    int c;

    while ((c = getc(fp)) != EOF)
        putc(c, stdout);
}

int main(int argc, char *argv[])
{
    FILE *fp;

    if (argc == 1)
        cat(stdin);
    else
        while (--argc && *++argv) {
            fp = fopen(*argv, "r");
            if (fp == NULL) {
                fprintf(stderr, "error open %s\n", *argv);
                exit(1);
            }
            cat(fp);
            fclose(fp);
        }
    exit(0);
}
```

Version 3:

```
/*
 * cat -- UNIX syscalls version
 */
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>

#define STDIN        0
#define STDOUT        1
#define STDERR        2

#define BUFSIZ        1024    /* should be divisor of FS block size */

void cat(int fd)
{
    char buf[BUFSIZ];
    int bc;

    while ((bc = read(fd, buf, BUFSIZ)) > 0)
        if (write(STDOUT, buf, bc) != bc) {
            write(STDERR, "cat err\n", 8);
```

```
            exit(1);
        }
}

int main(int argc, char *argv[])
{
    int fd;

    if (argc == 1)
        cat(STDIN);
    else
        while (--argc && *++argv) {
            fd = open(*argv, O_RDONLY, 0);
            if (fd == -1) {
                write(STDERR, "error opening ", 13);
                write(STDERR, *argv, strlen(*argv));
                exit(2);
            }
            cat(fd);
            close(fd);
        }
    exit(0);
}


```

Results for 15 million line file for one launch are as follows:

	fgets/fputs	getc/putc	read/write
real	0m4.959s	0m7.009s	0m0.157s
user	0m4.884s	0m6.933s	0m0.019s
sys	0m0.075s	0m0.075s	0m0.137s

It's quite clear, that UNIX system call version is much faster than any of the GNU C library.

Exercise 8-2: Rewrite `fopen` and `_fillbuf` with fields instead of explicit bit operations. Compare code size and execution speed.

```
/*
 * stdio.c implementation on UNIX syscalls
 */

#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>

#define OPEN_MAX    20
#define EOF        (-1)
#undef NULL        /* stdlib.h */
#define NULL        ((void *) 0)
#define BUFSIZ        (1 << 12)    /* better to be the divisor
```

```
                        of FS block */

struct flag {
    unsigned rd : 1;    /* read support */
    unsigned wr : 1;    /* write support */
    unsigned unbuf : 1;    /* buffersize = 1 */
    unsigned eof : 1;    /* end of file */
    unsigned err : 1;    /* error */
    unsigned : 0;        /* alignment by the next machine word */
};

typedef struct {
    /* buffer itself */
    char *buf;    /* base */
    int bp;        /* next char to read/write */
    int cnt;    /* cnt to read/write */

    int fd;        /* file descriptor */
    struct flag flag;    /* flags */
    int used;    /* is in use */
} FILE;

static FILE _fbuf[OPEN_MAX] = {
    { NULL, 0, 0, 0, {1, 0, 0, 0, 0}, 1 },    /* stdin */
    { NULL, 0, 0, 1, {0, 1, 0, 0, 0}, 1 },    /* stdout */
    { NULL, 0, 0, 2, {0, 1, 1, 0, 0}, 1 } /* stderr */
};

#define stdin        (&_fbuf[0])
#define stdout        (&_fbuf[1])
#define stderr        (&_fbuf[2])

/*
 * functions from libc (+ == done):
 *
 * open/close:
 + fopen -- FILE *fopen(const char *name, const char *mode);
 + fclose -- int fclose(FILE *fp);
 *
 * i/o:
 + getc -- int getc(FILE *fp);
 + putc -- int putc(int c, FILE *fp);
 + fgetc -- int fgetc(FILE *fp);
 + fputc -- int fputc(int c, FILE *fp);
 + getchar -- int getchar(void);
 + putchar -- int putchar(int c);
 + fgets -- char *fgets(char *s, int max, FILE *fp);
 + fputs -- int fputs(const char *s, FILE *fp);
 *
 * check:
```

```
 + feof -- int feof(FILE *fp);
 + ferror -- int ferror(FILE *fp);
 + fileno -- int fileno(FILE *fp);
 *
 * other:
 + fseek -- int fseek(FILE *fp, long off, int base);
 */

int feof(FILE *fp)
{
    return fp->flag.eof;
}

int ferror(FILE *fp)
{
    return fp->flag.err;
}

int fileno(FILE *fp)
{
    return fp->fd;
}

/*
 * supports following modes:
 * - "r" for read;
 * - "w" for write;
 * - "a" for append;
 * "b" modes are crap that exist on non-UNIX systems for binary
 * hint: it doesn't read the buffer, since file may never be read, only opened
 * for some reasons
 */
FILE *fopen(const char *name, const char *mode)
{
    register FILE *fp;
    int fd;
    struct flag tmp = { 0, 0, 0, 0, 0 };

    /* loop across table */
    for (fp = _fbuf; fp < &_fbuf[OPEN_MAX]; fp++)
        if (!fp->used)
            goto found;
    return NULL;        /* not found */

found:
    /* determine mode */
    switch (*mode) {
        case 'a':        /* append */
        case 'w':        /* replace */
            fd = open(name, O_WRONLY);
```

```
114             fp->flag.wr = 1;
115             break;
116         case 'r':        /* read */
117             fd = open(name, O_RDONLY);
118             fp->flag.rd = 1;
119             break;
120         default:        /* unknown option, some crap */
121             return NULL;
122     }
123
124     /* check correct open */
125     if (fd < 0) {
126         fp->flag = tmp;    /* don't need to set _ERR, it's not others fp */
127         return NULL;
128     }
129
130     /* lseek if append */
131     if (*mode == 'a')
132         lseek(fd, 0L, 2);
133
134     /* ok, now just finish */
135     fp->fd = fd;
136     fp->used = 1;    /* mark in use */
137     fp->buf = NULL;    /* no buffer yet, see description */
138     fp->bp = fp->cnt = 0;
139
140     return fp;
141 }
142
143 int _fillbuf(FILE *fp)
144 {
145     int bufsize;
146
147     if (!fp->flag.rd) {    /* check rights */
148         fp->flag.err = 1;
149         return EOF;
150     }
151
152     /* determine buffer size */
153     bufsize = fp->flag.unbuf ? 1 : BUFSIZ;
154
155     if (fp->buf == NULL) {    /* need alloc */
156         fp->buf = malloc(bufsize);
157         if (fp->buf == NULL) {
158             fp->flag.err = 1;
159             return EOF;
160         }
161     }
162
163     /* read buf */
```

```
164     if ((fp->cnt = read(fp->fd, fp->buf, bufsize)) < 0) {    /* err */
165         fp->flag.err = 1;
166         return EOF;
167     } else if (fp->cnt == 0) {        /* file finished */
168         fp->flag.eof = 1;
169         return EOF;
170     }
171
172     /* normal case */
173     fp->bp = 1;
174     fp->cnt--;
175
176     return fp->buf[0];
177 }
178
179 int getc(FILE *fp)    /* avoid using macro for ++ cases */
180 {
181     return fp->cnt-- ? fp->buf[fp->bp++] : _fillbuf(fp);
182 }
183
184 int fgetc(FILE *fp)    /* same shit */
185 {
186     return getc(fp);
187 }
188
189 int putc(int c, FILE *fp)
190 {
191     extern void _flushbuf(FILE *);
192
193     if (fp->cnt <= 0)
194         _flushbuf(fp);
195     fp->cnt--;
196
197     return fp->buf[fp->bp++] = c;
198 }
199
200 /* flush buffer to disk, write and return new symbol */
201 void _flushbuf(FILE *fp)
202 {
203     int bufsize;
204
205     if (!fp->flag.wr) {    /* check rights */
206         fp->flag.err = 1;
207         return;
208     }
209
210     /* determine size */
211     bufsize = fp->flag.unbuf ? 1 : BUFSIZ;
212
213     /* alloc buf if needed */
```

```
    if (fp->buf == NULL) {
        fp->buf = malloc(bufsize);
        if (fp->buf == NULL) {
            fp->flag.err = 1;
            return;
        }
    }

    /* write to disk */
    if (write(fp->fd, fp->buf, fp->bp) < 0) {
        fp->flag.err = 1;
        return;
    }

    /* update state */
    fp->cnt = bufsize;
    fp->bp = 0;
}

int fputc(int c, FILE *fp)
{
    return putc(c, fp);
}

int getchar(void)
{
    return getc(stdin);
}

int putchar(int c)
{
    return putc(c, stdout);
}

int fclose(FILE *fp)
{
    struct flag tmp = { 0, 0, 0, 0, 0};

    if (fp->flag.wr)    /* libc */
        _flushbuf(fp);
    fp->flag = tmp;
    fp->cnt = fp->bp = 0;
    free(fp->buf);
    fp->buf = NULL;
    fp->used = 0;
    return close(fp->fd);        /* unix */
}

int fflush(FILE *fp)
{
```

```
    if (!fp->flag.wr)
        return EOF;
    _flushbuf(fp);
}

/* gets/puts */

#define MAXLINE        100

int fgets(char *s, int max, FILE *fp)
{
    register char *b;
    register int c;

    for (b = s; (s - b) < (max - 1) &&
            (c = getc(fp)) != EOF; *s++ = c)
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

int fputs(const char *s, FILE *fp)
{
    for (; *s; s++) {
        putc(*s, fp);
        if (*s == '\n')
            fflush(fp);
    }

    return 0;
}

/* cat part to test */

void cat(FILE *fp)
{
    char buf[MAXLINE];

    while (fgets(buf, MAXLINE, fp) > 0)
        fputs(buf, stdout);
}

/* fseek */
int fseek(FILE *fp, long off, int base)
{
    if (fp->flag.wr)
```

```
        fflush(fp);
    lseek(fp->fd, off, base);

    return 0;
}

int main(int argc, char *argv[])
{
    FILE *fp;
    char *err = "can not open ";

    if (argc == 1)
        cat(stdin);
    else {
        while (--argc && *++argv) {
            fp = fopen(*argv, "r");
            if (fp == NULL) {
                fputs(err, stderr);
                fputs(*argv, stderr);
                putc('\n', stderr);
                putc('\n', stderr);
                continue;
            }
            cat(fp);
            fclose(fp);
        }
    }
    return 0;

}
```

Exercise 8-3: Design and write `_flushbuf`, `fflush`, and `fclose`.

```
/*
 * stdio.c implementation on UNIX syscalls
 */

#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>

#define OPEN_MAX    20
#define EOF        (-1)
#undef NULL        /* stdlib.h */
#define NULL        ((void *) 0)
#define BUFSIZ        1024        /* better to be the divisor
                       of FS block */

typedef struct {
    /* buffer itself */
    char *buf;    /* base */
```

```
    int bp;        /* next char to read/write */
    int cnt;    /* cnt to read/write */

    int fd;        /* file descriptor */
    int flag;    /* flags */
    int used;    /* is in use */
} FILE;

enum flags {
    _RD = (1 << 0),        /* read support */
    _WR = (1 << 1),        /* write support */
    _UNBUF = (1 << 2),    /* no bufferization */
    _EOF = (1 << 3),    /* end of file */
    _ERR = (1 << 4)        /* error */
};

static FILE _fbuf[OPEN_MAX] = {
    { NULL, 0, 0, 0, _RD, 1 },    /* stdin */
    { NULL, 0, 0, 1, _WR, 1 },    /* stdout */
    { NULL, 0, 0, 2, _WR | _UNBUF, 1 } /* stderr */
};

#define stdin        (&_fbuf[0])
#define stdout        (&_fbuf[1])
#define stderr        (&_fbuf[2])

/*
 * functions from libc (+ == done):
 *
 * open/close:
 + fopen -- FILE *fopen(const char *name, const char *mode);
 + fclose -- int fclose(FILE *fp);
 *
 * i/o:
 + getc -- int getc(FILE *fp);
 + putc -- int putc(int c, FILE *fp);
 + fgetc -- int fgetc(FILE *fp);
 + fputc -- int fputc(int c, FILE *fp);
 + getchar -- int getchar(void);
 + putchar -- int putchar(int c);
 + fgets -- char *fgets(char *s, int max, FILE *fp);
 + fputs -- int fputs(const char *s, FILE *fp);
 *
 * check:
 + feof -- int feof(FILE *fp);
 + ferror -- int ferror(FILE *fp);
 + fileno -- int fileno(FILE *fp);
 *
 * other:
 + fseek -- int fseek(FILE *fp, long off, int base);
```

```
 */

int feof(FILE *fp)
{
    return fp->flag & _EOF;
}

int ferror(FILE *fp)
{
    return fp->flag & _ERR;
}

int fileno(FILE *fp)
{
    return fp->fd;
}

/*
 * supports following modes:
 * - "r" for read;
 * - "w" for write;
 * - "a" for append;
 * "b" modes are crap that exist on non-UNIX systems for binary
 * hint: it doesn't read the buffer, since file may never be read, only opened
 * for some reasons
 */
FILE *fopen(const char *name, const char *mode)
{
    register FILE *fp;
    int fd;

    /* loop across table */
    for (fp = _fbuf; fp < &_fbuf[OPEN_MAX]; fp++)
        if (!fp->used)
            goto found;
    return NULL;        /* not found */

found:
    /* determine mode */
    switch (*mode) {
        case 'a':        /* append */
        case 'w':        /* replace */
            fd = open(name, O_WRONLY);
            fp->flag |= _WR;
            break;
        case 'r':        /* read */
            fd = open(name, O_RDONLY);
            fp->flag |= _RD;
            break;
        default:        /* unknown option, some crap */
```

```
119             return NULL;
120     }
121
122     /* check correct open */
123     if (fd < 0) {
124         fp->flag = 0;    /* don't need to set _ERR, it's not others fp */
125         return NULL;
126     }
127
128     /* lseek if append */
129     if (*mode == 'a')
130         lseek(fd, 0L, 2);
131
132     /* ok, now just finish */
133     fp->fd = fd;
134     fp->used = 1;    /* mark in use */
135     fp->buf = NULL;    /* no buffer yet, see description */
136     fp->bp = fp->cnt = 0;
137
138     return fp;
139 }
140
141 int _fillbuf(FILE *fp)
142 {
143     int bufsize;
144
145     if (!(fp->flag & _RD)) {    /* check rights */
146         fp->flag |= _ERR;
147         return EOF;
148     }
149
150     /* determine buffer size */
151     bufsize = (fp->flag & _UNBUF) ? 1 : BUFSIZ;
152
153     if (fp->buf == NULL) {    /* need alloc */
154         fp->buf = malloc(bufsize);
155         if (fp->buf == NULL) {
156             fp->flag |= _ERR;
157             return EOF;
158         }
159     }
160
161     /* read buf */
162     if ((fp->cnt = read(fp->fd, fp->buf, bufsize)) < 0) {    /* err */
163         fp->flag |= _ERR;
164         return EOF;
165     } else if (fp->cnt == 0) {        /* file finished */
166         fp->flag |= _EOF;
167         return EOF;
168     }
```

```
    /* normal case */
    fp->bp = 1;
    fp->cnt--;

    return fp->buf[0];
}

int getc(FILE *fp)    /* avoid using macro for ++ cases */
{
    return fp->cnt-- ? fp->buf[fp->bp++] : _fillbuf(fp);
}

int fgetc(FILE *fp)    /* same shit */
{
    return getc(fp);
}

int putc(int c, FILE *fp)
{
    extern void _flushbuf(FILE *);

    if (fp->cnt <= 0)
        _flushbuf(fp);
    fp->cnt--;

    return fp->buf[fp->bp++] = c;
}

/* flush buffer to disk, write and return new symbol */
void _flushbuf(FILE *fp)
{
    int bufsize;

    if (!(fp->flag & _WR)) {    /* check rights */
        fp->flag |= _ERR;
        return;
    }

    /* determine size */
    bufsize = (fp->flag & _UNBUF) ? 1 : BUFSIZ;

    /* alloc buf if needed */
    if (fp->buf == NULL) {
        fp->buf = malloc(bufsize);
        if (fp->buf == NULL) {
            fp->flag |= _ERR;
            return;
        }
    }
```

```
219
220     /* write to disk */
221     if (write(fp->fd, fp->buf, fp->bp) < 0) {
222         fp->flag |= _ERR;
223         return;
224     }
225
226     /* update state */
227     fp->cnt = bufsize;
228     fp->bp = 0;
229 }
230
231 int fputc(int c, FILE *fp)
232 {
233     return putc(c, fp);
234 }
235
236 int getchar(void)
237 {
238     return getc(stdin);
239 }
240
241 int putchar(int c)
242 {
243     return putc(c, stdout);
244 }
245
246 int fclose(FILE *fp)
247 {
248     if (fp->flag & _WR)    /* libc */
249         _flushbuf(fp);
250     fp->flag = fp->cnt = fp->bp = 0;
251     free(fp->buf);
252     fp->buf = NULL;
253     fp->used = 0;
254     return close(fp->fd);        /* unix */
255 }
256
257 int fflush(FILE *fp)
258 {
259     if (!(fp->flag & _WR))
260         return EOF;
261     _flushbuf(fp);
262 }
263
264 /* gets/puts */
265
266 #define MAXLINE        100
267
268 int fgets(char *s, int max, FILE *fp)
```

```
269 {
270     register char *b;
271     register int c;
272
273     for (b = s; (s - b) < (max - 1) &&
274             (c = getc(fp)) != EOF; *s++ = c)
275         if (c == '\n') {
276             *s++ = c;
277             break;
278         }
279     *s = '\0';
280
281     return s - b;
282 }
283
284 int fputs(const char *s, FILE *fp)
285 {
286     for (; *s; s++) {
287         putc(*s, fp);
288         if (*s == '\n')
289             fflush(fp);
290     }
291
292     return 0;
293 }
294
295 /* cat part to test */
296
297 void cat(FILE *fp)
298 {
299     char buf[MAXLINE];
300
301     while (fgets(buf, MAXLINE, fp) > 0)
302         fputs(buf, stdout);
303 }
304
305 /* fseek */
306 int fseek(FILE *fp, long off, int base)
307 {
308     if (fp->flag & _WR)
309         fflush(fp);
310     lseek(fp->fd, off, base);
311
312     return 0;
313 }
314
315 int main(int argc, char *argv[])
316 {
317     FILE *fp;
318     char *err = "can not open ";
```

```

    if (argc == 1)
        cat(stdin);
    else {
        while (--argc && *++argv) {
            fp = fopen(*argv, "r");
            if (fp == NULL) {
                fputs(err, stderr);
                fputs(*argv, stderr);
                putc('\n', stderr);
                putc('\n', stderr);
                continue;
            }
            /* + check */
            cat(fp);
            fclose(fp);
        }
    }
    return 0;

}
```

Exercise 8-4: The standart library function `int fseek(FILE *fp, long offset, int origin)` is identical to `lseek` except that `fp` is a file pointer instead of a file descriptor and the return value is an `int` status, not a position. Write `fseek`. Make sure that your `fseek` coordinates properly with the buffering done for the other functions of the library.

```
/*
 * stdio.c implementation on UNIX syscalls
 */

#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>

#define OPEN_MAX    20
#define EOF        (-1)
#undef NULL        /* stdlib.h */
#define NULL        ((void *) 0)
#define BUFSIZ        1024        /* better to be the divisor
                       of FS block */

typedef struct {
    /* buffer itself */
    char *buf;    /* base */
    int bp;        /* next char to read/write */
    int cnt;    /* cnt to read/write */

    int fd;        /* file descriptor */
    int flag;    /* flags */
    int used;    /* is in use */
```

```
25  } FILE;
26
27  enum flags {
28      _RD = (1 << 0),        /* read support */
29      _WR = (1 << 1),        /* write support */
30      _UNBUF = (1 << 2),    /* no bufferization */
31      _EOF = (1 << 3),    /* end of file */
32      _ERR = (1 << 4)        /* error */
33  };
34
35  static FILE _fbuf[OPEN_MAX] = {
36      { NULL, 0, 0, 0, _RD, 1 },    /* stdin */
37      { NULL, 0, 0, 1, _WR, 1 },    /* stdout */
38      { NULL, 0, 0, 2, _WR | _UNBUF, 1 } /* stderr */
39  };
40
41  #define stdin        (&_fbuf[0])
42  #define stdout        (&_fbuf[1])
43  #define stderr        (&_fbuf[2])
44
45  /*
46   * functions from libc (+ == done):
47   *
48   * open/close:
49   + fopen -- FILE *fopen(const char *name, const char *mode);
50   + fclose -- int fclose(FILE *fp);
51   *
52   * i/o:
53   + getc -- int getc(FILE *fp);
54   + putc -- int putc(int c, FILE *fp);
55   + fgetc -- int fgetc(FILE *fp);
56   + fputc -- int fputc(int c, FILE *fp);
57   + getchar -- int getchar(void);
58   + putchar -- int putchar(int c);
59   + fgets -- char *fgets(char *s, int max, FILE *fp);
60   + fputs -- int fputs(const char *s, FILE *fp);
61   *
62   * check:
63   + feof -- int feof(FILE *fp);
64   + ferror -- int ferror(FILE *fp);
65   + fileno -- int fileno(FILE *fp);
66   *
67   * other:
68   + fseek -- int fseek(FILE *fp, long off, int base);
69   */
70
71  int feof(FILE *fp)
72  {
73      return fp->flag & _EOF;
74  }
```

```
75
76  int ferror(FILE *fp)
77  {
78      return fp->flag & _ERR;
79  }
80
81  int fileno(FILE *fp)
82  {
83      return fp->fd;
84  }
85
86  /*
87   * supports following modes:
88   * - "r" for read;
89   * - "w" for write;
90   * - "a" for append;
91   * "b" modes are crap that exist on non-UNIX systems for binary
92   * hint: it doesn't read the buffer, since file may never be read, only opened
93   * for some reasons
94   */
95  FILE *fopen(const char *name, const char *mode)
96  {
97      register FILE *fp;
98      int fd;
99
100     /* loop across table */
101     for (fp = _fbuf; fp < &_fbuf[OPEN_MAX]; fp++)
102         if (!fp->used)
103             goto found;
104     return NULL;        /* not found */
105
106 found:
107     /* determine mode */
108     switch (*mode) {
109         case 'a':        /* append */
110         case 'w':        /* replace */
111             fd = open(name, O_WRONLY);
112             fp->flag |= _WR;
113             break;
114         case 'r':        /* read */
115             fd = open(name, O_RDONLY);
116             fp->flag |= _RD;
117             break;
118         default:        /* unknown option, some crap */
119             return NULL;
120     }
121
122     /* check correct open */
123     if (fd < 0) {
124         fp->flag = 0;    /* don't need to set _ERR, it's not others fp */
```

```
        return NULL;
    }

    /* lseek if append */
    if (*mode == 'a')
        lseek(fd, 0L, 2);

    /* ok, now just finish */
    fp->fd = fd;
    fp->used = 1;    /* mark in use */
    fp->buf = NULL;    /* no buffer yet, see description */
    fp->bp = fp->cnt = 0;

    return fp;
}

int _fillbuf(FILE *fp)
{
    int bufsize;

    if (!(fp->flag & _RD)) {    /* check rights */
        fp->flag |= _ERR;
        return EOF;
    }

    /* determine buffer size */
    bufsize = (fp->flag & _UNBUF) ? 1 : BUFSIZ;

    if (fp->buf == NULL) {    /* need alloc */
        fp->buf = malloc(bufsize);
        if (fp->buf == NULL) {
            fp->flag |= _ERR;
            return EOF;
        }
    }

    /* read buf */
    if ((fp->cnt = read(fp->fd, fp->buf, bufsize)) < 0) {    /* err */
        fp->flag |= _ERR;
        return EOF;
    } else if (fp->cnt == 0) {        /* file finished */
        fp->flag |= _EOF;
        return EOF;
    }

    /* normal case */
    fp->bp = 1;
    fp->cnt--;

    return fp->buf[0];
```

```
175  }
176
177  int getc(FILE *fp)    /* avoid using macro for ++ cases */
178  {
179      return fp->cnt-- ? fp->buf[fp->bp++] : _fillbuf(fp);
180  }
181
182  int fgetc(FILE *fp)    /* same shit */
183  {
184      return getc(fp);
185  }
186
187  int putc(int c, FILE *fp)
188  {
189      extern void _flushbuf(FILE *);
190
191      if (fp->cnt <= 0)
192          _flushbuf(fp);
193      fp->cnt--;
194
195      return fp->buf[fp->bp++] = c;
196  }
197
198  /* flush buffer to disk, write and return new symbol */
199  void _flushbuf(FILE *fp)
200  {
201      int bufsize;
202
203      if (!(fp->flag & _WR)) {    /* check rights */
204          fp->flag |= _ERR;
205          return;
206      }
207
208      /* determine size */
209      bufsize = (fp->flag & _UNBUF) ? 1 : BUFSIZ;
210
211      /* alloc buf if needed */
212      if (fp->buf == NULL) {
213          fp->buf = malloc(bufsize);
214          if (fp->buf == NULL) {
215              fp->flag |= _ERR;
216              return;
217          }
218      }
219
220      /* write to disk */
221      if (write(fp->fd, fp->buf, fp->bp) < 0) {
222          fp->flag |= _ERR;
223          return;
224      }
```

```
225
226         /* update state */
227         fp->cnt = bufsize;
228         fp->bp = 0;
229     }
230
231     int fputc(int c, FILE *fp)
232     {
233         return putc(c, fp);
234     }
235
236     int getchar(void)
237     {
238         return getc(stdin);
239     }
240
241     int putchar(int c)
242     {
243         return putc(c, stdout);
244     }
245
246     int fclose(FILE *fp)
247     {
248         if (fp->flag & _WR)    /* libc */
249             _flushbuf(fp);
250         fp->flag = fp->cnt = fp->bp = 0;
251         free(fp->buf);
252         fp->buf = NULL;
253         fp->used = 0;
254         return close(fp->fd);        /* unix */
255     }
256
257     int fflush(FILE *fp)
258     {
259         if (!(fp->flag & _WR))
260             return EOF;
261         _flushbuf(fp);
262     }
263
264     /* gets/puts */
265
266     #define MAXLINE        100
267
268     int fgets(char *s, int max, FILE *fp)
269     {
270         register char *b;
271         register int c;
272
273         for (b = s; (s - b) < (max - 1) &&
274                 (c = getc(fp)) != EOF; *s++ = c)
```

```
        if (c == '\n') {
            *s++ = c;
            break;
        }
    *s = '\0';

    return s - b;
}

int fputs(const char *s, FILE *fp)
{
    for (; *s; s++) {
        putc(*s, fp);
        if (*s == '\n')
            fflush(fp);
    }

    return 0;
}

/* cat part to test */

void cat(FILE *fp)
{
    char buf[MAXLINE];

    while (fgets(buf, MAXLINE, fp) > 0)
        fputs(buf, stdout);
}

/* fseek */
int fseek(FILE *fp, long off, int base)
{
    if (fp->flag & _WR)
        fflush(fp);
    lseek(fp->fd, off, base);

    return 0;
}

int main(int argc, char *argv[])
{
    FILE *fp;
    char *err = "can not open ";

    if (argc == 1)
        cat(stdin);
    else {
        while (--argc && *++argv) {
            fp = fopen(*argv, "w");
```

```
            if (fp == NULL) {
                fputs(err, stderr);
                fputs(*argv, stderr);
                putc('\n', stderr);
                putc('\n', stderr);
                continue;
            }
            fseek(fp, -20L, 2);
            fputs(err, fp);
            fclose(fp);
        }
    }
    return 0;

}
```

Exercise 8-5: Modify the `fsize` program to print the other information contained in the inode entry.

```
/*
 * directory walking and printing:
 - interface is system-independent;
 - real fundament is based on directory design in each system:
     - | ind_num | file_name | in Research UNIX/System V
    - | ind_num | name_len | rec_len | name | .... | in BSD
    - others for Linux file systems

 * normally there is a dirent.h file under UNIX system according to POSIX (Windows?),
 * this is its implementation (as with stdio.h for libc before)
 */

/* libc */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* unix */
#include <fcntl.h>    /* O_RDONLY, O_WRONLY, O_RDWR */
#include <sys/types.h>    /* DIR type in inode */
#include <unistd.h>    /* read/write */
#include <dirent.h>    /* DIR, Dirent, opendir, closedir, readdir */

typedef struct dirent Dirent;

int main(int argc, char **argv)
{
    extern void fsize(const char *);

    if (argc == 1)
        fsize(".");
    else
```

```
        while (--argc > 0)
            fsize(*++argv);
    return 0;
}

/* 1. stat;
 * 2. if DIR -> dirwalk
 * 3. any case -> print size (dir also)
 */
void fsize(const char *name)
{
    extern void dirwalk(const char *name, void (*fcn)(const char *));
    struct stat stbuf, *st;

    if (stat(name, &stbuf) == -1) {    /* error stat */
        fprintf(stderr, "fsize: cann't access %s\n", name);
        return;
    }
    st = &stbuf;
    if ((st->st_mode & S_IFMT) == S_IFDIR)
        dirwalk(name, fsize);
    printf("%ld %ld %o %d %d %ld %ld %s\n",
            st->st_ino, st->st_blksize, st->st_mode, st->st_uid,
            st->st_gid, st->st_nlink, st->st_size, name);
}

#define MAX_PATH    1024

void dirwalk(const char *dir, void (*fcn)(const char *))
{
    char name[MAX_PATH];
    Dirent *dp;
    DIR *dfd;

    if ((dfd = opendir(dir)) == NULL) {
        fprintf(stderr, "dirwalk: cann't open %s\n", dir);
        return;
    }
    while ((dp = readdir(dfd)) != NULL) {
        if (strcmp(dp->d_name, ".") == 0 ||
            strcmp(dp->d_name, "..") == 0)
            continue;
        if (strlen(dir) + strlen(dp->d_name) + 2 > sizeof(name))
            fprintf(stderr, "dirwalk: name %s %s too long\n",
                    dir, dp->d_name);
        else {
            sprintf(name, "%s/%s", dir, dp->d_name);
            (*fcn)(name);
        }
    }
```

```
    closedir(dfd);
}
```

Exercise 8-6: The standart library function `calloc(n,size)` returns a pointer to `n` objects of size `size`, with the storage initialized to zero. Write `calloc`, by calling `malloc` or by modifying it.

```
/*
 * malloc, from K&R;
 * calloc added
 */

#include <unistd.h>
#include <stdio.h>

typedef long Align;

union header {
    struct {
        union header *ptr;
        unsigned size;
    } s;
    Align x;
};

typedef union header Header;

static Header base;
static Header *freep = NULL;
static Header *morecore(unsigned);

void *_malloc(unsigned bc)
{
    Header *p, *prevp;
    unsigned nunits;

    nunits = (bc + sizeof(Header) - 1) / sizeof(Header) + 1;
    if ((prevp = freep) == NULL) {    /* no list yet */
        base.s.ptr = freep = prevp = &base;
        base.s.size = 0;
    }
    for (p = prevp->s.ptr; ; prevp = p, p = p->s.ptr) {
        if (p->s.size >= nunits) {    /* enough space */
            if (p->s.size == nunits)    /* exactly */
                prevp->s.ptr = p->s.ptr;
            else {
                p->s.size -= nunits;
                p += p->s.size;
                p->s.size = nunits;
            }
            freep = prevp;
            return (void *)(p + 1);
```

```
        }
        if (p == freep)        /* empty list */
            if ((p = morecore(nunits)) == NULL)
                return NULL;
    }
}

#define NALLOC        1024

static Header *morecore(unsigned nu)
{
    void *cp;
    void *sbrk(intptr_t);
    void _free(void *);
    Header *up;

    if (nu < NALLOC)
        nu = NALLOC;
    cp = sbrk(nu * sizeof(Header));
    if (cp == (void *)-1)
        return NULL;
    up = (Header *) cp;
    up->s.size = nu;
    _free((void *)(up + 1));

    return freep;
}

void _free(void *ap)
{
    Header *bp, *p;

    bp = (Header *)ap - 1;        /* find a future place in a free list */
    for (p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
        if (p >= p->s.ptr && (bp > p || bp < p->s.ptr))
            break;

    if (bp + bp->s.size == p->s.ptr) {    /* merge with higher neighbour */
        bp->s.size += p->s.ptr->s.size;
        bp->s.ptr = p->s.ptr->s.ptr;
    } else
        bp->s.ptr = p->s.ptr;

    if (p + p->s.size == bp) {        /* merge with lower neighbour */
        p->s.size += bp->s.size;
        p->s.ptr = bp->s.ptr;
    } else
        p->s.ptr = bp;
    freep = p;
}
```

```

void mmset(void *mm, int c, unsigned bc)
{
    register char *m;

    for (m = mm; bc--; m++)
        *m = c;
}

void *_calloc(unsigned n, unsigned sz)
{
    void *mm;
    unsigned bc = n * sz;

    mm = _malloc(bc);
    if (mm != NULL)
        mmset(mm, 0, bc);
    return mm;
}

#define NARR        1000

int main(void)
{
    int *arr, i;

    arr = _calloc(NARR, sizeof(int));
    for (i = 0; i < NARR; i++) {
        printf("%d ", arr[i]);
    }
    putchar('\n');

    return 0;
}
```

Exercise 8-7: `malloc` accepts a size request without checking its plausibility; `free` believes that the block it is asked to free contains a valid size field. Improve there routines so they take more pains with error checking.

```
/*
 * malloc, from K&R;
 * calloc and checks added
 */

#include <unistd.h>
#include <stdio.h>

typedef long Align;

union header {
    struct {
```

```
13          union header *ptr;
14          unsigned size;
15      } s;
16      Align x;
17  };
18
19  typedef union header Header;
20
21  static Header base;
22  static Header *freep = NULL;
23  static Header *morecore(unsigned);
24
25  void *_malloc(unsigned bc)
26  {
27      Header *p, *prevp;
28      unsigned nunits;
29
30      if (bc <= 0)
31          return NULL;
32      nunits = (bc + sizeof(Header) - 1) / sizeof(Header) + 1;
33      if ((prevp = freep) == NULL) {    /* no list yet */
34          base.s.ptr = freep = prevp = &base;
35          base.s.size = 0;
36      }
37      for (p = prevp->s.ptr; ; prevp = p, p = p->s.ptr) {
38          if (p->s.size >= nunits) {    /* enough space */
39              if (p->s.size == nunits)    /* exactly */
40                  prevp->s.ptr = p->s.ptr;
41              else {
42                  p->s.size -= nunits;
43                  p += p->s.size;
44                  p->s.size = nunits;
45              }
46              freep = prevp;
47              return (void *)(p + 1);
48          }
49          if (p == freep)        /* empty list */
50              if ((p = morecore(nunits)) == NULL)
51                  return NULL;
52      }
53  }
54
55  #define NALLOC        1024
56
57  static Header *morecore(unsigned nu)
58  {
59      void *cp;
60      void *sbrk(intptr_t);
61      void _free(void *);
62      Header *up;
```

```
63
64      if (nu < NALLOC)
65          nu = NALLOC;
66      cp = sbrk(nu * sizeof(Header));
67      if (cp == (void *)-1)
68          return NULL;
69      up = (Header *) cp;
70      up->s.size = nu;
71      _free((void *)(up + 1));
72
73      return freep;
74  }
75
76  void _free(void *ap)
77  {
78      Header *bp, *p;
79
80      bp = (Header *)ap - 1;        /* find a future place in a free list */
81      if (bp->s.size <= 0)
82          return;
83      for (p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
84          if (p >= p->s.ptr && (bp > p || bp < p->s.ptr))
85              break;
86
87      if (bp + bp->s.size == p->s.ptr) {    /* merge with higher neighbour */
88          bp->s.size += p->s.ptr->s.size;
89          bp->s.ptr = p->s.ptr->s.ptr;
90      } else
91          bp->s.ptr = p->s.ptr;
92
93      if (p + p->s.size == bp) {        /* merge with lower neighbour */
94          p->s.size += bp->s.size;
95          p->s.ptr = bp->s.ptr;
96      } else
97          p->s.ptr = bp;
98      freep = p;
99  }
100
101 void mmset(void *mm, int c, unsigned bc)
102 {
103     register char *m;
104
105     for (m = mm; bc--; m++)
106         *m = c;
107 }
108
109 void *_calloc(unsigned n, unsigned sz)
110 {
111     void *mm;
112     unsigned bc = n * sz;
```

```

    mm = _malloc(bc);
    if (mm != NULL)
        mmset(mm, 0, bc);
    return mm;
}

#define NARR        1000

int main(void)
{
    int *arr, i;

    arr = _calloc(NARR, sizeof(int));
    for (i = 0; i < NARR; i++) {
        printf("%d ", arr[i]);
    }
    putchar('\n');

    return 0;
}
```

Exercise 8-8: Write a routine `bfree(p,n)` that will free an arbitrary block `p` of `n` characters into the free list maintaned by `malloc` and `free`. By using `bfree`, a user can add a static or external array to the free list at any time.

```
/*
 * malloc, from K&R;
 * calloc, checks and bfree added
 */

#include <unistd.h>
#include <stdio.h>

typedef long Align;

union header {
    struct {
        union header *ptr;
        unsigned size;
    } s;
    Align x;
};

typedef union header Header;

static Header base;
static Header *freep = NULL;
static Header *morecore(unsigned);

void init_malloc(void)
```

```
26  {
27      base.s.ptr = freep  = &base;
28      base.s.size = 0;
29  }
30
31  void *_malloc(unsigned bc)
32  {
33      Header *p, *prevp;
34      unsigned nunits;
35
36      if (bc <= 0)        /* too small */
37          return NULL;
38
39      nunits = (bc + sizeof(Header) - 1) / sizeof(Header) + 1;
40
41      if (freep == NULL)    /* no list yet */
42          init_malloc();
43
44      for (prevp = freep, p = prevp->s.ptr; ; prevp = p, p = p->s.ptr) {
45          if (p->s.size >= nunits) {    /* enough space */
46              if (p->s.size == nunits)    /* exactly */
47                  prevp->s.ptr = p->s.ptr;
48              else {
49                  p->s.size -= nunits;
50                  p += p->s.size;
51                  p->s.size = nunits;
52              }
53              freep = prevp;
54              return (void *)(p + 1);
55          }
56          if (p == freep)        /* empty list */
57              if ((p = morecore(nunits)) == NULL)
58                  return NULL;
59      }
60  }
61
62  #define NALLOC        1024
63
64  static Header *morecore(unsigned nu)
65  {
66      void *cp;
67      void *sbrk(intptr_t);
68      void _free(void *);
69      Header *up;
70
71      if (nu < NALLOC)
72          nu = NALLOC;
73      cp = sbrk(nu * sizeof(Header));
74      if (cp == (void *)-1)
75          return NULL;
```

```
    up = (Header *) cp;
    up->s.size = nu;
    _free((void *)(up + 1));

    return freep;
}

void _free(void *ap)
{
    Header *bp, *p;

    if (freep == NULL)        /* no list yet */
        init_malloc();

    bp = (Header *)ap - 1;        /* find a future place in a free list */
    if (bp->s.size <= 0)        /* too small */
        return;

    for (p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
        if (p >= p->s.ptr && (bp > p || bp < p->s.ptr))
            break;

    if (bp + bp->s.size == p->s.ptr) {    /* merge with higher neighbour */
        bp->s.size += p->s.ptr->s.size;
        bp->s.ptr = p->s.ptr->s.ptr;
    } else
        bp->s.ptr = p->s.ptr;

    if (p + p->s.size == bp) {        /* merge with lower neighbour */
        p->s.size += bp->s.size;
        p->s.ptr = bp->s.ptr;
    } else
        p->s.ptr = bp;
    freep = p;
}

void mmset(void *mm, int c, unsigned bc)
{
    register char *m;

    for (m = mm; bc--; m++)
        *m = c;
}

void *_calloc(unsigned n, unsigned sz)
{
    void *mm;
    unsigned bc = n * sz;

    mm = _malloc(bc);
```

```
    if (mm != NULL)
        mmset(mm, 0, bc);
    return mm;
}

void bfree(void *mm, unsigned bc)
{
    Header *p;

    if (bc < 2 * sizeof(*p))    /* too small */
        return;

    /* imitate header */
    p = (Header *)mm;
    p->s.size = bc / sizeof(*p);

    _free(p + 1);    /* free */
}

static int arr[] = { 12, 545, 6, 12, 23, 4, 56, 6, 14, 56 };

int main(void)
{
    int *p;

    bfree(arr, sizeof(arr));
    p = _malloc(sizeof(*p * 3));
    printf("%d %d %d\n", *p, p[1], p[2]);
    _free(p);
    return 0;
}
```